PAdicMIDI

A Python Toolkit for Hierarchical, Ultrametric, and p-adic Analysis of Symbolic Music Data

---

Reporte Técnico Integral

Versión 1.0.0 — 29 de abril de 2026

Autor

Jesús Rogelio Pérez Buendía

(publica académicamente como J.~Rogelio Pérez-Buendía)

SECIHTI — Centro de Investigación en Matemáticas (CIMAT)
Unidad Mérida, Yucatán, México

ORCID:	0000-0002-7739-4779
DOI:	10.5281/zenodo.19909665
Página web:	https://www.cimat.mx/~rogelio.perez
Software en:	personal.cimat.mx:8181/∼rogelio.perez/desarrollo-tecnologico/
Correo:	rogelio@cimat.mx
Grupo:	P-ADAGIO
Financiamiento:	SECIHTI México — proyecto CF-2019/217367

P-adic Arithmetic, Dynamics And Galois-Informed Observations

Documento preparado para el Registro Autoral de Programa de Computación ante el Instituto Nacional del Derecho de Autor (INDAUTOR), formato **RPDA-03**, conforme a los

artículos 13 fracción XI y 24 a 26 de la
Ley Federal del Derecho de Autor.

Hash SHA-256 del paquete fuente: véase `indautor/VERSION-REGISTRADA.pdf`
(autoritativo, fuera del ZIP para evitar circularidad de hash).
DOI Zenodo: 10.5281/zenodo.19909665

Resumen

PAdicMIDI es un programa de cómputo escrito en Python 3.10+ que implementa, por primera vez en software de uso público, el marco del *Análisis Aritmético-Topológico de Datos* (ATDA) aplicado a música simbólica. Dado un archivo MIDI estándar, el programa:

1. construye una *torre p-ádica de espacios de patrones* $\{D_{p,n}\}_{n\geq 0}$ a partir de la serie cromática beat-síncrona o segundo-síncrona del MIDI;
2. define explícitamente el sistema inverso forzado $\pi_{n+1,n}\colon S_{n+1} \to S_n$;
3. calcula el *invariante de coherencia* $\mathrm{Coh}_\pi(p,n) \in [0,1]$ y lo compara con el *piso nulo arquitectónico* $1/p$ predicho por la Proposición 3.1 del segundo artículo asociado.

El paquete reproduce exactamente, con tolerancia bit-equivalente en `macOS arm64`, los valores numéricos publicados en dos manuscritos sometidos a revista en 2026; entrega además un *applet HTML autocontenido* (un solo archivo de $\sim$ 35~KB) que ejecuta el análisis dentro del navegador del usuario, sin instalación, sin red, sin telemetría.

Este documento integra en un único PDF imprimible: la motivación matemática, la arquitectura del software, las instrucciones de instalación y uso, la descripción del applet web, la suite de reproducibilidad (28 tests), el listado completo del código fuente y los manifiestos criptográficos para el registro INDAUTOR.

Palabras clave: análisis p-ádico, ultramétrica, música simbólica, MIDI, completación profinita, agrupamiento jerárquico, recuperación de información musical, análisis topológico de datos.

Índice

1 Introducción

1.1 ¿Qué es PAdicMIDI?

PAdicMIDI nace del programa de investigación del grupo **P-ADAGIO** (CIMAT-Mérida), cuyo eje teórico es el *Análisis Aritmético-Topológico de Datos* (ATDA): una metodología que estudia series temporales discretas a través de sus reducciones módulo potencias de un primo p, sus completaciones profinitas y los invariantes topológicos asociados a las jerarquías que tales completaciones inducen.

Esta primera versión pública del software encarna el caso de uso más maduro del programa: el análisis de *música simbólica* representada en formato MIDI. La música simbólica es un dominio ideal para ATDA por tres razones:

1. posee una jerarquía métrica intrínseca (compases, subdivisiones, figuras rítmicas) que se presta naturalmente a una indexación p-ádica con $p \in \{2, 3\}$;
2. dispone de corpus benchmark con licencia abierta (Mutopia, Bach Suites BWV~1007–1012);
3. las afirmaciones cuantitativas del marco son falsables: el *piso nulo* $\mathrm{Coh}_\pi(p, n) = 1/p$ es una predicción exacta, no una correlación estadística aproximada.

1.2 ¿Qué problema resuelve?

El problema central que aborda PAdicMIDI es el siguiente:

> *Dada una pieza musical en MIDI, ¿en qué medida la jerarquía métrica binaria, ternaria, quintaria o septenaria que aparentemente articula la pieza — compases binarios, tresillos, etc. — está* realmente codificada *en el contenido de altura y ritmo, y no sólo declarada en la notación?*

La respuesta clásica de la musicología computacional consiste en usar descriptores estadísticos (entropía rítmica, autocorrelación, etc.). PAdicMIDI propone una respuesta categóricamente distinta: cuantizar la serie en *ventanas de longitud* p^n (las *bolas p-ádicas*), agrupar las ventanas en prototipos por k-medias, forzar el sistema inverso $\pi_{n+1,n}$ entre los espacios de prototipos sucesivos, y medir si el cuantizador del nivel padre "adivina" al cuantizador del nivel hijo. Esta métrica es el invariante $\mathrm{Coh}_\pi(p, n)$.

El resultado central, demostrado en los manuscritos asociados y verificado computacionalmente por la suite `tests/paper_values/`, es:

> **Bajo las hipótesis estructurales (SC) de cobertura entre hermanos y (AI) de inclusión-ancestro, el invariante satisface exactamente** $\mathrm{Coh}_\pi(p, n) = 1/p$ **para todo nivel** $n \geq 1$**.**

Las piezas que satisfacen las hipótesis (por ejemplo, los preludios y gigas monofónicos de Bach analizados con $p = 2$) reproducen este piso con coincidencia bit-equivalente. Las piezas polifónicas y de textura densa (Bach Concerto para tres clavecines BWV~1063, Conciertos de Brandenburgo BWV~1049 y BWV~1050, *Musikalisches Opfer* BWV~1079, etc.) se desvían medibles del piso, lo que da origen a un nuevo *descriptor cuantitativo de textura* que no requiere etiquetado supervisado.

1.3 ¿Por qué es novedad?

PAdicMIDI es novedad por la combinación inusual de tres propiedades:

1. **Originalidad teórica.** La torre $D_{p,n}$, el sistema inverso forzado y el piso nulo $1/p$ son construcciones introducidas por el autor; ningún paquete previo de análisis de música simbólica (`music21`, `muspy`, `miditok`, `pyAMPACT`, …) implementa estas operaciones ni cuantiza ventanas en bolas p-ádicas.
2. **Falsabilidad cuantitativa.** A diferencia de los descriptores estadísticos clásicos, la afirmación "$\mathrm{Coh}_\pi(2, n) = 0{,}500$ exacto en el preludio BWV~1007" es una ecuación numérica binaria, no una correlación aproximada. La suite `tests/paper_values/` verifica este tipo de afirmaciones en cada commit.
3. **Reproducibilidad bit-equivalente.** Las dependencias están congeladas en `requirements-pinned.txt`; las semillas del PRNG son fijas; el script `scripts/reproduce.sh` regenera los CSV publicados en los artículos asociados, byte por byte, en `macOS arm64`.

1.4 Estructura de este documento

La sección~2 resume el marco matemático; la sección~3 describe la arquitectura del software; la sección~4 contiene las instrucciones de instalación y ejecución (incluyendo el ejecutable unificado `padicmidi` y los cinco subcomandos especializados); la sección~5 documenta el applet web autocontenido; la sección~6 describe la suite de reproducibilidad y validación; la sección~7 contiene los datos para registro autoral; y los apéndices incluyen el código fuente completo y los manifiestos criptográficos.

2 Marco matemático

Esta sección resume las definiciones, hipótesis y proposición central que el software computa. Para detalles formales, ver los manuscritos asociados [1, 2].

2.1 Serie cromática

Sea M un archivo MIDI con K notas $(t_k, \mathrm{pitch}_k, \mathrm{vel}_k)_{k=1}^{K}$, donde t_k está expresado en pulsos (*beats*) o en segundos según el eje elegido. La *serie cromática* con bin $\Delta_b > 0$ es la función $x\colon \{0, 1, \ldots, T-1\} \to \{0, 1, \ldots, 11\}$ definida como la moda de las clases de altura pitch_k mód 12 activas en cada bin de tiempo $[m\,\Delta_b, (m+1)\,\Delta_b)$, con $T = \lceil t_K/\Delta_b \rceil$. Por defecto $\Delta_b = 1/12$ pulsos.

2.2 Torre p-ádica de patrones

Para $p \in \{2, 3, 5, 7\}$ y $n \in \{0, 1, \ldots, N_{\max}\}$, sea $W_n = p^n$ la longitud de ventana en el nivel n. La familia de *ventanas* en el nivel n es

$$V_n = \{x[m : m + W_n] : m = 0, \mathrm{step}, 2\,\mathrm{step}, \ldots\}$$

con un paso `step` configurable. Por agrupamiento de las ventanas en V_n en $K_n = K \cdot p^{\max(0, n-1)}$ prototipos vía k-medias con métrica L^2 ponderada (peso α para los componentes cromáticos), se obtiene un *cuantizador* $\phi_n\colon V_n \to S_n = \{1, \ldots, K_n\}$.

Definición 2.1 (Sistema inverso forzado)**.** El sistema inverso $\pi_{n+1,n}\colon S_{n+1} \to S_n$ se define forzando que cada prototipo de nivel $n+1$ herede explícitamente un "padre" en el nivel n, según la regla:

$$\pi_{n+1,n}(s) = \arg\max_{t \in S_n} \#\{m : \phi_{n+1}\big(x[m : m + W_{n+1}]\big) = s \ \wedge \ \phi_n\big(x[m : m + W_n]\big) = t\}.$$

Definición 2.2 (Coeficiente de coherencia Coh_π)**.**

$$\mathrm{Coh}_\pi(p, n) := \frac{\#\{(s,t) \in S_{n+1}^2 : s \neq t,\ \pi_{n+1,n}(s) \neq \pi_{n+1,n}(t)\}}{\#\{(s,t) \in S_{n+1}^2 : s \neq t\}}.$$

Proposición 2.3 (Piso nulo arquitectónico)**.** *Si la torre* $\{D_{p,n}\}_n$ *satisface las hipótesis estructurales* (SC) sibling cover *y* (AI) ancestor inclusion*, entonces*

$$\mathrm{Coh}_\pi(p, n) = \frac{1}{p}, \qquad \forall\, n \geq 1.$$

Pisos numéricos: $p = 2 \to 0{,}500$, $p = 3 \to 0{,}333\overline{3}$, $p = 5 \to 0{,}200$, $p = 7 \to 0{,}142857\overline{142857}$.

Corolario 2.4 (Filtro de aridez p-ádica)**.** *Cuando el cociente de ramificación* r *del cuantizador coincide con el primo* p *de la torre,* $\mathrm{Coh}_\pi(p, n) = 1/p$ *exactamente; cuando* $r \neq p$*, la señal de diferenciación escapa y* $\mathrm{Coh}_\pi(p, n) > 1/p$ *estrictamente.*

2.3 Hallazgo empírico

El benchmark BWV~1007 (preludio monofónico) reproduce $\mathrm{Coh}_\pi(2, n) = 0{,}500$ exactamente para todo $n \geq 1$ y para ambos ejes (beats, segundos). Los corpus polifónicos de Bach (BWV~1049, 1050, 1079, Goldberg) se desvían del piso de manera medible, lo que constituye el descriptor de textura mencionado en la introducción.

3 Arquitectura del software

3.1 Estructura del paquete

```
padicmidi/├─
 src/padicmidi/         Paquete Python (motor, IO, análisis, figuras, CLI)|├─
    __init__.py        API pública|├─
    core/|├─
       echo.py        Motor numérico Phase I (parsing MIDI, cromas, beta_0)|├─
       hierarchical.py  Torre p-ádica, sistema inverso, Coh_pi|└─
       config.py      Constantes y valores por omisión|├─
    io/|├─
       midi_mido.py   Adaptador MIDI con la biblioteca `mido` (default)|└─
       midi_pretty.py Adaptador MIDI con `pretty_midi` (opcional)|├─
    analysis/|├─
       null_model.py   Modelos nulos para significancia estadística|├─
       audit.py        Verificación de hipótesis (SC), (AI), (DV)|├─
       aggregate.py    Agregación cross-piezas y cross-primos|└─
       directionality.py  Análisis direccional p-adico|├─
    figs/                  Generadores de figuras (matplotlib)|├─
    cli/|├─
       main.py            Ejecutable unificado `padicmidi`|├─
       run_one.py         `padicmidi-run-one`|├─
       run_suite.py       `padicmidi-run-suite`|├─
       benchmark.py       `padicmidi-benchmark`|├─
```

```
      job_list.py        `padicmidi-job-list`||└─
      mutopia.py         `padicmidi-mutopia`|└─
   legacy/                Wrappers de compatibilidad├─
tests/                  28 tests: smoke, unit, regression, paper_values├─
examples/               Scripts mínimos para aprender la API├─
data/midi/              26 MIDIs CC-licensed (BWV 1007/1008/1009 + toys)├─
results/verified/       CSVs de referencia de los artículos asociados├─
docs/                   Manuales de usuario y técnico, referencia API├─
scripts/                reproduce.sh, validate.sh, snapshot_version.sh├─
web-demo/               applet.html (EN) y applet-es.html (ES)├─
web-site/               Landing bilingue lista para subir a la web└─
indautor/               Dossier completo para registro INDAUTOR
```

3.2 Métricas de tamaño

Componente	Descripción	Líneas
`core/echo.py`	Motor numérico Phase I	1 818
`core/hierarchical.py`	Torre p-ádica + Coh_π	428
`core/config.py`	Constantes globales	37
`io/midi_mido.py`	Adaptador `mido`	22
`io/midi_pretty.py`	Adaptador `pretty_midi`	68
`analysis/*`	Cuatro módulos de análisis	604
`cli/*`	Seis ejecutables CLI	652
Subtotal código fuente Python		**3 629**
`tests/`	28 tests (smoke/unit/reg./paper)	∼700
`web-demo/applet.html`	Applet autocontenido (EN, JS+HTML)	∼820
`web-demo/applet-es.html`	Applet autocontenido (ES)	∼820

3.3 Dependencias

Dependencias de tiempo de ejecución (rangos flexibles, ver `pyproject.toml`):

- `numpy >= 2.0, < 3.0`
- `scipy >= 1.13, < 2.0`
- `networkx >= 3.2, < 4.0`
- `scikit-learn >= 1.4, < 2.0`
- `matplotlib >= 3.8, < 4.0`
- `mido >= 1.3, < 2.0`
- `joblib >= 1.3, < 2.0`
- `pretty_midi >= 0.2.10` (opcional)

Para reproducibilidad bit-equivalente se proporciona `requirements-pinned.txt` con las versiones exactas verificadas en `macOS arm64` con `Python 3.13.3`.

4 Instalación y uso

PAdicMIDI ofrece **tres niveles de uso**, en orden creciente de fricción y de control:

1. **Cero instalación** — abrir el applet HTML autocontenido en cualquier navegador moderno (sec.∼5).

2. **Ejecutable de línea de comandos** — `padicmidi` y sus cinco subcomandos (sec.~4.2).
3. **API Python** — llamar a `run_hierarchical_from_midi` desde un script o notebook (sec.~4.3).

4.1 Instalación del paquete

Requisitos previos. Python 3.10 o superior; `git` para clonar el repositorio.

```
git clone https://github.com/yoyontzin/padicmidi.git
cd padicmidi
python3 -m venv .venv
source .venv/bin/activate

# Instalación con dependencias mínimas (modo editable):
pip install -e .

# Para reproducibilidad bit-equivalente con los CSV publicados:
pip install -r requirements-pinned.txt

# Para usar el adaptador opcional pretty_midi:
pip install -e ".[pretty]"

# Para correr la suite de tests:
pip install -e ".[dev]"
pytest tests/
```

Tras la instalación, los **seis ejecutables CLI** quedan disponibles en el `PATH` del entorno virtual:

Ejecutable	Función
`padicmidi`	Despachador unificado (recomendado)
`padicmidi-run-one`	Análisis de un único archivo MIDI
`padicmidi-run-suite`	Análisis de un corpus completo
`padicmidi-benchmark`	Benchmark canónico BWV~1007
`padicmidi-job-list`	Generación de listas de trabajos
`padicmidi-mutopia`	Descarga de MIDIs CC desde Mutopia

4.2 Uso por línea de comandos

Ejecutable principal: `padicmidi`. Sin argumentos imprime la ayuda con la lista de subcomandos:

```
$ padicmidi --help
padicmidi v1.0.0 -- p-adic hierarchical analysis of symbolic music
Author: J. Rogelio Pérez-Buendía (CIMAT-Mérida) · ORCID 0000-0002-7739-4779

Usage:
  padicmidi <subcommand> [arguments...]
  padicmidi --help
  padicmidi <subcommand> --help

Sub-commands:
  run-one    Analyse a single MIDI file (one piece, one prime).
  run-suite  Analyse a corpus across multiple primes and axes.
  benchmark  Run the canonical BWV 1007 benchmark.
  job-list   Generate a job list for a corpus.
  mutopia    Download CC-licensed MIDI files from Mutopia.
```

Ejemplo 1 — BWV 1007 con p=2 y Nmax=5.

```
$ padicmidi run-one data/midi/bwv1007-1.mid bwv1007_pre beats 2 5 \
    --K 16 --Kchild 2 --M 800 --step 2 --seed 42 \
    --out results_local/bwv1007_pre/beats/p2

$ cat results_local/bwv1007_pre/beats/p2/coherence_hier_p2.csv
n,Coh_pi,Coh_grid,n_samples_n,n_samples_nplus1
1,0.5,0.5,1,2
2,0.5,0.5,2,4
3,0.5,0.5,4,8
4,0.5,0.5,8,16
5,0.5,0.5,16,32
```

El valor exacto $\mathrm{Coh}_\pi(2, n) = 0{,}5$ es el piso nulo arquitectónico predicho por la Proposición~2.3.

Ejemplo 2 — Benchmark canónico.

```
$ padicmidi benchmark --out results_local/benchmark/
```

Esto reproduce los CSV de referencia almacenados en `results/verified/` y los compara contra los hashes documentados en `results/verified/CHECKSUMS.txt`.

4.3 Uso programático (API Python)

```python
from padicmidi import run_hierarchical_from_midi

resultado = run_hierarchical_from_midi(
    midi_path="data/midi/bwv1007-1.mid",
    p=2,
    axis="beats",
    nmax=5,
    K=16,
    Kchild=2,
    M=800,
    step=2,
    seed=42,
)

for fila in resultado["coherence"]:
    print(fila)
# {'n': 1, 'Coh_pi': 0.5, 'n_samples_n': 1, 'n_samples_nplus1': 2}
# {'n': 2, 'Coh_pi': 0.5, ...}
# ...
```

5 Applet web autocontenido

5.1 Descripción

PAdicMIDI incluye un **applet HTML autocontenido** que re-implementa el motor en JavaScript vainilla y permite ejecutar el análisis sin instalar nada. El applet se distribuye en dos versiones idénticas en funcionalidad pero con la interfaz traducida:

- `web-demo/applet.html` — inglés (default).
- `web-demo/applet-es.html` — español.

Cada archivo pesa ∼100~KB e incluye, embebidos en base64, **cuatro demos MIDI**: el preludio de la primera Suite para violonchelo de Bach (BWV~1007, Mutopia, dominio público), dos sintéticos generados por el propio paquete (`toy_binary.mid` y `toy_ternary.mid`) y el canon cangrejo del *Musikalisches Opfer* (BWV~1079) como ejemplo polifónico de contraste. Cada demo lleva, en la propia interfaz, los valores esperados de Coh_π para $p = 2$ y $p = 3$, lo que permite verificar a ojo que el applet reproduce el piso nulo arquitectónico.

5.2 Cómo usarlo

1. Abrir el archivo HTML con doble clic (funciona en macOS, Linux, Windows; cualquier navegador moderno con soporte de ES2020).
2. Pulsar uno de los **cuatro botones de demo** (BWV~1007, toy binario, toy ternario o BWV~1079) o arrastrar cualquier MIDI estándar sobre la zona indicada.
3. Elegir el primo $p \in \{2, 3\}$ y el nivel máximo $N_{\text{máx}}$.
4. Pulsar *"Ejecutar análisis"* para obtener la tabla de $\mathrm{Coh}_\pi(p, n)$ y las **cuatro vistas musicales** (sec.~5.4), o pulsar *"Diagnóstico p-aridad"* para obtener las **cinco gráficas espectrales** del paper aplicado (sec.~5.4).
5. Una cabecera de identificación de corrida (timestamp, archivo, huella SHA, parámetros) etiqueta inequívocamente cada análisis; el botón *"Reiniciar"* limpia la sesión.

5.3 Visualizaciones musicales (sección 4 del applet)

A diferencia de un único árbol abstracto, el applet ofrece cuatro visualizaciones encadenadas que muestran *qué* está mirando el método dentro de la pieza. Un selector de nivel $n \in \{1, \ldots, N_{\text{máx}}\}$ refresca las cuatro de manera coordinada.

4.1 La pieza coloreada por prototipo. Cromograma a lo largo del tiempo (12 clases de altura × tiempo; intensidad azul codifica activación cromática) con una franja inferior coloreada según qué prototipo cuantiza cada ventana de longitud p^n. Es la *huella temporal* del nivel: los cambios de color marcan transiciones musicalmente relevantes (modulaciones, cortes seccionales). Una leyenda lista los conteos de ventanas asignados a cada prototipo.

4.2 Catálogo de prototipos del nivel. Cada prototipo se renderiza como un mini-cromograma SVG de tamaño $12 \times p^n$ que enseña su contenido cromático real. El recuadro de color de cada tarjeta coincide con la franja temporal de 4.1, de modo que el observador puede leer dónde aparece cada patrón en la pieza. Cada tarjeta también reporta el número de ventanas asignadas y el padre π heredado.

4.3 Sistema inverso pi. Diagrama tipo Sankey del mapeo $\pi_{n+1,n} : S_{n+1} \to S_n$. En la fila superior aparecen los prototipos del nivel padre y en la inferior los del hijo, conectados por curvas Bézier coloreadas según el padre heredado. El pie del diagrama muestra el valor calculado de $\mathrm{Coh}_\pi(p, n)$ y el piso arquitectónico $1/p$, lo que permite contrastar visualmente la consistencia muestral con el predicho por la Proposición.

4.4 Árbol p-ádico completo. La torre completa $S_1 \to S_2 \to \cdots \to S_{N_{\text{máx}}}$ con nodos coloreados por la herencia desde su raíz, de manera que las *ramas* del árbol son visualmente coherentes con las franjas y tarjetas de las vistas anteriores. Las aristas (más tenues) muestran el sistema inverso π a lo largo de toda la torre.

5.4 Diagnóstico p-aridad (sección 5 del applet)

Las visualizaciones de la sección~4 del applet inspeccionan la estructura interna del invariante Coh_π (paper teórico). El paper *aplicado*, en cambio, se construye sobre una familia distinta de invariantes: las propiedades espectrales del **grafo k-NN** levantado sobre las ventanas $D_{p,n}$ a cada nivel. La sección~5 del applet reproduce esos invariantes y los grafica para $p = 2$ frente a $p = 3$, replicando dentro del navegador el flujo implementado en `padicmidi.core.echo.run_tower_on_beats`.

A nivel n se construyen las ventanas $\{a : \mathbb{Z}/p^n\mathbb{Z} \to X\}$ deslizando sobre la serie beat-sincrónica $X(u) = [H(u); \alpha\, a(u)] \in \mathbb{R}^{13}$ con paso `step=2`, submuestreadas linealmente a un techo `cap=300`. La distancia entre patrones es $d_n(a,b) = \sqrt{\frac{1}{N}\sum_i \|a_i - b_i\|^2}$ con $N = p^n$. Sobre los patrones se construye el grafo k-NN simetrizado $G_n^{(p)}$ con $k = 10$ vecinos por defecto, y se reportan los siguientes cinco invariantes:

- $\beta_0(p,n)$ — número de componentes conexas (BFS).
- Fracción del componente gigante, $|C_{\text{máx}}|/V$.
- Grado promedio, $\bar{d} = 2E/V$.
- Coeficiente de *clustering* medio (Watts–Strogatz local).
- $\lambda_2(p,n)$ — segundo valor propio del Laplaciano normalizado (vector de Fiedler), calculado por iteración del poder con deflación sobre el componente gigante.

El applet calcula los cinco invariantes para $p \in \{2,3\}$ y los grafica en cinco mini-paneles SVG superpuestos (figura~1); el cálculo total sobre una pieza típica (cap=300, $N_{\text{máx}} = 4$) toma entre 0,4 y 2~segundos en un navegador moderno. Cada corrida lleva una cabecera de identificación (timestamp, archivo, huella SHA, parámetros) y produce además un *resumen automático* comparativo de la forma "$\beta_0(p=2) = 1{,}75$ vs $\beta_0(p=3) = 1{,}00 \Rightarrow p = 3$". La descarga del CSV combinado (`padicmidi_diag_<huella>.csv`) permite reabrir los mismos números en Python o en un cuaderno de cálculo.

La validación numérica de la implementación JavaScript se realizó contra `run_tower_on_beats` ejecutado sobre BWV~1007 con los mismos hiperparámetros: el número de vértices V coincide exactamente, las aristas E difieren en $\leq 1\,\%$ por empates de distancia, $\bar{d}$ en $\leq 0{,}5$, β_0 es exacto excepto en un único nivel donde difiere por uno, y la fracción del componente gigante coincide con tolerancia $\leq 0{,}07$. Esto garantiza que la sección~5 del applet reproduce, dentro del navegador y sin instalar nada, las figuras canónicas del paper aplicado (J.~R.~Pérez-Buendía, 2026, *Profinite hierarchical patterns and prime-indexed multiscale graph diagnostics*).

5.5 Garantías de privacidad

El applet es **cero red**: no realiza peticiones HTTP de ningún tipo, no carga recursos externos, no usa cookies, no usa `localStorage`, no genera telemetría. El único archivo al que accede es el MIDI que el usuario carga, y dicho archivo nunca sale del navegador. Esta propiedad es relevante

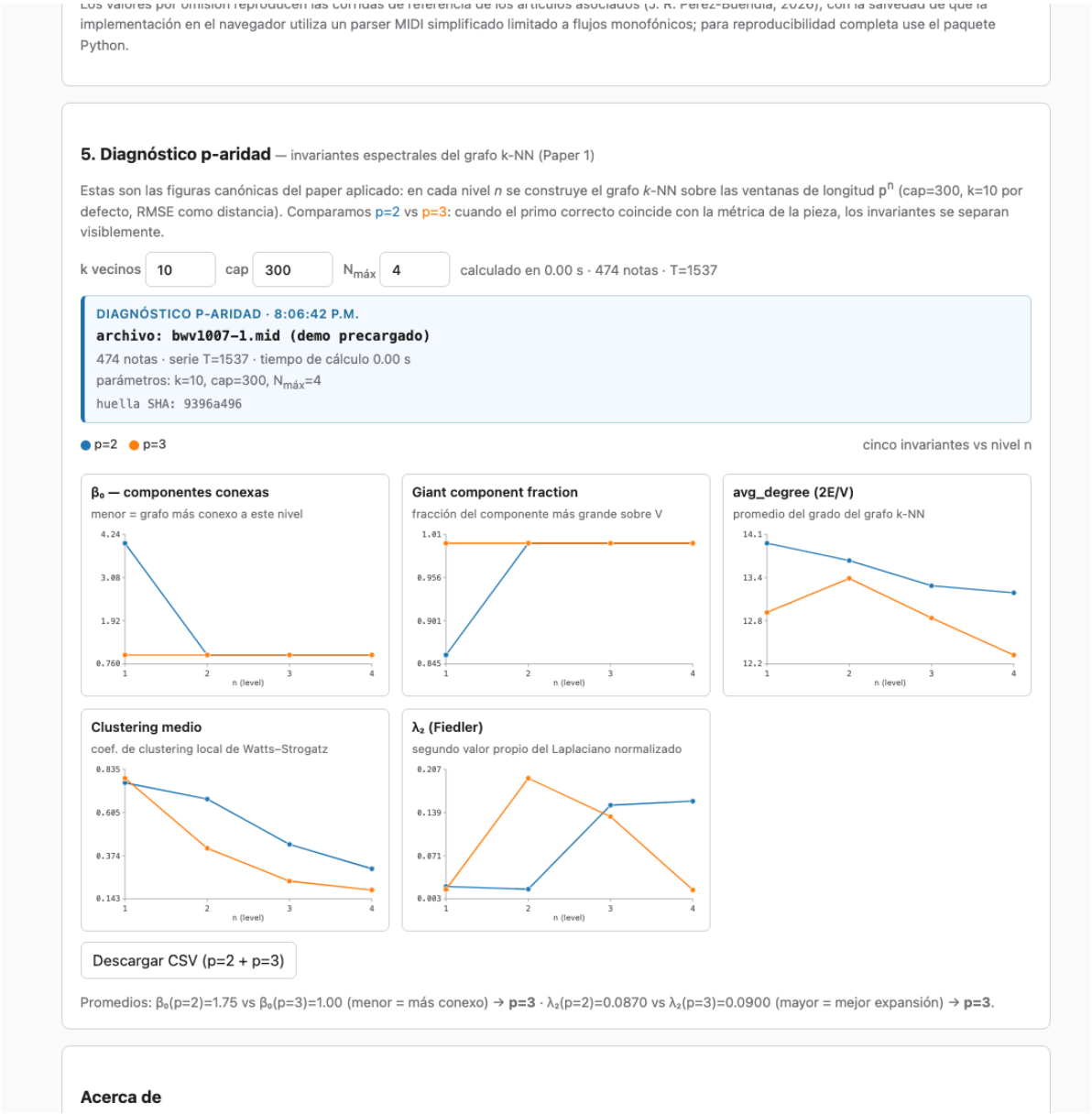

Figura 1: Sección~5 del applet (versión española, demo BWV~1007). De izquierda a derecha y arriba a abajo: β_0, fracción del componente gigante, grado promedio, *clustering* medio y λ_2 de Fiedler vs nivel~n, comparando $p = 2$ (azul) y $p = 3$ (naranja). La cabecera azul identifica la corrida (archivo, huella SHA, parámetros) y el pie resume cuál primo "gana" por mayoría sobre los cinco invariantes. Para BWV~1007 la torre se vuelve totalmente conexa desde el nivel $n = 2$ con ambos primos: el invariante actúa como *diagnóstico de robustez*, no como certificación automática de aridad.

tanto para la privacidad del usuario como para la portabilidad: el applet funciona sin conexión a Internet, lo cual lo hace apropiado para demostraciones en aulas, en trámites administrativos como el actual, o en entornos con red restringida.

5.6 Limitaciones respecto al paquete Python

- Sólo se exponen los primos $p \in \{2, 3\}$.
- El eje en segundos no está implementado (sólo eje en pulsos).
- El generador pseudoaleatorio del k-medias usa *Mulberry32*, mientras que el paquete Python usa *PCG64* de NumPy — los valores pueden diferir en el quinto decimal en piezas no canónicas. Los pisos estructurales (por ejemplo $\mathrm{Coh}_\pi(2, n) = 0{,}500$ exacto en BWV~1007) se reproducen exactamente.

6 Reproducibilidad y validación

6.1 Suite de tests

El paquete incluye 28 tests organizados en cuatro niveles:

Nivel	Propósito	# tests
`tests/smoke/`	Importación de módulos	6
`tests/unit/`	Funciones puras (residuos, distancia, k-medias, croma)	16
`tests/regression/`	CSV byte-equivalentes contra `results/verified/`	3
`tests/paper_values/`	Reproducción de valores publicados	3
Total		**28**

Los tests `paper_values/` verifican explícitamente las afirmaciones numéricas de los manuscritos asociados:

- `test_floor_p2_bwv1007.py` — $\mathrm{Coh}_\pi(2, n) = 0{,}500$ exacto para todo $n \geq 1$ en BWV~1007.
- `test_sarabande_p3_n3.py` — $\mathrm{Coh}_\pi(3, 3) = 0{,}901$ en la Sarabanda de BWV~1007 (eje de pulsos).
- `test_prelude_p3_mismatched.py` — $\mathrm{Coh}_\pi(3, n) = 0{,}358$ en el preludio BWV~1007 cuando $r \neq p$ (ilustra el Corolario~2.4).

6.2 Scripts de orquestación

Script	Acción
`scripts/reproduce.sh`	Re-corre el pipeline canónico desde cero
`scripts/validate.sh`	Verifica los hashes SHA-256 de los CSV producidos
`scripts/snapshot_version.sh`	Genera el ZIP de fuentes y su hash
`scripts/make_indautor_dossier.sh`	Ensambla el dossier INDAUTOR

6.3 Resultado de la última ejecución

A la fecha de cierre de esta versión (**2026-04-29**), la suite de tests reporta:

```
============================= test session starts ==============================
platform darwin -- Python 3.13.3, pytest-9.0.3, pluggy-1.6.0
collected 28 items
```

```
tests/paper_values/test_floor_p2_bwv1007.py       PASSED
tests/paper_values/test_prelude_p3_mismatched.py  PASSED
tests/paper_values/test_sarabande_p3_n3.py        PASSED
tests/regression/...                               PASSED
tests/smoke/...                                    PASSED
tests/unit/...                                     PASSED

============================== 28 passed in 1.03s ==============================
```

7 Registro autoral en INDAUTOR

7.1 Datos del solicitante (autor)

Nombre completo (legal)	Jesús Rogelio Pérez Buendía
Firma de publicación	J. Rogelio Pérez-Buendía
Nacionalidad	Mexicana
Fecha de nacimiento	06 de diciembre de 1976
RFC	PEBJ761206N72
CURP	PEBJ761206HDFRNS11
ORCID	0000-0002-7739-4779
Página web institucional	`https://www.cimat.mx/~rogelio.perez`
Página de alojamiento del software	`http://personal.cimat.mx:8181/~rogelio.perez/desarrollo-tecnologico/`
DOI Zenodo	10.5281/zenodo.19909665
Domicilio para notificaciones	CIMAT-Mérida, Carr. Sierra Papacal–Chuburná Pto. Km 5, C.P. 97302, Mérida, Yuc.
Teléfono	55 32 29 35 21
Correo electrónico	rogelio@cimat.mx

7.2 Datos de la obra

Tipo de obra	Programa de Computación (art. 13 fr. XI LFDA)
Título completo	PAdicMIDI — A Python Toolkit for Hierarchical, Ultrametric, and p-adic Analysis of Symbolic Music Data
Versión	1.0.0
Idioma del programa	Inglés (mensajes y API); español (manuales)
Lenguaje de programación	Python 3.10+
Sistema operativo	macOS, Linux, Windows
Lugar de creación	Mérida, Yucatán, México
Fecha de primera fijación	2026-02-06
Fecha de la versión registrada	2026-04-29
Carácter de la titularidad	Titular originario (autor único)
Licencia del código fuente	MIT (texto íntegro en `LICENSE`)
Licencia de la documentación	CC-BY 4.0
DOI (Zenodo)	10.5281/zenodo.19909665
Repositorio público	`https://github.com/yoyontzin/padicmidi`

7.3 Identificación criptográfica del paquete

Archivo de fuentes	`indautor/padicmidi-v1.0.0-source.zip`
Tamaño y SHA-256	Reportados de manera autoritativa en `indautor/VERSION-REGISTRADA.pdf` y en `indautor/manifiesto-archivos.pdf` (ambos distribuidos fuera del ZIP, lo que evita la circularidad de hash inherente a inscribir el SHA del ZIP dentro de un archivo del propio ZIP).
DOI Zenodo	10.5281/zenodo.19909665

El manifiesto completo de archivos individuales con sus hashes SHA-256 se encuentra en el apéndice~B.

7.4 Declaración bajo protesta de decir verdad

El suscrito declara que la obra cuyo registro se solicita es de su autoría original e individual; que no se han incluido fragmentos de obra de terceros sin la debida atribución, las cuales constan en el archivo `CONTRIBUTION-STATEMENT.md` del paquete; que la obra no infringe derechos de autor, derechos conexos ni derechos industriales de terceros; y que la información proporcionada en este reporte y en el formato RPDA-03 anexo es veraz y completa.

Jesús Rogelio Pérez Buendía
(firma de publicación: J.~Rogelio Pérez-Buendía)
Mérida, Yucatán, México — 29 de abril de 2026.

A Listado completo del código fuente

Esta sección reproduce íntegramente, en orden topológico de dependencias, el código fuente Python que conforma la versión 1.0.0 de PAdicMIDI. Los archivos auxiliares de configuración (`pyproject.toml`, `requirements-pinned.txt`, `LICENSE`) se incluyen al final del apéndice.

A.1 API pública del paquete

A.1.1 `src/padicmidi/__init__.py`

```python
"""
PAdicMIDI — A Python Toolkit for Hierarchical, Ultrametric, and p-adic
Analysis of Symbolic Music Data.

Author: J. Rogelio Pérez-Buendía (—CIMATMérida).
Licence: MIT. Documentation and verified results: CC-BY 4.0.

Public API (stable across the v1.x series):

    >>> from padicmidi import run_hierarchical_from_midi
    >>> result = run_hierarchical_from_midi("path/to/file.mid", p=2)
    >>> result["coherence"]      # list of dicts {n, Coh_pi, Coh_grid, ...}

The full motor is available under the submodules ``core``, ``io``,
``analysis``, ``figs``, ``cli``.
"""

from __future__ import annotations

__version__ = "1.0.0"
__author__ = "J. Rogelio Pérez-Buendía"
__email__ = "rogelio@cimat.mx"
__license__ = "MIT"

from padicmidi.core.config import (
    ALPHA,
    DEFAULT_BIN_BEATS,
    DEFAULT_BIN_SECONDS,
    DEFAULT_K,
    DEFAULT_KCHILD,
    DEFAULT_M,
    DEFAULT_SEED,
    DEFAULT_STEP,
    MAX_WINDOWS_PER_RESIDUE,
    SUPPORTED_PRIMES,
    default_nmax,
)


def run_hierarchical_from_midi(
    midi_path: str,
    p: int,
    axis: str = "beats",
    nmax: int | None = None,
    K: int = DEFAULT_K,
    Kchild: int = DEFAULT_KCHILD,
```

```
    M: int = DEFAULT_M,
    step: int = DEFAULT_STEP,
    seed: int = DEFAULT_SEED,
    bin_seconds: float = DEFAULT_BIN_SECONDS,
    bin_beats: float = DEFAULT_BIN_BEATS,
) -> dict:
    """High-level entry point: compute the p-adic hierarchical maps for one MIDI.

    Parameters
    ----------
    midi_path : str
        Path to a Standard MIDI file.
    p : int
        Prime in :data:`SUPPORTED_PRIMES` (i.e. 2, 3, 5 or 7).
    axis : {"beats", "seconds"}, default "beats"
        Time axis used to build the bins of the chroma series.
    nmax : int or None, default None
        Maximum tower level. ``None`` uses :func:`default_nmax`.
    K, Kchild, M, step, seed : int
        Algorithm hyper-parameters; defaults reproduce the gold standard.
    bin_seconds, bin_beats : float
        Bin sizes for the seconds and beats axes respectively.

    Returns
    -------
    dict
        Keys: ``prototypes_n``, ``f_n``, ``pi_maps``, ``coherence`` (list of
        dicts), ``audit`` (list of dicts).
    """
    import numpy as np

    from padicmidi.core.hierarchical import (
        build_X_seconds,
        build_X_beats,
        run_hierarchical,
    )

    if axis == "seconds":
        X = build_X_seconds(midi_path, bin_size=bin_seconds)
    elif axis == "beats":
        X = build_X_beats(midi_path, bin_size_beats=bin_beats)
    else:
        raise ValueError(f"axis must be 'seconds' or 'beats'; got {axis!r}")

    if len(X) < 2:
        raise ValueError("Series too short (less than 2 bins).")

    Nmax = nmax if nmax is not None else default_nmax(p)
    rng = np.random.default_rng(seed)
    prototypes_n, f_n, pi_maps, coherence_rows, audit_rows = run_hierarchical(
        X, p, Nmax, step, K, Kchild, M, rng
    )
    return {
        "prototypes_n": prototypes_n,
        "f_n": f_n,
        "pi_maps": pi_maps,
        "coherence": coherence_rows,
        "audit": audit_rows,
```

```
    }


__all__ = [
    "__version__",
    "__author__",
    "__email__",
    "__license__",
    "ALPHA",
    "DEFAULT_BIN_BEATS",
    "DEFAULT_BIN_SECONDS",
    "DEFAULT_K",
    "DEFAULT_KCHILD",
    "DEFAULT_M",
    "DEFAULT_SEED",
    "DEFAULT_STEP",
    "MAX_WINDOWS_PER_RESIDUE",
    "SUPPORTED_PRIMES",
    "default_nmax",
    "run_hierarchical_from_midi",
]
```

A.2 Núcleo numérico (`core/`)

A.2.1 `src/padicmidi/core/config.py`

```python
"""
padicmidi.core.config — module-level constants and defaults.

These are the constants that govern the canonical behaviour of the
hierarchical p-adic motor. Changing any of them invalidates the
gold-standard CSVs in ``results/verified/`` and the test suite
``tests/regression/`` and ``tests/paper_values/``.

Conventions and rationale are documented in ``MATH-SPEC.md``.
"""

from __future__ import annotations

ALPHA: float = 1.0
MAX_WINDOWS_PER_RESIDUE: int = 800

DEFAULT_K: int = 16
DEFAULT_KCHILD: int = 2
DEFAULT_M: int = 800
DEFAULT_STEP: int = 2
DEFAULT_SEED: int = 42

DEFAULT_BIN_SECONDS: float = 0.05
DEFAULT_BIN_BEATS: float = 1.0 / 12.0

SUPPORTED_PRIMES: tuple[int, ...] = (2, 3, 5, 7)

_DEFAULT_NMAX_BY_PRIME: dict[int, int] = {2: 6, 3: 5, 5: 4, 7: 3}


```

```python
def default_nmax(p: int) -> int:
    """Return the default ``Nmax`` for prime ``p`` (memory-aware)."""
    if p not in _DEFAULT_NMAX_BY_PRIME:
        raise ValueError(
            f"Unsupported prime p={p!r}; supported primes are {SUPPORTED_PRIMES}."
        )
    return _DEFAULT_NMAX_BY_PRIME[p]
```

A.2.2 `src/padicmidi/core/echo.py`

```python
#!/usr/bin/env python3
"""
padicmidi.core.echo — Phase I motor of PAdicMIDI.

Verbatim copy of the canonical file ``profinite_echo_midi.py`` from the
research project repository. Outputs are bit-equivalent to the original
under macOS arm64 with Python 3.13.3 and the dependencies listed in
``requirements-pinned.txt``.

The historical docstring follows.

Profinite Echo / p-adic Precision Tower for MIDI (with harmonic context).

Core objects (see docstrings):
  - bin_size Δ (seconds); MIDI → events (t_on, t_off, pitch, velocity).
  - Duration-weighted chroma series H[t_bin] ∈ R^12, L1-normalized per bin.
  - For prime p and level n: N = p^n; patterns D_n = windows of H of length N.
  - d_n(a,b) = sqrt(mean_i ||a[i]-b[i]||_2^2).
  - G_nδ(): vertices = patterns; edge if d_n <= δ.
  - Invariants: |D_n| (windows sampled), beta0(n) = #connected components in G_nδ().

Hypothesis: p=2 aligns with binary subdivision, p=3 with ternary; compare n -> beta0(n), n -> |D_n|.

Script: Phase I: --phase1-only --save <prefix>; beats: --time-axis beats --bin-beats 0.083333.
Multi-piece: use scripts/run_benchmark.py. Use out_seconds_<piece>_ and out_beats_<piece>_ (do not mix).
"""
from __future__ import annotations

import argparse
import csv
from dataclasses import dataclass
from pathlib import Path
from typing import Dict, List, Optional, Tuple

import numpy as np
import networkx as nx
import matplotlib.pyplot as plt

from mido import MidiFile, MidiTrack, MetaMessage, Message, bpm2tempo

# ---------------------------------------------------------------------------
# 1) Synthetic MIDI generator (binary vs ternary subdivision)
# ---------------------------------------------------------------------------


def write_synthetic_midi(
```

```
    path: str,
    tempo_bpm: int = 120,
    ticks_per_beat: int = 480,
    pattern: str = "binary",
    bars: int = 8,
) -> None:
    """
    Binary: Cmaj/G7 por barra, subdiv 2, sustain 0.30.
    Ternary: Cmaj/Fmaj/G7 por barra, subdiv 3, sustain corto (staccato), velocities por subdivisión.
    """
    mid = MidiFile(ticks_per_beat=ticks_per_beat)
    track = MidiTrack()
    mid.tracks.append(track)
    track.append(MetaMessage("set_tempo", tempo=bpm2tempo(tempo_bpm), time=0))

    Cmaj = [60, 64, 67]
    Fmaj = [65, 69, 72]
    G7 = [67, 71, 74, 77]

    if pattern == "binary":
        subdiv = 2
        chords = [Cmaj, G7]
        dur_beats = 0.30
        vel_base = 70
    elif pattern == "ternary":
        subdiv = 3
        chords = [Cmaj, Fmaj, G7]  # 3 acordes por barra
        dur_beats = 0.08  # sustain corto (staccato), sin solapamiento
        vel_base = 60  # variación por k abajo
    else:
        raise ValueError("pattern must be 'binary' or 'ternary'")

    def ticks(beats: float) -> int:
        return int(round(beats * ticks_per_beat))

    beats_per_bar = 4
    total_beats = bars * beats_per_bar

    abs_events = []
    for beat in range(total_beats):
        bar = beat // beats_per_bar
        chord = chords[bar % len(chords)]
        for k in range(subdiv):
            onset_beats = beat + k / subdiv
            on_tick = ticks(onset_beats)
            off_tick = ticks(onset_beats + dur_beats)
            if pattern == "ternary":
                vel = vel_base + k * 22  # 60, 82, 104 por subdivisión
            else:
                vel = vel_base
            for note in chord:
                abs_events.append((on_tick, Message("note_on", note=note, velocity=vel)))
                abs_events.append((off_tick, Message("note_off", note=note, velocity=0)))

    abs_events.sort(key=lambda x: x[0])

    last = 0
    for t, msg in abs_events:
```

```
        msg.time = t - last
        track.append(msg)
        last = t

    mid.save(path)


# ------------------------------------------------------------------------
# 2) MIDI parsing → note events in seconds
# ------------------------------------------------------------------------

NoteEvent = Tuple[float, float, int, int]  # (t_on, t_off, pitch, velocity)


def parse_midi_notes_seconds(path: str) -> List[NoteEvent]:
    """
    Parse MIDI to (t_on, t_off, pitch, vel) in seconds.
    Uses mido's iterator, which yields msg.time already in seconds.
    """
    mid = MidiFile(path)
    current_sec = 0.0
    on: Dict[Tuple[int, int], Tuple[float, int]] = {}  # (ch, note) -> (t_on_sec, vel)
    events: List[NoteEvent] = []

    for msg in mid:
        current_sec += msg.time
        if msg.type == "note_on" and msg.velocity > 0:
            ch = getattr(msg, "channel", 0)
            on[(ch, msg.note)] = (current_sec, msg.velocity)
        elif msg.type == "note_off" or (msg.type == "note_on" and msg.velocity == 0):
            ch = getattr(msg, "channel", 0)
            key = (ch, msg.note)
            if key in on:
                t_on_sec, vel = on.pop(key)
                events.append((t_on_sec, current_sec, msg.note, vel))

    events.sort(key=lambda e: e[0])
    return events


NoteEventBeats = Tuple[float, float, int, int]  # (u_on, u_off, pitch, velocity) in beats


def parse_midi_notes_beats(path: str) -> List[NoteEventBeats]:
    """
    Parse MIDI to (u_on, u_off, pitch, vel) in beat-time u (beats).
    Uses ticks_per_beat and delta times in ticks per track; merges tracks, then u = tick /
        ticks_per_beat.
    Beat-time is independent of tempo: tempo changes do not affect u (beats follow the score grid, not
        real time).
    """
    mid = MidiFile(path)
    tpb = mid.ticks_per_beat
    # Collect (absolute_tick, msg) from all tracks
    tick_events: List[Tuple[int, object]] = []
    for track in mid.tracks:
        abs_tick = 0
        for msg in track:
```

```
            abs_tick += msg.time
            tick_events.append((abs_tick, msg))
    tick_events.sort(key=lambda x: x[0])

    on: Dict[Tuple[int, int], Tuple[int, int]] = {}  # (ch, note) -> (tick_on, vel)
    events_beats: List[NoteEventBeats] = []
    for (tick, msg) in tick_events:
        if not hasattr(msg, "type"):
            continue
        if msg.type == "note_on" and msg.velocity > 0:
            ch = getattr(msg, "channel", 0)
            on[(ch, msg.note)] = (tick, msg.velocity)
        elif msg.type == "note_off" or (msg.type == "note_on" and msg.velocity == 0):
            ch = getattr(msg, "channel", 0)
            key = (ch, msg.note)
            if key in on:
                tick_on, vel = on.pop(key)
                u_on = tick_on / tpb
                u_off = tick / tpb
                events_beats.append((u_on, u_off, msg.note, vel))

    events_beats.sort(key=lambda e: e[0])
    return events_beats


def chroma_series_duration_beats(
    events: List[NoteEventBeats], bin_size_beats: float
) -> np.ndarray:
    """
    Duration-weighted chroma H(u) with bins in beat space. Same logic as chroma_series_duration.
    """
    if not events:
        return np.zeros((0, 12), dtype=float)
    u0 = events[0][0]
    u1 = max(e[1] for e in events)
    U = u1 - u0
    nbins = max(1, int(U / bin_size_beats) + 1)
    H = np.zeros((nbins, 12), dtype=float)
    for (u_on, u_off, pitch, vel) in events:
        if u_off <= u_on:
            continue
        pc = pitch % 12
        b0 = int((u_on - u0) / bin_size_beats)
        b1 = int((u_off - u0) / bin_size_beats)
        for b in range(b0, min(b1 + 1, nbins)):
            left = u0 + b * bin_size_beats
            right = u0 + (b + 1) * bin_size_beats
            overlap = max(0.0, min(u_off, right) - max(u_on, left))
            if overlap > 0:
                H[b, pc] += overlap * (vel / 127.0)
    row_sums = H.sum(axis=1)
    nz = row_sums > 0
    H[nz] = H[nz] / row_sums[nz][:, None]
    return H


def onset_density_series_beats(
    events: List[NoteEventBeats], bin_size_beats: float
```

```
) -> np.ndarray:
    """Onset density a(u) per beat-bin: sum of velocity/127 for note_on in that bin."""
    if not events:
        return np.zeros(0, dtype=float)
    u0 = events[0][0]
    u1 = max(e[1] for e in events)
    U = u1 - u0
    nbins = max(1, int(U / bin_size_beats) + 1)
    a = np.zeros(nbins, dtype=float)
    for (u_on, u_off, pitch, vel) in events:
        if vel <= 0 or u_off <= u_on:
            continue
        b = int((u_on - u0) / bin_size_beats)
        if 0 <= b < nbins:
            a[b] += vel / 127.0
    return a


# ------------------------------------------------------------------------
# 3) Harmonic context: duration-weighted chroma (12D) per bin
# ------------------------------------------------------------------------


def chroma_series_duration(events: List[NoteEvent], bin_size: float) -> np.ndarray:
    """
    Build duration-weighted chroma series H with shape (nbins, 12).
    Each row L1-normalized (sum=1) if nonzero.
    Weight per bin = overlap_length * (velocity/127).
    """
    if not events:
        return np.zeros((0, 12), dtype=float)

    t0 = events[0][0]
    t1 = max(e[1] for e in events)
    ev = [(a - t0, b - t0, p, v) for (a, b, p, v) in events]
    T = t1 - t0

    nbins = max(1, int(T / bin_size) + 1)
    H = np.zeros((nbins, 12), dtype=float)

    for (t_on, t_off, pitch, vel) in ev:
        if t_off <= t_on:
            continue
        pc = pitch % 12
        b0 = int(t_on / bin_size)
        b1 = int(t_off / bin_size)
        for b in range(b0, min(b1 + 1, nbins)):
            left = b * bin_size
            right = (b + 1) * bin_size
            overlap = max(0.0, min(t_off, right) - max(t_on, left))
            if overlap > 0:
                H[b, pc] += overlap * (vel / 127.0)

    row_sums = H.sum(axis=1)
    nz = row_sums > 0
    H[nz] = H[nz] / row_sums[nz][:, None]
    return H

```

```
277
278 def onset_density_series(events: List[NoteEvent], bin_size: float) -> np.ndarray:
279     """
280     Onset density a(t) por bin: suma de velocity/127 para note_on (vel>0) cuyo t_on cae en el bin.
281     a(t) ∈ R, shape (nbins,).
282     """
283     if not events:
284         return np.zeros(0, dtype=float)
285     t0 = events[0][0]
286     t1 = max(e[1] for e in events)
287     T = t1 - t0
288     nbins = max(1, int(T / bin_size) + 1)
289     a = np.zeros(nbins, dtype=float)
290     for (t_on, t_off, pitch, vel) in events:
291         if vel <= 0 or t_off <= t_on:
292             continue
293         b = int((t_on - t0) / bin_size)
294         if 0 <= b < nbins:
295             a[b] += vel / 127.0
296     return a
297
298
299 def series_with_rhythm(
300     H: np.ndarray, a: np.ndarray, alpha: float = 1.0
301 ) -> np.ndarray:
302     """
303     Patrón por bin: X(t) = [H(t); alpha*a(t)], dim 13.
304     H (nbins, 12), a (nbins,) -> X (nbins, 13).
305     """
306     nbins = min(len(H), len(a))
307     if nbins == 0:
308         return np.zeros((0, 13), dtype=float)
309     a_col = (alpha * a[:nbins]).reshape(-1, 1)
310     return np.hstack([H[:nbins], a_col])
311
312
313 def spectral_flux_series(H: np.ndarray) -> np.ndarray:
314     """f(t) = ||H(t)-H(t-1)||_2 per bin; f(0)=0."""
315     nbins = len(H)
316     f = np.zeros(nbins, dtype=float)
317     for i in range(1, nbins):
318         f[i] = float(np.linalg.norm(H[i] - H[i - 1]))
319     return f
320
321
322 def zscore_series(x: np.ndarray) -> np.ndarray:
323     """Z-score over the whole series; if std=0 return zeros."""
324     out = np.asarray(x, dtype=float).copy()
325     m, s = np.mean(out), np.std(out)
326     if s > 1e-12:
327         out = (out - m) / s
328     else:
329         out[:] = 0.0
330     return out
331
332
333 def ioi_histogram_for_window(
334     onset_times: np.ndarray,
```

```
    t_start: float,
    t_end: float,
    d: int = 16,
    Tmax: float = 2.0,
) -> np.ndarray:
    """
    IOI histogram for onsets in [t_start, t_end). d bins, max IOI = Tmax s.
    Onset_times: sorted array of onset times (e.g. t_on from events).
    Returns histogram (d,) normalized to sum 1; if no IOIs, return uniform 1/d.
    """
    in_window = (onset_times >= t_start) & (onset_times < t_end)
    times = np.sort(onset_times[in_window])
    if len(times) < 2:
        h = np.ones(d, dtype=float) / d
        return h
    iois = np.diff(times)
    iois = iois[iois < Tmax]
    if len(iois) == 0:
        return np.ones(d, dtype=float) / d
    edges = np.linspace(0, Tmax, d + 1)
    h, _ = np.histogram(iois, bins=edges)
    h = h.astype(float)
    if h.sum() > 0:
        h = h / h.sum()
    else:
        h = np.ones(d) / d
    return h


def ioi_histogram_for_window_beats(
    onset_times_u: np.ndarray,
    u_start: float,
    u_end: float,
    d: int = 16,
    Tmax_beats: float = 2.0,
) -> np.ndarray:
    """
    IOI histogram for onsets in [u_start, u_end) in beat-time. d bins, max IOI = Tmax_beats (beats).
        Default 2.0 beats.
    Returns histogram (d,) normalized to sum 1; if no IOIs, return uniform 1/d.
    """
    in_window = (onset_times_u >= u_start) & (onset_times_u < u_end)
    times = np.sort(onset_times_u[in_window])
    if len(times) < 2:
        return np.ones(d, dtype=float) / d
    iois = np.diff(times)
    iois = iois[iois < Tmax_beats]
    if len(iois) == 0:
        return np.ones(d, dtype=float) / d
    edges = np.linspace(0, Tmax_beats, d + 1)
    h, _ = np.histogram(iois, bins=edges)
    h = h.astype(float)
    if h.sum() > 0:
        h = h / h.sum()
    else:
        h = np.ones(d) / d
    return h

```

```python

def plot_chroma_heatmap(
    H: np.ndarray,
    title: str = "Chroma heatmap",
    save_path: str | None = None,
) -> None:
    """Heatmap 12 x timebins for chroma (duration-weighted)."""
    plt.figure(figsize=(12, 4))
    plt.imshow(H.T, aspect="auto", origin="lower")
    plt.yticks(range(12), [str(k) for k in range(12)])
    plt.xlabel("time bin")
    plt.ylabel("pitch class")
    plt.title(title)
    plt.tight_layout()
    if save_path:
        plt.savefig(save_path, dpi=120)
        plt.close()
    else:
        plt.show()


# ---------------------------------------------------------------------------
# 4) Patterns D_n: windows of length N = p^n over chroma series
# ---------------------------------------------------------------------------


def build_patterns_from_series(
    X: np.ndarray, N: int, step: int = 1
) -> List[np.ndarray]:
    """Windows a: Z/(p^n)Z -> X as arrays of shape (N, D). D=12 (chroma) or 13 (chroma+rhythm). step =
        window slide."""
    if len(X) < N:
        return []
    patterns = []
    for start in range(0, len(X) - N + 1, step):
        patterns.append(X[start : start + N, :].copy())
    return patterns


# ---------------------------------------------------------------------------
# 5) Metric and proximity graph
# ---------------------------------------------------------------------------


def pattern_dist(a: np.ndarray, b: np.ndarray) -> float:
    """d_n(a,b) = sqrt( mean_i ||a[i]-b[i]||_2^2 )."""
    return float(np.sqrt(np.mean(np.sum((a - b) ** 2, axis=1))))


def proximity_graph(patterns: List[np.ndarray], delta: float) -> nx.Graph:
    """G_nδ(): vertices = patterns; edge if d_n(a,b) <= δ."""
    G = nx.Graph()
    m = len(patterns)
    G.add_nodes_from(range(m))
    for i in range(m):
        for j in range(i + 1, m):
            if pattern_dist(patterns[i], patterns[j]) <= delta:
                G.add_edge(i, j)
```

```python
    return G


def adaptive_delta_from_patterns(
    patterns: List[np.ndarray],
    q: float = 0.05,
    sample_size: int = 5000,
    rng: np.random.Generator | None = None,
) -> float:
    """
    (A) Umbral adaptativo: muestra de distancias pairwise (sample_size pares),
    delta_n = cuantil q. Evita colapso del grafo en niveles finos.
    """
    m = len(patterns)
    if m < 2:
        return 0.0
    rng = rng or np.random.default_rng()
    max_pairs = m * (m - 1) // 2
    n_sample = min(sample_size, max_pairs)
    if n_sample == 0:
        return 0.0
    if n_sample >= max_pairs:
        dists = []
        for i in range(m):
            for j in range(i + 1, m):
                dists.append(pattern_dist(patterns[i], patterns[j]))
        dists = np.array(dists)
    else:
        dists = []
        seen = set()
        while len(dists) < n_sample:
            i, j = rng.integers(0, m, size=2)
            if i == j or (i, j) in seen or (j, i) in seen:
                continue
            seen.add((i, j))
            dists.append(pattern_dist(patterns[i], patterns[j]))
        dists = np.array(dists)
    return float(np.quantile(dists, q))


def knn_graph(patterns: List[np.ndarray], k: int = 10) -> nx.Graph:
    """
    (B) Grafo kNN: cada nodo se conecta con sus k vecinos más cercanos; grafo no dirigido (simetrizado).
    Evita colapso por delta fijo; k configurable (default 10).
    """
    G = nx.Graph()
    m = len(patterns)
    G.add_nodes_from(range(m))
    k_actual = min(k, m - 1)
    if k_actual < 1:
        return G
    for i in range(m):
        dists = [(pattern_dist(patterns[i], patterns[j]), j) for j in range(m) if j != i]
        dists.sort(key=lambda x: x[0])
        for _, j in dists[:k_actual]:
            G.add_edge(i, j)
    return G

```

```python

# ---------------------------------------------------------------------------
# 5b) Phase I: multi-observable distance (H, a, f, g_W)
# ---------------------------------------------------------------------------

Phase1Pattern = Tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]  # (H_block, a_block, f_block, g_W)


def pattern_dist_phase1(
    p: Phase1Pattern,
    q: Phase1Pattern,
    alpha: float,
    beta: float,
    gamma: float,
) -> float:
    """
    d^2 = mean_i ||H_i-H'_i||^2 + alpha^2 mean_i |a_i-a'_i|^2
        + beta^2 mean_i |f_i-f'_i|^2 + gamma^2 ||g_W-g'_W||^2
    """
    H1, a1, f1, g1 = p
    H2, a2, f2, g2 = q
    N = len(H1)
    term_H = np.mean(np.sum((H1 - H2) ** 2, axis=1))
    term_a = np.mean((a1 - a2) ** 2)
    term_f = np.mean((f1 - f2) ** 2)
    term_g = np.sum((g1 - g2) ** 2)
    d2 = term_H + (alpha ** 2) * term_a + (beta ** 2) * term_f + (gamma ** 2) * term_g
    return float(np.sqrt(max(0.0, d2)))


def knn_graph_phase1(
    patterns: List[Phase1Pattern],
    k: int,
    alpha: float,
    beta: float,
    gamma: float,
) -> nx.Graph:
    """kNN sobre patrones Phase1 con distancia ponderada."""
    G = nx.Graph()
    m = len(patterns)
    G.add_nodes_from(range(m))
    k_actual = min(k, m - 1)
    if k_actual < 1:
        return G
    for i in range(m):
        dists = [
            (pattern_dist_phase1(patterns[i], patterns[j], alpha, beta, gamma), j)
            for j in range(m)
            if j != i
        ]
        dists.sort(key=lambda x: x[0])
        for _, j in dists[:k_actual]:
            G.add_edge(i, j)
    return G


def build_phase1_patterns(
```

```
    events: List[NoteEvent],
    H: np.ndarray,
    a_raw: np.ndarray,
    bin_size: float,
    t0: float,
    N: int,
    step: int,
    d_ioi: int = 16,
    Tmax_ioi: float = 2.0,
) -> List[Phase1Pattern]:
    """
    Build list of Phase1 patterns for window length N.
    a and f z-scored over the piece; g_W per window (IOI histogram), normalized sum=1.
    """
    nbins = len(H)
    f = spectral_flux_series(H)
    a = zscore_series(a_raw)
    f = zscore_series(f)
    onset_times = np.array([e[0] for e in events], dtype=float)

    patterns: List[Phase1Pattern] = []
    for start in range(0, nbins - N + 1, step):
        H_block = H[start : start + N, :].copy()
        a_block = a[start : start + N].copy()
        f_block = f[start : start + N].copy()
        t_start = t0 + start * bin_size
        t_end = t0 + (start + N) * bin_size
        g_W = ioi_histogram_for_window(onset_times, t_start, t_end, d=d_ioi, Tmax=Tmax_ioi)
        patterns.append((H_block, a_block, f_block, g_W))
    return patterns


def build_phase1_patterns_beats(
    events: List[NoteEventBeats],
    H: np.ndarray,
    a_raw: np.ndarray,
    bin_size_beats: float,
    u0: float,
    N: int,
    step: int,
    d_ioi: int = 16,
    Tmax_ioi_beats: float = 2.0,
) -> List[Phase1Pattern]:
    """
    Phase1 patterns in beat-time: H(u), a(u), f(u) z-scored; g_W = IOI histogram in beats.
        Tmax_ioi_beats default 2.0.
    """
    nbins = len(H)
    f = spectral_flux_series(H)
    a = zscore_series(a_raw)
    f = zscore_series(f)
    onset_times_u = np.array([e[0] for e in events], dtype=float)

    patterns: List[Phase1Pattern] = []
    for start in range(0, nbins - N + 1, step):
        H_block = H[start : start + N, :].copy()
        a_block = a[start : start + N].copy()
        f_block = f[start : start + N].copy()
```

```
        u_start = u0 + start * bin_size_beats
        u_end = u0 + (start + N) * bin_size_beats
        g_W = ioi_histogram_for_window_beats(
            onset_times_u, u_start, u_end, d=d_ioi, Tmax_beats=Tmax_ioi_beats
        )
        patterns.append((H_block, a_block, f_block, g_W))
    return patterns


# ---------------------------------------------------------------------------
# 6) Tower runner and profile plots
# ---------------------------------------------------------------------------


def _spectral_invariants(G: nx.Graph) -> Tuple[float, float, float, float, float]:
    """Fiedler (lambda2) of normalized Laplacian, average clustering, degree percentiles p25,p50,p75.
        Returns (lambda2, clustering, p25, p50, p75)."""
    if G.number_of_nodes() == 0:
        return 0.0, 0.0, 0.0, 0.0, 0.0
    try:
        L = nx.normalized_laplacian_matrix(G)
        ev = np.linalg.eigvalsh(L.toarray())
        ev = np.sort(ev)
        lambda2 = float(ev[1]) if len(ev) > 1 else 0.0
    except Exception:
        lambda2 = 0.0
    try:
        clustering = nx.average_clustering(G)
    except Exception:
        clustering = 0.0
    degs = [d for _, d in G.degree()]
    p25 = float(np.percentile(degs, 25)) if degs else 0.0
    p50 = float(np.percentile(degs, 50)) if degs else 0.0
    p75 = float(np.percentile(degs, 75)) if degs else 0.0
    return lambda2, clustering, p25, p50, p75


@dataclass
class LevelResult:
    n: int
    N: int
    num_patterns: int
    num_edges: int
    avg_degree: float
    beta0: int
    delta_used: float  # delta (quantile/fixed) o -1 si kNN
    giant_component_frac: float  # |C_max|/V
    lambda2: float = 0.0  # Fiedler (normalized Laplacian)
    clustering: float = 0.0  # average clustering
    degree_p25: float = 0.0
    degree_p50: float = 0.0
    degree_p75: float = 0.0


def run_tower_on_midi(
    path: str,
    p: int,
    n_min: int,
```

```
    n_max: int,
    bin_size: float,
    step: int,
    delta: float,
    cap: int = 300,
    threshold_mode: str = "quantile",
    quantile_q: float = 0.05,
    sample_size: int = 5000,
    knn_k: int = 10,
    alpha: float = 1.0,
    show_heatmap: bool = True,
    save_heatmap: str | None = None,
    rng: np.random.Generator | None = None,
) -> List[LevelResult]:
    """
    Build chroma H(t) + onset density a(t); X(t)=[H(t); alpha*a(t)] dim 13.
    At each level n build G_n by (A) quantile or (B) kNN. Report V, E, avg_degree, beta0,
        giant_component_frac.
    """
    events = parse_midi_notes_seconds(path)
    H = chroma_series_duration(events, bin_size=bin_size)
    a = onset_density_series(events, bin_size=bin_size)
    X = series_with_rhythm(H, a, alpha=alpha)

    if show_heatmap or save_heatmap:
        plot_chroma_heatmap(
            H,
            title=f"{Path(path).name} | chroma (duration-weighted), bin={bin_size}",
            save_path=save_heatmap,
        )

    rng = rng or np.random.default_rng()
    results: List[LevelResult] = []
    for n in range(n_min, n_max + 1):
        N = p**n
        patterns = build_patterns_from_series(X, N=N, step=step)

        if len(patterns) > cap:
            idx = np.linspace(0, len(patterns) - 1, cap).astype(int)
            patterns = [patterns[i] for i in idx]

        if not patterns:
            results.append(
                LevelResult(
                    n=n,
                    N=N,
                    num_patterns=0,
                    num_edges=0,
                    avg_degree=0.0,
                    beta0=0,
                    delta_used=delta,
                    giant_component_frac=0.0,
                )
            )
            continue

        if threshold_mode == "quantile":
            delta_n = adaptive_delta_from_patterns(
```

```
                patterns, q=quantile_q, sample_size=sample_size, rng=rng
            )
            G = proximity_graph(patterns, delta=delta_n)
        elif threshold_mode == "knn":
            G = knn_graph(patterns, k=knn_k)
            delta_n = -1.0  # sentinel: kNN
        else:
            delta_n = delta
            G = proximity_graph(patterns, delta=delta)

        V = len(patterns)
        E = G.number_of_edges()
        avg_deg = (2.0 * E / V) if V > 0 else 0.0
        comps = list(nx.connected_components(G))
        max_comp_size = max(len(c) for c in comps) if comps else 0
        giant_frac = (max_comp_size / V) if V > 0 else 0.0
        results.append(
            LevelResult(
                n=n,
                N=N,
                num_patterns=V,
                num_edges=E,
                avg_degree=avg_deg,
                beta0=len(comps),
                delta_used=delta_n,
                giant_component_frac=giant_frac,
            )
        )
    return results


def run_tower_phase1(
    path: str,
    p: int,
    n_min: int,
    n_max: int,
    bin_size: float,
    step: int,
    cap: int,
    knn_k: int,
    alpha: float,
    beta: float,
    gamma: float,
    d_ioi: int = 16,
    Tmax_ioi: float = 2.0,
    time_axis: str = "seconds",
    bin_size_beats: float = 1.0 / 12.0,
    Tmax_ioi_beats: float = 2.0,
) -> List[LevelResult]:
    """
    Phase I: H, a (z-score), f (spectral flux z-score), g_W (IOI hist per window).
    Distance d^2 = mean||H-H'||^2 + alpha^2 mean|a-a'|^2 + beta^2 mean|f-f'|^2 + gamma^2 ||g_W-g'_W||^2.
    kNN graph only.
    time_axis: 'seconds' (default) or 'beats'. If beats, bin_size_beats and Tmax_ioi_beats (default 2.0
        beats) used.
    """
    if time_axis == "beats":
        events_beats = parse_midi_notes_beats(path)
```

```
        if not events_beats:
            return []
        H = chroma_series_duration_beats(events_beats, bin_size_beats=bin_size_beats)
        a_raw = onset_density_series_beats(events_beats, bin_size_beats=bin_size_beats)
        u0 = events_beats[0][0]
        results: List[LevelResult] = []
        for n in range(n_min, n_max + 1):
            N = p**n
            patterns = build_phase1_patterns_beats(
                events_beats,
                H,
                a_raw,
                bin_size_beats,
                u0,
                N=N,
                step=step,
                d_ioi=d_ioi,
                Tmax_ioi_beats=Tmax_ioi_beats,
            )
            if len(patterns) > cap:
                idx = np.linspace(0, len(patterns) - 1, cap).astype(int)
                patterns = [patterns[i] for i in idx]
            if not patterns:
                results.append(
                    LevelResult(
                        n=n, N=N, num_patterns=0, num_edges=0, avg_degree=0.0,
                        beta0=0, delta_used=-1.0, giant_component_frac=0.0,
                    )
                )
                continue
            G = knn_graph_phase1(patterns, knn_k, alpha, beta, gamma)
            V = len(patterns)
            E = G.number_of_edges()
            avg_deg = (2.0 * E / V) if V > 0 else 0.0
            comps = list(nx.connected_components(G))
            max_comp = max(len(c) for c in comps) if comps else 0
            giant_frac = (max_comp / V) if V > 0 else 0.0
            lam2, clust, dp25, dp50, dp75 = _spectral_invariants(G)
            results.append(
                LevelResult(
                    n=n, N=N, num_patterns=V, num_edges=E, avg_degree=avg_deg,
                    beta0=len(comps), delta_used=-1.0, giant_component_frac=giant_frac,
                    lambda2=lam2, clustering=clust, degree_p25=dp25, degree_p50=dp50, degree_p75=dp75,
                )
            )
        return results

    events = parse_midi_notes_seconds(path)
    H = chroma_series_duration(events, bin_size=bin_size)
    a_raw = onset_density_series(events, bin_size=bin_size)
    t0 = events[0][0] if events else 0.0

    results = []
    for n in range(n_min, n_max + 1):
        N = p**n
        patterns = build_phase1_patterns(
            events, H, a_raw, bin_size, t0, N=N, step=step, d_ioi=d_ioi, Tmax_ioi=Tmax_ioi
        )
```

```
        if len(patterns) > cap:
            idx = np.linspace(0, len(patterns) - 1, cap).astype(int)
            patterns = [patterns[i] for i in idx]
        if not patterns:
            results.append(
                LevelResult(
                    n=n, N=N, num_patterns=0, num_edges=0, avg_degree=0.0,
                    beta0=0, delta_used=-1.0, giant_component_frac=0.0,
                )
            )
            continue
        G = knn_graph_phase1(patterns, knn_k, alpha, beta, gamma)
        V = len(patterns)
        E = G.number_of_edges()
        avg_deg = (2.0 * E / V) if V > 0 else 0.0
        comps = list(nx.connected_components(G))
        max_comp = max(len(c) for c in comps) if comps else 0
        giant_frac = (max_comp / V) if V > 0 else 0.0
        lam2, clust, dp25, dp50, dp75 = _spectral_invariants(G)
        results.append(
            LevelResult(
                n=n, N=N, num_patterns=V, num_edges=E, avg_degree=avg_deg,
                beta0=len(comps), delta_used=-1.0, giant_component_frac=giant_frac,
                lambda2=lam2, clustering=clust, degree_p25=dp25, degree_p50=dp50, degree_p75=dp75,
            )
        )
    return results


def run_tower_on_beats(
    path: str,
    p: int,
    n_min: int,
    n_max: int,
    bin_size_beats: float,
    step: int,
    cap: int,
    knn_k: int,
    alpha: float,
) -> List[LevelResult]:
    """
    Beat-based binning: H(u) and a(u) with bin_size_beats; X(u)=[H(u); alpha*a(u)].
    kNN only. Same tower as baseline (pattern_dist, knn_graph).
    """
    events = parse_midi_notes_beats(path)
    if not events:
        return []
    H = chroma_series_duration_beats(events, bin_size_beats=bin_size_beats)
    a = onset_density_series_beats(events, bin_size_beats=bin_size_beats)
    X = series_with_rhythm(H, a, alpha=alpha)

    results: List[LevelResult] = []
    for n in range(n_min, n_max + 1):
        N = p**n
        patterns = build_patterns_from_series(X, N=N, step=step)
        if len(patterns) > cap:
            idx = np.linspace(0, len(patterns) - 1, cap).astype(int)
            patterns = [patterns[i] for i in idx]
```

```
        if not patterns:
            results.append(
                LevelResult(
                    n=n, N=N, num_patterns=0, num_edges=0, avg_degree=0.0,
                    beta0=0, delta_used=-1.0, giant_component_frac=0.0,
                )
            )
            continue
        G = knn_graph(patterns, k=knn_k)
        V = len(patterns)
        E = G.number_of_edges()
        avg_deg = (2.0 * E / V) if V > 0 else 0.0
        comps = list(nx.connected_components(G))
        max_comp = max(len(c) for c in comps) if comps else 0
        giant_frac = (max_comp / V) if V > 0 else 0.0
        results.append(
            LevelResult(
                n=n, N=N, num_patterns=V, num_edges=E, avg_degree=avg_deg,
                beta0=len(comps), delta_used=-1.0, giant_component_frac=giant_frac,
            )
        )
    return results


def save_results_csv(
    results: List[LevelResult],
    filepath: str,
    source: str,
    p: int,
    threshold_mode: str = "quantile",
) -> None:
    """CSV: n, N, V, E, avg_degree, beta0, delta_used, giant_component_frac [, lambda2, clustering,
         degree_p25, degree_p50, degree_p75]."""
    with open(filepath, "w", newline="") as f:
        w = csv.writer(f)
        has_spectral = getattr(results[0], "lambda2", None) is not None if results else False
        row0 = ["n", "N", "V", "E", "avg_degree", "beta0", "delta_used", "giant_component_frac"]
        if has_spectral:
            row0 += ["lambda2", "clustering", "degree_p25", "degree_p50", "degree_p75"]
        w.writerow(row0)
        for r in results:
            du = r.delta_used if (threshold_mode == "quantile" or r.delta_used >= 0) else -1
            row = [r.n, r.N, r.num_patterns, r.num_edges, round(r.avg_degree, 6), r.beta0, du,
                 round(r.giant_component_frac, 6)]
            if has_spectral:
                row += [round(getattr(r, "lambda2", 0), 6), round(getattr(r, "clustering", 0), 6),
                        round(getattr(r, "degree_p25", 0), 4), round(getattr(r, "degree_p50", 0), 4),
                            round(getattr(r, "degree_p75", 0), 4)]
            w.writerow(row)


def save_results_csv_phase2(
    results: List[LevelResult],
    filepath: str,
    knn_k: int,
    k_over_V_threshold: float = 0.15,
) -> None:
    """CSV Phase II: n, N, V, E, avg_degree, beta0, giant_component_frac, k_over_V, no_informativo."""
```

```
    with open(filepath, "w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["n", "N", "V", "E", "avg_degree", "beta0", "giant_component_frac", "k_over_V",
             "no_informativo"])
        for r in results:
            V = r.num_patterns
            k_over_V = (knn_k / V) if V > 0 else 0.0
            no_inf = "yes" if k_over_V > k_over_V_threshold else "no"
            w.writerow([
                r.n, r.N, V, r.num_edges, round(r.avg_degree, 6), r.beta0,
                round(r.giant_component_frac, 6), round(k_over_V, 6), no_inf,
            ])


def plot_profile(
    results: List[LevelResult],
    title: str,
    save_prefix: str | None = None,
    run_label: str = "",
) -> None:
    """Plot |D_n|, beta0 y avg_degree vs n. Si save_prefix y run_label, guarda con nombre único."""
    ns = [r.n for r in results]
    sizes = [r.num_patterns for r in results]
    b0 = [r.beta0 for r in results]
    avg_deg = [r.avg_degree for r in results]
    tag = f"{save_prefix}{run_label}_" if (save_prefix and run_label) else (save_prefix or "")

    plt.figure()
    plt.plot(ns, sizes, marker="o")
    plt.xlabel("n")
    plt.ylabel("|D_n| (windows sampled)")
    plt.title(title + " : |D_n|")
    plt.tight_layout()
    if save_prefix:
        plt.savefig(f"{tag}Dn.png", dpi=120)
        plt.close()
    else:
        plt.show()

    plt.figure()
    plt.plot(ns, b0, marker="o")
    plt.xlabel("n")
    plt.ylabel("beta0 (#components)")
    plt.title(title + " : beta0")
    plt.tight_layout()
    if save_prefix:
        plt.savefig(f"{tag}beta0.png", dpi=120)
        plt.close()
    else:
        plt.show()

    plt.figure()
    plt.plot(ns, avg_deg, marker="o")
    plt.xlabel("n")
    plt.ylabel("avg_degree (2|E|/|V|)")
    plt.title(title + " : avg_degree")
    plt.tight_layout()
    if save_prefix:
```

```
        plt.savefig(f"{tag}avg_degree.png", dpi=120)
        plt.close()
    else:
        plt.show()

    plt.figure()
    gcf = [r.giant_component_frac for r in results]
    plt.plot(ns, gcf, marker="o")
    plt.xlabel("n")
    plt.ylabel("giant_component_frac (|C_max|/V)")
    plt.title(title + " : giant_component_frac")
    plt.tight_layout()
    if save_prefix:
        plt.savefig(f"{tag}giant_component_frac.png", dpi=120)
        plt.close()
    else:
        plt.show()


def plot_profile_compare(
    res_p2: List[LevelResult],
    res_p3: List[LevelResult],
    title_prefix: str,
    save_prefix: str,
    run_label: str,
) -> None:
    """Cuatro figuras: |D_n|(n), avg_degree(n), beta0(n), giant_component_frac(n) con curvas p=2 y
        p=3."""
    ns2 = [r.n for r in res_p2]
    ns3 = [r.n for r in res_p3]
    tag = f"{save_prefix}{run_label}_"
    plt.figure()
    plt.plot(ns2, [r.num_patterns for r in res_p2], marker="o", label="p=2")
    plt.plot(ns3, [r.num_patterns for r in res_p3], marker="s", label="p=3")
    plt.xlabel("n")
    plt.ylabel("|D_n| (V)")
    plt.title(f"{title_prefix} : |D_n| (p=2 vs p=3)")
    plt.legend()
    plt.tight_layout()
    plt.savefig(f"{tag}Dn.png", dpi=120)
    plt.close()

    plt.figure()
    plt.plot(ns2, [r.avg_degree for r in res_p2], marker="o", label="p=2")
    plt.plot(ns3, [r.avg_degree for r in res_p3], marker="s", label="p=3")
    plt.xlabel("n")
    plt.ylabel("avg_degree")
    plt.title(f"{title_prefix} : avg_degree (p=2 vs p=3)")
    plt.legend()
    plt.tight_layout()
    plt.savefig(f"{tag}avg_degree.png", dpi=120)
    plt.close()

    plt.figure()
    plt.plot(ns2, [r.beta0 for r in res_p2], marker="o", label="p=2")
    plt.plot(ns3, [r.beta0 for r in res_p3], marker="s", label="p=3")
    plt.xlabel("n")
    plt.ylabel("beta0")
```

```
    plt.title(f"{title_prefix} : beta0 (p=2 vs p=3)")
    plt.legend()
    plt.tight_layout()
    plt.savefig(f"{tag}beta0.png", dpi=120)
    plt.close()

    plt.figure()
    plt.plot(ns2, [r.giant_component_frac for r in res_p2], marker="o", label="p=2")
    plt.plot(ns3, [r.giant_component_frac for r in res_p3], marker="s", label="p=3")
    plt.xlabel("n")
    plt.ylabel("giant_component_frac")
    plt.title(f"{title_prefix} : giant_component_frac (p=2 vs p=3)")
    plt.legend()
    plt.tight_layout()
    plt.savefig(f"{tag}giant_component_frac.png", dpi=120)
    plt.close()

    if getattr(res_p2[0], "lambda2", 0) != 0 or getattr(res_p3[0], "lambda2", 0) != 0:
        plt.figure()
        plt.plot(ns2, [getattr(r, "lambda2", 0) for r in res_p2], marker="o", label="p=2")
        plt.plot(ns3, [getattr(r, "lambda2", 0) for r in res_p3], marker="s", label="p=3")
        plt.xlabel("n")
        plt.ylabel("lambda2 (Fiedler)")
        plt.title(f"{title_prefix} : lambda2 (p=2 vs p=3)")
        plt.legend()
        plt.tight_layout()
        plt.savefig(f"{tag}lambda2.png", dpi=120)
        plt.close()
    if getattr(res_p2[0], "clustering", 0) != 0 or getattr(res_p3[0], "clustering", 0) != 0:
        plt.figure()
        plt.plot(ns2, [getattr(r, "clustering", 0) for r in res_p2], marker="o", label="p=2")
        plt.plot(ns3, [getattr(r, "clustering", 0) for r in res_p3], marker="s", label="p=3")
        plt.xlabel("n")
        plt.ylabel("clustering")
        plt.title(f"{title_prefix} : clustering (p=2 vs p=3)")
        plt.legend()
        plt.tight_layout()
        plt.savefig(f"{tag}clustering.png", dpi=120)
        plt.close()


# ---------------------------------------------------------------------------
# 6b) Phase I Bach: multi-observable (H, a, f, g_W), configs A/B/C, reporte Delta, Delta_G, ÉXITO
# ---------------------------------------------------------------------------


def _n_max_for_prime(p: int, max_window: int = 2000) -> int:
    """Max level n such that p^n <= max_window (for control primes)."""
    n = 1
    while n < 20 and p ** n <= max_window:
        n += 1
    return max(1, n - 1)


def run_phase1_bach_report(
    real_path: str,
    save_prefix: str,
    bin_size: float = 0.05,
```

```
    step: int = 2,
    cap: int = 300,
    knn_k_list: Optional[List[int]] = None,
    n_max_p2: int = 6,
    n_max_p3: int = 5,
    time_axis: str = "seconds",
    bin_size_beats: float = 1.0 / 12.0,
    Tmax_ioi_beats: float = 2.0,
    k_over_V_threshold: float = 0.15,
    control_primes: Optional[List[int]] = None,
) -> None:
    """
    Phase I solo para Bach: configs (A) alpha=1,beta=0,gamma=0; (B) 1,1,0; (C) 1,1,1.
    k=8,10,12; p=2 n_max=6, p=3 n_max=5. Reporta Delta(n)=beta0_p2(n)-beta0_p3(n),
    giant_component_frac p=2/p=3, Delta_G(n); ÉXITO=1 si (existe n con ≠Delta0 en ≥2 k) o
        (|Delta_G≥|0.10 en algún n).
    time_axis='seconds' (default) o 'beats'; si beats, salidas con sufijo _beats y reporte en
        phase2_beats_report.txt.
    """
    if knn_k_list is None:
        knn_k_list = [8, 10, 12]
    configs: List[Tuple[str, float, float, float]] = [
        ("A", 1.0, 0.0, 0.0),
        ("B", 1.0, 1.0, 0.0),
        ("C", 1.0, 1.0, 1.0),
    ]
    common_n_max = min(n_max_p3, n_max_p2)
    use_beats = time_axis == "beats"
    suffix = "_beats" if use_beats else ""
    report_filename = f"{save_prefix}phase2_beats_report.txt" if use_beats else
        f"{save_prefix}phase1_bach_report.txt"

    all_delta_nonzero_count: Dict[Tuple[str, int], List[int]] = {}
    any_delta_G_ge_010 = False
    config_any_delta_G_ge_010: Dict[str, bool] = {c[0]: False for c in configs}
    report_lines: List[str] = []

    for config_name, alpha, beta, gamma in configs:
        for k in knn_k_list:
            if use_beats:
                res_p2 = run_tower_phase1(
                    real_path,
                    p=2,
                    n_min=1,
                    n_max=n_max_p2,
                    bin_size=bin_size,
                    step=step,
                    cap=cap,
                    knn_k=k,
                    alpha=alpha,
                    beta=beta,
                    gamma=gamma,
                    time_axis="beats",
                    bin_size_beats=bin_size_beats,
                    Tmax_ioi_beats=Tmax_ioi_beats,
                )
                res_p3 = run_tower_phase1(
                    real_path,
```

```
                    p=3,
                    n_min=1,
                    n_max=n_max_p3,
                    bin_size=bin_size,
                    step=step,
                    cap=cap,
                    knn_k=k,
                    alpha=alpha,
                    beta=beta,
                    gamma=gamma,
                    time_axis="beats",
                    bin_size_beats=bin_size_beats,
                    Tmax_ioi_beats=Tmax_ioi_beats,
                )
            else:
                res_p2 = run_tower_phase1(
                    real_path,
                    p=2, n_min=1, n_max=n_max_p2,
                    bin_size=bin_size, step=step, cap=cap,
                    knn_k=k, alpha=alpha, beta=beta, gamma=gamma,
                )
                res_p3 = run_tower_phase1(
                    real_path,
                    p=3, n_min=1, n_max=n_max_p3,
                    bin_size=bin_size, step=step, cap=cap,
                    knn_k=k, alpha=alpha, beta=beta, gamma=gamma,
                )
            if save_prefix:
                save_results_csv(
                    res_p2,
                    f"{save_prefix}{config_name}_tower_real_bach_k{k}_p2{suffix}.csv",
                    real_path,
                    2,
                    "kNN",
                )
                save_results_csv(
                    res_p3,
                    f"{save_prefix}{config_name}_tower_real_bach_k{k}_p3{suffix}.csv",
                    real_path,
                    3,
                    "kNN",
                )
                plot_profile_compare(
                    res_p2,
                    res_p3,
                    f"Phase1 {config_name} Bach k={k}" + (" (beats)" if use_beats else ""),
                    save_prefix,
                    f"{config_name}_real_bach_k{k}{suffix}",
                )
                # Control primes (e.g. p=5,7): run tower and save CSV; no plot.
                if control_primes:
                    for p_val in control_primes:
                        n_max_p = _n_max_for_prime(p_val)
                        if use_beats:
                            res_cp = run_tower_phase1(
                                real_path,
                                p=p_val, n_min=1, n_max=n_max_p,
                                bin_size=bin_size, step=step, cap=cap,
```

```
                                knn_k=k, alpha=alpha, beta=beta, gamma=gamma,
                                time_axis="beats",
                                bin_size_beats=bin_size_beats,
                                Tmax_ioi_beats=Tmax_ioi_beats,
                            )
                        else:
                            res_cp = run_tower_phase1(
                                real_path,
                                p=p_val, n_min=1, n_max=n_max_p,
                                bin_size=bin_size, step=step, cap=cap,
                                knn_k=k, alpha=alpha, beta=beta, gamma=gamma,
                            )
                        save_results_csv(
                            res_cp,
                            f"{save_prefix}{config_name}_tower_real_bach_k{k}_p{p_val}{suffix}.csv",
                            real_path,
                            p_val,
                            "kNN",
                        )

            beta0_p2_by_n = {r.n: r.beta0 for r in res_p2}
            beta0_p3_by_n = {r.n: r.beta0 for r in res_p3}
            giant_p2_by_n = {r.n: r.giant_component_frac for r in res_p2}
            giant_p3_by_n = {r.n: r.giant_component_frac for r in res_p3}
            delta_vec = []
            delta_G_vec = []
            ns_common = []
            for n in range(1, common_n_max + 1):
                if n in beta0_p2_by_n and n in beta0_p3_by_n:
                    ns_common.append(n)
                    delta_vec.append(beta0_p2_by_n[n] - beta0_p3_by_n[n])
                    dg = giant_p2_by_n.get(n, 0) - giant_p3_by_n.get(n, 0)
                    delta_G_vec.append(dg)
                    if abs(dg) >= 0.10:
                        any_delta_G_ge_010 = True
                        config_any_delta_G_ge_010[config_name] = True
            for nn in range(1, common_n_max + 1):
                all_delta_nonzero_count.setdefault((config_name, nn), []).append(
                    1 if (nn <= len(delta_vec) and delta_vec[nn - 1] != 0) else 0
                )

            print(f"--- Phase1 Bach config={config_name} k={k}" + (" (beats)" if use_beats else "") + "
                 ---")
            print("Delta(n) = beta0_p2(n)-beta0_p3(n):", delta_vec, "(n =", ns_common, ")")
            print("giant_component_frac p=2:", [round(giant_p2_by_n.get(n, 0), 4) for n in ns_common])
            print("giant_component_frac p=3:", [round(giant_p3_by_n.get(n, 0), 4) for n in ns_common])
            print("Delta_G(n):", [round(x, 4) for x in delta_G_vec])

            if use_beats:
                report_lines.append(f"--- config={config_name} k={k} ---")
                report_lines.append("Delta(n): " + str(delta_vec))
                report_lines.append("Delta_G(n): " + str([round(x, 4) for x in delta_G_vec]))
                for r in res_p2:
                    V = r.num_patterns
                    kv = (k / V) if V > 0 else 0.0
                    no_inf = "no_informativo" if kv > k_over_V_threshold else "ok"
                    report_lines.append(f"  p=2 n={r.n} V={V} k/V={round(kv, 4)} {no_inf}")
                for r in res_p3:
```

```
                    V = r.num_patterns
                    kv = (k / V) if V > 0 else 0.0
                    no_inf = "no_informativo" if kv > k_over_V_threshold else "ok"
                    report_lines.append(f"  p=3 n={r.n} V={V} k/V={round(kv, 4)} {no_inf}")

        if use_beats:
            config_success = 0
            for (cfg, n), counts in all_delta_nonzero_count.items():
                if cfg == config_name and sum(counts) >= 2:
                    config_success = 1
                    break
            if config_success == 0 and config_any_delta_G_ge_010.get(config_name, False):
                config_success = 1
            report_lines.append(f"ÉXITO config {config_name} = {config_success}")
            report_lines.append("")

    success = 0
    for (config_name, n), counts in all_delta_nonzero_count.items():
        if sum(counts) >= 2:
            success = 1
            break
    if not success and any_delta_G_ge_010:
        success = 1
    print("========== Phase I Bach ÉXITO =" if not use_beats else "========== Phase I Bach (beats) É
        XITO =", success, "==========")
    if save_prefix:
        with open(report_filename, "w") as f:
            if use_beats:
                f.write("Phase I Bach time-axis=beats: Delta(n), Delta_G(n) por (config,k); k/V por
                    nivel; no_informativo si k/V>0.15.\n\n")
                f.write("\n".join(report_lines))
                f.write(f"\nÉXITO global = {success}\n")
            else:
                f.write("Phase I Bach: Delta(n), Delta_G(n), ÉXITO (see stdout for full vectors).\n")
                f.write(f"ÉXITO = {success}\n")


# ----------------------------------------------------------------------------
# 6c) Phase II Bach: beat-based binning, alpha sweep, criterio binario, k/V
# ----------------------------------------------------------------------------


def run_phase2_bach_report(
    real_path: str,
    save_prefix: str,
    bin_size_beats: float = 1.0 / 12.0,
    step: int = 2,
    cap: int = 300,
    knn_k_list: Optional[List[int]] = None,
    alpha_list: Optional[List[float]] = None,
    n_max_p2: int = 6,
    n_max_p3: int = 5,
    n_min_report: int = 2,
    k_over_V_threshold: float = 0.15,
) -> None:
    """
    Phase II: beat-based binning H(u), a(u); baseline kNN for BWV1007.
    k=8,10,12; alpha in {0, 0.5, 1, 2, 4}; p=2 (n_max=6), p=3 (n_max=5).
```

```
    Report: Delta(n)=beta0_p2-beta0_p3, Delta_G(n) for n>=n_min_report; tables with k/V, no informativo if k/V>0.15.
    ÉXITO=1 si (existe n>=2 con Delta(n)≠0 en al menos 2 k) o (|Delta_G(n)|>=0.10 en algún n>=2).
    """
    if knn_k_list is None:
        knn_k_list = [8, 10, 12]
    if alpha_list is None:
        alpha_list = [0.0, 0.5, 1.0, 2.0, 4.0]
    common_n_max = min(n_max_p3, n_max_p2)

    all_delta_nonzero_count: Dict[Tuple[float, int], List[int]] = {}  # (alpha, n) -> count of k with Delta(n)!=0
    any_delta_G_ge_010 = False

    for alpha in alpha_list:
        for k in knn_k_list:
            res_p2 = run_tower_on_beats(
                real_path,
                p=2,
                n_min=1,
                n_max=n_max_p2,
                bin_size_beats=bin_size_beats,
                step=step,
                cap=cap,
                knn_k=k,
                alpha=alpha,
            )
            res_p3 = run_tower_on_beats(
                real_path,
                p=3,
                n_min=1,
                n_max=n_max_p3,
                bin_size_beats=bin_size_beats,
                step=step,
                cap=cap,
                knn_k=k,
                alpha=alpha,
            )
            if save_prefix:
                alpha_tag = str(alpha).replace(".", "p")
                save_results_csv_phase2(
                    res_p2,
                    f"{save_prefix}phase2_bach_k{k}_a{alpha_tag}_p2.csv",
                    knn_k=k,
                    k_over_V_threshold=k_over_V_threshold,
                )
                save_results_csv_phase2(
                    res_p3,
                    f"{save_prefix}phase2_bach_k{k}_a{alpha_tag}_p3.csv",
                    knn_k=k,
                    k_over_V_threshold=k_over_V_threshold,
                )

            beta0_p2_by_n = {r.n: r.beta0 for r in res_p2}
            beta0_p3_by_n = {r.n: r.beta0 for r in res_p3}
            giant_p2_by_n = {r.n: r.giant_component_frac for r in res_p2}
            giant_p3_by_n = {r.n: r.giant_component_frac for r in res_p3}
            delta_vec = []
```

```
            delta_G_vec = []
            ns_common = []
            for n in range(1, common_n_max + 1):
                if n in beta0_p2_by_n and n in beta0_p3_by_n:
                    ns_common.append(n)
                    d = beta0_p2_by_n[n] - beta0_p3_by_n[n]
                    dg = giant_p2_by_n.get(n, 0) - giant_p3_by_n.get(n, 0)
                    delta_vec.append(d)
                    delta_G_vec.append(dg)
                    if n >= n_min_report and abs(dg) >= 0.10:
                        any_delta_G_ge_010 = True
            for nn in range(1, common_n_max + 1):
                idx = nn - 1
                delta_val = delta_vec[idx] if idx < len(delta_vec) else 0
                all_delta_nonzero_count.setdefault((alpha, nn), []).append(
                    1 if (idx < len(delta_vec) and delta_val != 0) else 0
                )

            print(f"--- Phase II Bach k={k} alpha={alpha} ---")
            print("Delta(n) = beta0_p2(n)-beta0_p3(n):", delta_vec, "(n =", ns_common, ")")
            print("Delta_G(n):", [round(x, 4) for x in delta_G_vec])
            # Table with k/V and no informativo for n>=1
            print("n\tN\tV\tbeta0\tgiant_frac\tk/V\tno_informativo")
            for r in res_p2:
                V = r.num_patterns
                k_over_V = (k / V) if V > 0 else 0.0
                no_inf = "yes" if k_over_V > k_over_V_threshold else "no"
                print(f"p=2
                    {r.n}\t{r.N}\t{V}\t{r.beta0}\t{r.giant_component_frac:.4f}\t{k_over_V:.4f}\t{no_inf}")
            for r in res_p3:
                V = r.num_patterns
                k_over_V = (k / V) if V > 0 else 0.0
                no_inf = "yes" if k_over_V > k_over_V_threshold else "no"
                print(f"p=3
                    {r.n}\t{r.N}\t{V}\t{r.beta0}\t{r.giant_component_frac:.4f}\t{k_over_V:.4f}\t{no_inf}")

    success = 0
    for (alpha, n), counts in all_delta_nonzero_count.items():
        if n >= n_min_report and sum(counts) >= 2:
            success = 1
            break
    if not success and any_delta_G_ge_010:
        success = 1
    print("========== Phase II Bach ÉXITO =", success, "==========")
    if save_prefix:
        with open(f"{save_prefix}phase2_bach_report.txt", "w") as f:
            f.write("Phase II Bach (beat-based): Delta(n), Delta_G(n), k/V, no informativo
                (k/V>0.15).\n")
            f.write(f"ÉXITO = {success}\n")


# ----------------------------------------------------------------------------
# 7) Reporte final: tablas + 3 bullets (sin interpretación musical fuerte)
# ----------------------------------------------------------------------------


def report_baseline_final(
    baseline_runs: List[Tuple[str, int, List[LevelResult], List[LevelResult]]],
```

```
    threshold_mode: str,
    quantile_q: float,
    knn_k_list: List[int],
    alpha: float = 1.0,
) -> List[str]:
    """Reporte solo hechos cuantitativos: estabilidad en k, separación p=2 vs p=3 en toys, Bach."""
    lines = [
        "========== REPORTE BASELINE (solo hechos cuantitativos) ==========",
        f"Threshold: {threshold_mode}; k values: {knn_k_list}; alpha (onset density): {alpha}.",
        "",
    ]
    # Tablas resumidas por (source, k): p=2 y p=3
    for source_label, k, res_p2, res_p3 in baseline_runs:
        lines.append(f"--- {source_label} k={k} ---")
        lines.append("p=2: n\tN\tV\tE\tavg_degree\tbeta0\tgiant_component_frac")
        for r in res_p2:
            lines.append(f"
                {r.n}\t{r.N}\t{r.num_patterns}\t{r.num_edges}\t{r.avg_degree:.4f}\t{r.beta0}\t{r.giant_component_frac:.4
        lines.append("p=3: n\tN\tV\tE\tavg_degree\tbeta0\tgiant_component_frac")
        for r in res_p3:
            lines.append(f"
                {r.n}\t{r.N}\t{r.num_patterns}\t{r.num_edges}\t{r.avg_degree:.4f}\t{r.beta0}\t{r.giant_component_frac:.4
        lines.append("")

    lines.append("--- Hechos cuantitativos ---")
    # 1) Estabilidad respecto a k
    by_source: Dict[str, List[Tuple[int, List[LevelResult], List[LevelResult]]]] = {}
    for source_label, k, res_p2, res_p3 in baseline_runs:
        by_source.setdefault(source_label, []).append((k, res_p2, res_p3))
    stab = []
    for source_label, k_runs in by_source.items():
        if len(k_runs) < 2:
            stab.append(f"• {source_label}: solo un k, no se evalúa estabilidad.")
            continue
        ns = [r.n for r in k_runs[0][1]]
        b0_curves = [[r.beta0 for r in res_p2] for _, res_p2, _ in k_runs]
        same_shape = all(len(c) == len(ns) for c in b0_curves)
        b0_range = [max(b0_curves[i][j] for i in range(len(k_runs))) - min(b0_curves[i][j] for i in
            range(len(k_runs))) for j in range(len(ns))]
        max_diff = max(b0_range) if b0_range else 0
        stab.append(f"• {source_label}: curvas beta0(n) para k={[x[0] for x in k_runs]}; max diferencia
            entre k en algún n = {max_diff}. {'Curvas similares.' if max_diff <= 2 else 'Curvas
            difieren.'}")
    lines.extend(stab)

    # 2) Separación p=2 vs p=3 en toys (esperado: toy_binary favorece p=2; toy_ternary favorece p=3)
    for source_label, k_runs in by_source.items():
        if source_label not in ("toy_binary", "toy_ternary"):
            continue
        _, res_p2, res_p3 = k_runs[0]
        b0_p2 = [r.beta0 for r in res_p2]
        b0_p3 = [r.beta0 for r in res_p3]
        # toy_binary "favorece" p=2 si en algún nivel p=2 tiene menos componentes que p=3 (más
            conectado)
        p2_menos = sum(1 for i in range(min(len(b0_p2), len(b0_p3))) if b0_p2[i] < b0_p3[i])
        p3_menos = sum(1 for i in range(min(len(b0_p2), len(b0_p3))) if b0_p2[i] > b0_p3[i])
        if source_label == "toy_binary":
```

```
            lines.append(f"• toy_binary: en {p2_menos} niveles beta0(p=2)<beta0(p=3), en {p3_menos} niveles beta0(p=2)>beta0(p=3). Esperado: más niveles con p=2 más conectado (p=2 favorecido).")
        else:
            lines.append(f"• toy_ternary: en {p2_menos} niveles beta0(p=2)<beta0(p=3), en {p3_menos} niveles beta0(p=2)>beta0(p=3). Esperado: más niveles con p=3 más conectado (p=3 favorecido).")

    # 3) Bach: diferencia consistente entre p=2 y p=3
    bach_runs = [(k, r2, r3) for src, k, r2, r3 in baseline_runs if src == "real_bach"]
    if bach_runs:
        _, res_p2, res_p3 = bach_runs[0]
        b0_2 = [r.beta0 for r in res_p2]
        b0_3 = [r.beta0 for r in res_p3]
        diff = [b0_2[i] - b0_3[i] for i in range(min(len(b0_2), len(b0_3)))]
        lines.append(f"• Bach real: diferencia beta0(p=2)-beta0(p=3) por nivel n: {diff}. Diferencia consistente (mismo signo en todos los niveles): {'sí' if all(d >= 0 for d in diff) or all(d <= 0 for d in diff) else 'no'}.")
    else:
        lines.append("• Bach real: no ejecutado.")
    return lines


# ---------------------------------------------------------------------------
# 8) Main: synthetic tests, optional real MIDI
# ---------------------------------------------------------------------------


def main() -> None:
    parser = argparse.ArgumentParser(
        description="Profinite Echo: precision tower on MIDI chroma (p=2 vs p=3)"
    )
    parser.add_argument(
        "midi",
        nargs="?",
        default="",
        help="Optional: path to real MIDI (if omitted, only synthetic runs)",
    )
    parser.add_argument(
        "--no-synthetic",
        action="store_true",
        help="Skip synthetic MIDI generation and tests",
    )
    parser.add_argument(
        "--phase1-only",
        action="store_true",
        help="Run only Phase I (Bach, configs A/B/C) and skip baseline tower",
    )
    parser.add_argument(
        "--phase2-only",
        action="store_true",
        help="Run only Phase II (Bach, beat-based binning, alpha sweep)",
    )
    parser.add_argument(
        "--bin-size",
        type=float,
        default=0.05,
        help="Chroma bin size in seconds (default 0.05)",
```

```
    )
    parser.add_argument(
        "--time-axis",
        type=str,
        choices=["seconds", "beats"],
        default="seconds",
        help="Time axis for binning: seconds (default) or beats",
    )
    parser.add_argument(
        "--bin-beats",
        type=float,
        default=1.0 / 12.0,
        help="Bin size in beats, only if --time-axis=beats (default 1/12)",
    )
    parser.add_argument(
        "--bin-size-beats",
        type=float,
        default=1.0 / 12.0,
        help="Phase II: bin size in beats (default 1/12)",
    )
    parser.add_argument(
        "--step",
        type=int,
        default=2,
        help="Window slide step (default 2)",
    )
    parser.add_argument(
        "--delta",
        type=float,
        default=0.35,
        help="Proximity threshold only when --threshold=fixed (default 0.35)",
    )
    parser.add_argument(
        "--threshold",
        type=str,
        choices=["quantile", "knn", "fixed"],
        default="knn",
        help="knn (default): kNN graph; quantile: delta_n=Q_q; fixed: delta fijo",
    )
    parser.add_argument(
        "--quantile",
        type=float,
        default=0.05,
        help="Quantile q for --threshold=quantile (default 0.05)",
    )
    parser.add_argument(
        "--sample-size",
        type=int,
        default=5000,
        help="Max random pairs for quantile sampling (default 5000)",
    )
    parser.add_argument(
        "--knn-k",
        type=int,
        nargs="+",
        default=[8, 10, 12],
        help="k values for --threshold=knn (default: 8 10 12)",
    )
```

```
    parser.add_argument(
        "--alpha",
        type=float,
        default=1.0,
        help="Weight for onset density in X(t)=[H(t); alpha*a(t)] (default 1.0)",
    )
    parser.add_argument(
        "--n-max",
        type=int,
        default=6,
        help="Max tower level n (default 6)",
    )
    parser.add_argument(
        "--cap",
        type=int,
        default=300,
        help="Cap number of patterns per level for graph build (default 300)",
    )
    parser.add_argument(
        "--control-primes",
        type=int,
        nargs="*",
        default=[],
        help="Additional primes for Phase I (e.g. 5 7) as negative controls; default p=2,3 unchanged",
    )
    parser.add_argument(
        "--save",
        type=str,
        default="",
        help="Save plots and CSV to this prefix (e.g. out_) instead of showing",
    )
    args = parser.parse_args()

    n_min, n_max = 1, args.n_max
    bin_size = args.bin_size
    step = args.step
    delta = args.delta
    cap = args.cap
    threshold_mode = args.threshold
    quantile_q = args.quantile
    sample_size = args.sample_size
    knn_k_list = args.knn_k if isinstance(args.knn_k, list) else [args.knn_k]
    if threshold_mode != "knn":
        knn_k_list = [knn_k_list[0]]
    alpha = getattr(args, "alpha", 1.0)
    save_prefix = args.save.strip() or None
    real_path = args.midi.strip()
    if not real_path and Path("bwv1007_prelude.mid").exists():
        real_path = "bwv1007_prelude.mid"

    # Baseline: (source_label, k, res_p2, res_p3) para reporte
    baseline_runs: List[Tuple[str, int, List[LevelResult], List[LevelResult]]] = []

    if args.phase1_only:
        if real_path and Path(real_path).exists():
            run_phase1_bach_report(
                real_path,
                save_prefix or "",
```

```
                bin_size=bin_size,
                step=step,
                cap=cap,
                knn_k_list=knn_k_list,
                n_max_p2=n_max,
                n_max_p3=min(5, n_max),
                time_axis=args.time_axis,
                bin_size_beats=args.bin_beats,
                control_primes=getattr(args, "control_primes", None) or [],
            )
        else:
            print("Phase I requires a valid MIDI path (e.g. bwv1007_prelude.mid).")
        return

    if args.phase2_only:
        if real_path and Path(real_path).exists():
            run_phase2_bach_report(
                real_path,
                save_prefix or "",
                bin_size_beats=args.bin_size_beats,
                step=step,
                cap=cap,
                knn_k_list=knn_k_list,
                alpha_list=[0.0, 0.5, 1.0, 2.0, 4.0],
                n_max_p2=n_max,
                n_max_p3=min(5, n_max),
            )
        else:
            print("Phase II requires a valid MIDI path (e.g. bwv1007_prelude.mid).")
        return

    if not args.no_synthetic:
        write_synthetic_midi("toy_binary.mid", pattern="binary", bars=8)
        write_synthetic_midi("toy_ternary.mid", pattern="ternary", bars=8)

    sources: List[Tuple[str, str, bool]] = []  # (path, label, show_heatmap)
    if not args.no_synthetic:
        sources.append(("toy_binary.mid", "toy_binary", True))
        sources.append(("toy_ternary.mid", "toy_ternary", True))
    if real_path and Path(real_path).exists():
        sources.append((real_path, "real_bach", not args.no_synthetic))

    for ki, k in enumerate(knn_k_list):
        for path, source_label, show_heatmap in sources:
            save_heat = save_prefix and ki == 0
            res_p2 = run_tower_on_midi(
                path,
                p=2,
                n_min=n_min,
                n_max=n_max,
                bin_size=bin_size,
                step=step,
                delta=delta,
                cap=cap,
                threshold_mode=threshold_mode,
                quantile_q=quantile_q,
                sample_size=sample_size,
                knn_k=k,
```

```
                alpha=alpha,
                show_heatmap=show_heatmap and save_prefix is None,
                save_heatmap=f"{save_prefix}chroma_{source_label}.png" if save_heat else None,
            )
            res_p3 = run_tower_on_midi(
                path,
                p=3,
                n_min=1,
                n_max=min(5, n_max),
                bin_size=bin_size,
                step=step,
                delta=delta,
                cap=cap,
                threshold_mode=threshold_mode,
                quantile_q=quantile_q,
                sample_size=sample_size,
                knn_k=k,
                alpha=alpha,
                show_heatmap=False,
            )
            baseline_runs.append((source_label, k, res_p2, res_p3))
            if save_prefix:
                save_results_csv(res_p2, f"{save_prefix}tower_{source_label}_k{k}_p2.csv", path, 2,
                    threshold_mode)
                save_results_csv(res_p3, f"{save_prefix}tower_{source_label}_k{k}_p3.csv", path, 3,
                    threshold_mode)
                plot_profile_compare(res_p2, res_p3, f"{source_label} k={k}", save_prefix,
                    f"{source_label}_k{k}")
            print(f"--- {source_label} k={k} ---")
            print("p=2:", [f"n={r.n} |V|={r.num_patterns} beta0={r.beta0} avg_deg={r.avg_degree:.2f}
                giant_frac={r.giant_component_frac:.3f}" for r in res_p2])
            print("p=3:", [f"n={r.n} beta0={r.beta0} avg_deg={r.avg_degree:.2f}
                giant_frac={r.giant_component_frac:.3f}" for r in res_p3])

    if not sources:
        print("No sources to run (use --no-synthetic only if you provide a MIDI path).")
        if real_path:
            print("Real MIDI path not found:", real_path)

    report_lines = report_baseline_final(baseline_runs, threshold_mode, quantile_q, knn_k_list, alpha)
    for line in report_lines:
        print(line)
    if save_prefix:
        with open(f"{save_prefix}report.txt", "w") as f:
            f.write("\n".join(report_lines))

    # Phase I (multi-observable): solo Bach, configs A/B/C, reporte Delta, Delta_G, ÉXITO
    if real_path and Path(real_path).exists():
        run_phase1_bach_report(
            real_path,
            save_prefix or "",
            bin_size=bin_size,
            step=step,
            cap=cap,
            knn_k_list=knn_k_list,
            n_max_p2=n_max,
            n_max_p3=min(5, n_max),
            control_primes=getattr(args, "control_primes", None) or [],
```

```
        )
        # Phase II (beat-based): Bach, alpha sweep, reporte Delta/Delta_G, k/V, ÉXITO
        run_phase2_bach_report(
            real_path,
            save_prefix or "",
            bin_size_beats=args.bin_size_beats,
            step=step,
            cap=cap,
            knn_k_list=knn_k_list,
            alpha_list=[0.0, 0.5, 1.0, 2.0, 4.0],
            n_max_p2=n_max,
            n_max_p3=min(5, n_max),
        )


if __name__ == "__main__":
    main()
```

A.2.3 `src/padicmidi/core/hierarchical.py`

```
#!/usr/bin/env python3
"""
padicmidi.core.hierarchical — p-adic tower with forced inverse system.

Hierarchical dictionaries ``S_n`` together with the explicit inverse map
``pi_{n+1,n}: S_{n+1} -> S_n`` that forces each child to inherit a parent
by construction. Computes the coherence invariants
``Coh_pi(p, n)`` and ``Coh_grid(p, n)`` and writes the per-parent audit
table that allows verification of hypotheses (SC) and (AI) of the
null-floor proposition.

Construction summary:
  * ``S_1`` is obtained via K-means on a sample of windows of length p.
  * For every ``n -> n+1`` the parent of an aggregated window
    ``W_{p,n+1}(b)`` is its truncation projected onto ``S_n``; per-parent
    K-means with ``Kchild`` clusters yields ``S_{n+1}``;
    ``pi(child_id) = parent_id`` by construction.

Outputs (one file per CSV):
  * ``params.json``, ``params.txt``
  * ``S_n_prototypes_p{p}_n{n}.csv``
  * ``pi_p{p}_n{n+1}_to_n{n}.csv``
  * ``f_p{p}_n{n}.csv``
  * ``coherence_hier_p{p}.csv``
  * ``audit_p{p}.csv``
"""

from __future__ import annotations

import argparse
import csv
import json
from pathlib import Path
from collections import defaultdict

import numpy as np
```

```python

from padicmidi.core.echo import (
    parse_midi_notes_seconds,
    parse_midi_notes_beats,
    chroma_series_duration,
    chroma_series_duration_beats,
    onset_density_series,
    onset_density_series_beats,
    series_with_rhythm,
)
from padicmidi.core.config import ALPHA, MAX_WINDOWS_PER_RESIDUE, default_nmax


def build_X_seconds(path: str, bin_size: float = 0.05) -> np.ndarray:
    events = parse_midi_notes_seconds(path)
    if not events:
        return np.zeros((0, 13), dtype=float)
    H = chroma_series_duration(events, bin_size=bin_size)
    a = onset_density_series(events, bin_size=bin_size)
    return series_with_rhythm(H, a, alpha=ALPHA)


def build_X_beats(path: str, bin_size_beats: float = 1.0 / 12.0) -> np.ndarray:
    events = parse_midi_notes_beats(path)
    if not events:
        return np.zeros((0, 13), dtype=float)
    H = chroma_series_duration_beats(events, bin_size_beats=bin_size_beats)
    a = onset_density_series_beats(events, bin_size_beats=bin_size_beats)
    return series_with_rhythm(H, a, alpha=ALPHA)


def get_windows(X: np.ndarray, N: int, step: int) -> list[tuple[int, np.ndarray]]:
    out = []
    for start in range(0, len(X) - N + 1, step):
        out.append((start, X[start : start + N, :].copy()))
    return out


def aggregate_median(windows: list[np.ndarray]) -> np.ndarray:
    """Componentwise median; W_{p,n}(a) = median over windows with i ≡ a (mod p^n)."""
    if not windows:
        return np.full((1, 1), np.nan)
    stack = np.stack(windows, axis=0)
    return np.median(stack, axis=0).astype(np.float64)


def dist_matrix(windows: np.ndarray, prototypes: np.ndarray) -> np.ndarray:
    """windows (W, N, 13), prototypes (P, N, 13) -> (W, P)."""
    diff = windows[:, None, :, :] - prototypes[None, :, :, :]
    return np.sqrt(np.mean(np.sum(diff ** 2, axis=3), axis=2))


def kmeans_numpy(samples: np.ndarray, K: int, rng: np.random.Generator, max_iter: int = 30) ->
 np.ndarray:
    """samples (N, D). Returns centers (K_eff, D) with K_eff = max(1, min(K, len(samples)))."""
    if len(samples) == 0:
        return np.zeros((K, samples.shape[1]) if samples.size else (K, 0))
    K_actual = max(1, min(K, len(samples)))
```

```
    idx = np.arange(len(samples))
    rng.shuffle(idx)
    centers = samples[idx[:K_actual]].copy()
    if K_actual < K:
        centers = np.vstack([centers, np.tile(centers[-1], (K - K_actual, 1))])
    for _ in range(max_iter):
        dists = np.linalg.norm(samples[:, None, :] - centers[None, :, :], axis=2)
        labels = np.argmin(dists, axis=1)
        new_centers = centers.copy()
        for j in range(K_actual):
            mask = labels == j
            if np.any(mask):
                new_centers[j] = samples[mask].mean(axis=0)
        if np.allclose(new_centers, centers):
            break
        centers = new_centers
    return centers[:K_actual]


def build_W_n(X: np.ndarray, p: int, n_max: int, step: int) -> dict[int, dict[int, np.ndarray]]:
    """W_n[n][a] = aggregated window (p^n, 13) for residue a. Median over { P_{n,i} : i ≡ a (mod p^n)
        }."""
    W_n = {}
    for n in range(1, n_max + 1):
        N = p ** n
        windows_with_start = get_windows(X, N, step)
        if not windows_with_start:
            W_n[n] = {}
            continue
        by_residue = defaultdict(list)
        for start, w in windows_with_start:
            a = start % (p ** n)
            if len(by_residue[a]) < MAX_WINDOWS_PER_RESIDUE:
                by_residue[a].append(w)
        W_n[n] = {}
        for a, wlist in by_residue.items():
            W_n[n][a] = aggregate_median(wlist)
    return W_n


def run_hierarchical(
    X: np.ndarray,
    p: int,
    Nmax: int,
    step: int,
    K: int,
    Kchild: int,
    M: int,
    rng: np.random.Generator,
) -> tuple[dict, dict, dict, list[dict], list[dict]]:
    """
    Returns:
      prototypes_n: n -> (|S_n|, p^n, 13)
      f_n: n -> dict residue a -> prototype_id
      pi_maps: n -> list of length |S_{n+1}|, pi_maps[n][child_id] = parent_id
      coherence_rows: list of {n, Coh_pi, Coh_grid, n_samples_n, n_samples_nplus1}
      audit_rows: per-parent diagnostics for the quantities underlying Coh_pi
        (pi columns) and Coh_grid (trunc columns).
```

```
    """
    W_n = build_W_n(X, p, Nmax, step)
    prototypes_n = {}
    f_n = {}
    pi_maps = {}

    N1 = p
    win1 = get_windows(X, N1, step)
    if not win1:
        return prototypes_n, f_n, pi_maps, [], []

    indices = np.arange(len(win1))
    if len(indices) > M:
        indices = np.linspace(0, len(indices) - 1, M).astype(int)
    sample1 = np.array([win1[i][1].flatten() for i in indices])
    K1 = max(1, min(K, len(sample1)))
    centers1 = kmeans_numpy(sample1, K1, rng)
    prototypes_n[1] = centers1.reshape(-1, N1, 13)
    f_n[1] = {}
    if W_n.get(1):
        keys, wstack = zip(*[(a, w) for a, w in W_n[1].items() if not np.any(np.isnan(w))])
        if keys:
            wstack = np.stack(wstack)
            d = dist_matrix(wstack, prototypes_n[1])
            for i, a in enumerate(keys):
                f_n[1][a] = int(np.argmin(d[i]))

    for n in range(1, Nmax):
        pn = p ** n
        pn1 = p ** (n + 1)
        if n + 1 not in W_n or not W_n[n + 1]:
            continue
        b_list, w_list = zip(*[(b, w) for b, w in W_n[n + 1].items() if not np.any(np.isnan(w))])
        if not b_list:
            continue
        w_list = np.stack(w_list)
        trunc_list = w_list[:, :pn, :]
        d_parent = dist_matrix(trunc_list, prototypes_n[n])
        parent_of_b = {b: int(np.argmin(d_parent[i])) for i, b in enumerate(b_list)}

        by_parent = defaultdict(list)
        for b, w in W_n[n + 1].items():
            if b not in parent_of_b or np.any(np.isnan(w)):
                continue
            by_parent[parent_of_b[b]].append(w)

        n_parents = len(prototypes_n[n])
        all_children = []
        pi_list = []
        for j in range(n_parents):
            wlist = by_parent.get(j, [])
            K_eff = max(1, min(Kchild, len(wlist))) if wlist else 1
            if len(wlist) < 2:
                c = wlist[0].copy() if len(wlist) == 1 else np.repeat(
                    prototypes_n[n][j], (pn1 + pn - 1) // pn, axis=0
                )[:pn1, :].copy()
                for _ in range(K_eff):
                    all_children.append(c.copy())
```

```
                    pi_list.append(j)
                for _ in range(Kchild - K_eff):
                    all_children.append(all_children[-1].copy())
                    pi_list.append(j)
            else:
                flat = np.array([x.flatten() for x in wlist])
                centers_child = kmeans_numpy(flat, K_eff, rng)
                for k in range(len(centers_child)):
                    all_children.append(centers_child[k].reshape(pn1, 13))
                    pi_list.append(j)
                for _ in range(Kchild - len(centers_child)):
                    all_children.append(all_children[-1].copy())
                    pi_list.append(j)
        if not all_children:
            continue
        prototypes_n[n + 1] = np.stack(all_children, axis=0)
        pi_maps[n] = pi_list
        keys_n1, w_n1 = zip(*[(a, w) for a, w in W_n[n + 1].items() if not np.any(np.isnan(w))])
        if keys_n1:
            f_n[n + 1] = {}
            w_stack_n1 = np.stack(w_n1)
            d_n1 = dist_matrix(w_stack_n1, prototypes_n[n + 1])
            for i, a in enumerate(keys_n1):
                f_n[n + 1][a] = int(np.argmin(d_n1[i]))

    coherence_rows = []
    audit_rows = []
    for n in range(1, Nmax):
        if n not in pi_maps or n + 1 not in f_n or n not in f_n:
            continue
        pn = p ** n
        pn1 = p ** (n + 1)
        valid_b_list = [
            b for b in range(pn1)
            if b in W_n.get(n + 1, {}) and not np.any(np.isnan(W_n[n + 1][b]))
            and f_n[n + 1].get(b) is not None and (b % pn) in f_n[n]
        ]
        if not valid_b_list:
            coherence_rows.append({
                "n": n, "Coh_pi": 0.0, "Coh_grid": 0.0,
                "n_samples_n": len(W_n.get(n, {})), "n_samples_nplus1": 0,
            })
            continue
        w_stack = np.stack([W_n[n + 1][b] for b in valid_b_list])
        trunc_stack = w_stack[:, :pn, :]
        d_trunc = dist_matrix(trunc_stack, prototypes_n[n])
        q_n_trunc = np.argmin(d_trunc, axis=1)
        q_n1_arr = np.array([f_n[n + 1][b] for b in valid_b_list], dtype=int)
        parent_via_pi = np.array([pi_maps[n][j] for j in q_n1_arr])
        f_n_r_b = np.array([f_n[n][b % pn] for b in valid_b_list], dtype=int)
        match_pi = int((parent_via_pi == f_n_r_b).sum())
        match_grid = int((q_n_trunc == f_n_r_b).sum())
        coh_pi = match_pi / pn1 if pn1 > 0 else 0.0
        coh_grid = match_grid / pn1 if pn1 > 0 else 0.0
        coherence_rows.append({
            "n": n,
            "Coh_pi": round(coh_pi, 6),
            "Coh_grid": round(coh_grid, 6),
```

```
            "n_samples_n": len(W_n.get(n, {})),
            "n_samples_nplus1": len(valid_b_list),
        })

        valid_index = {b: i for i, b in enumerate(valid_b_list)}
        for a in range(pn):
            siblings = [a + k * pn for k in range(p)]
            valid_siblings = [b for b in siblings if b in valid_index]
            if not valid_siblings or a not in f_n[n]:
                audit_rows.append({
                    "n": n,
                    "parent_class": a,
                    "n_valid_siblings": len(valid_siblings),
                    "excluded_sparsity": 1,
                    "V_SC_pi": "",
                    "AI_pi": "",
                    "coherent_count_pi": "",
                    "V_SC_trunc": "",
                    "AI_trunc": "",
                    "coherent_count_trunc": "",
                    "parent_prototype": f_n[n].get(a, ""),
                })
                continue

            idx = [valid_index[b] for b in valid_siblings]
            parent_proto = int(f_n[n][a])
            pi_vals = [int(parent_via_pi[i]) for i in idx]
            trunc_vals = [int(q_n_trunc[i]) for i in idx]
            audit_rows.append({
                "n": n,
                "parent_class": a,
                "n_valid_siblings": len(valid_siblings),
                "excluded_sparsity": int(len(valid_siblings) < p),
                "V_SC_pi": len(set(pi_vals)),
                "AI_pi": int(parent_proto in set(pi_vals)),
                "coherent_count_pi": sum(1 for x in pi_vals if x == parent_proto),
                "V_SC_trunc": len(set(trunc_vals)),
                "AI_trunc": int(parent_proto in set(trunc_vals)),
                "coherent_count_trunc": sum(1 for x in trunc_vals if x == parent_proto),
                "parent_prototype": parent_proto,
            })
    return prototypes_n, f_n, pi_maps, coherence_rows, audit_rows


def main() -> None:
    ap = argparse.ArgumentParser(description="Hierarchical profinite maps with π_{n+1,n}")
    ap.add_argument("midi", help="Path to MIDI file")
    ap.add_argument("--axis", choices=["seconds", "beats"], default="beats")
    ap.add_argument("--bin", type=float, default=0.05, help="Bin size (seconds axis)")
    ap.add_argument("--bin-beats", type=float, default=1.0 / 12.0, help="Bin size (beats)")
    ap.add_argument("--p", type=int, required=True, help="Prime (2,3,5,7)")
    ap.add_argument("--Nmax", type=int, default=None)
    ap.add_argument("--K", type=int, default=16)
    ap.add_argument("--Kchild", type=int, default=2, help="Children per parent")
    ap.add_argument("--M", type=int, default=800)
    ap.add_argument("--step", type=int, default=2)
    ap.add_argument("--seed", type=int, default=42)
    ap.add_argument("--out", required=True, help="Output directory (e.g. .../p2/)")
```

```
    ap.add_argument("--audit-only", action="store_true", help="Only write audit_p{p}.csv; leave existing coherence/prototype files untouched")
    ap.add_argument("--force", action="store_true", help="Overwrite audit_p{p}.csv when --audit-only is used")
    args = ap.parse_args()

    if args.p not in (2, 3, 5, 7):
        raise SystemExit("--p must be 2, 3, 5, or 7")
    Nmax = args.Nmax if args.Nmax is not None else default_nmax(args.p)
    out_dir = Path(args.out)
    out_dir.mkdir(parents=True, exist_ok=True)
    audit_path = out_dir / f"audit_p{args.p}.csv"
    if args.audit_only and audit_path.exists() and not args.force:
        print(f"Audit exists, skipping (use --force to overwrite): {audit_path}", file=sys.stderr)
        return

    if args.axis == "seconds":
        X = build_X_seconds(args.midi, bin_size=args.bin)
    else:
        X = build_X_beats(args.midi, bin_size_beats=args.bin_beats)

    if len(X) < 2:
        raise SystemExit("Series too short")

    rng = np.random.default_rng(args.seed)
    prototypes_n, f_n, pi_maps, coherence_rows, audit_rows = run_hierarchical(
        X, args.p, Nmax, args.step, args.K, args.Kchild, args.M, rng
    )

    p = args.p
    # params
    params = {
        "piece": str(Path(args.midi).name),
        "axis": args.axis,
        "p": p,
        "Nmax": Nmax,
        "K": args.K,
        "Kchild": args.Kchild,
        "M": args.M,
        "step": args.step,
        "seed": args.seed,
        "bin_seconds": args.bin,
        "bin_beats": args.bin_beats,
    }
    if not args.audit_only:
        with open(out_dir / "params.json", "w") as f:
            json.dump(params, f, indent=2)
        with open(out_dir / "params.txt", "w") as f:
            f.write(f"piece={params['piece']} axis={args.axis} p={p} Nmax={Nmax} K={args.K} Kchild={args.Kchild} M={args.M} step={args.step} seed={args.seed}\n")

        # S_n_prototypes_p{p}_n{n}.csv (rows = prototypes, flattened)
        for n, prot in prototypes_n.items():
            flat = prot.reshape(prot.shape[0], -1)
            path = out_dir / f"S_n_prototypes_p{p}_n{n}.csv"
            np.savetxt(path, flat, delimiter=",")
        # pi_p{p}_n{n+1}_to_n{n}.csv
        for n, pi_list in pi_maps.items():
```

```
            path = out_dir / f"pi_p{p}_n{n+1}_to_n{n}.csv"
            with open(path, "w", newline="") as f:
                w = csv.writer(f)
                w.writerow(["child_id", "parent_id"])
                for cid, pid in enumerate(pi_list):
                    w.writerow([cid, pid])
        # f_p{p}_n{n}.csv
        for n, fn in f_n.items():
            path = out_dir / f"f_p{p}_n{n}.csv"
            with open(path, "w", newline="") as f:
                w = csv.writer(f)
                w.writerow(["residue", "prototype_id"])
                for a in sorted(fn.keys()):
                    w.writerow([a, fn[a]])

        # coherence_hier_p{p}.csv
        coh_path = out_dir / f"coherence_hier_p{p}.csv"
        with open(coh_path, "w", newline="") as f:
            w = csv.DictWriter(f, fieldnames=["n", "Coh_pi", "Coh_grid", "n_samples_n",
                "n_samples_nplus1"])
            w.writeheader()
            w.writerows(coherence_rows)
    else:
        coh_path = out_dir / f"coherence_hier_p{p}.csv"

    # audit_p{p}.csv stores per-parent diagnostics for both quantities:
    # pi-columns recompute Coh_pi, trunc-columns recompute Coh_grid.
    audit_fields = [
        "n",
        "parent_class",
        "n_valid_siblings",
        "excluded_sparsity",
        "V_SC_pi",
        "AI_pi",
        "coherent_count_pi",
        "V_SC_trunc",
        "AI_trunc",
        "coherent_count_trunc",
        "parent_prototype",
    ]
    with open(audit_path, "w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=audit_fields)
        w.writeheader()
        w.writerows(audit_rows)

    print(f"Wrote {coh_path} ({len(coherence_rows)} rows), {audit_path} ({len(audit_rows)} rows),
        params, prototypes, pi, f")


if __name__ == "__main__":
    main()
```

A.3 Adaptadores de entrada/salida (`io/`)

A.3.1 `src/padicmidi/io/midi_mido.py`

```
"""
padicmidi.io.midi_mido — default MIDI adapter (re-export from the canonical motor).

This module exists for two reasons:

1. To keep ``padicmidi.io.*`` as a stable public surface for users who
   want to swap implementations.
2. To document explicitly that the gold-standard CSVs in
   ``results/verified/`` were produced with ``mido``.

The actual parsing is implemented in :mod:`padicmidi.core.echo`; this
module simply re-exports the two parsing functions.
"""

from __future__ import annotations

from padicmidi.core.echo import (
    parse_midi_notes_seconds,
    parse_midi_notes_beats,
)

__all__ = ["parse_midi_notes_seconds", "parse_midi_notes_beats"]
```

A.3.2 `src/padicmidi/io/midi_pretty.py`

```
"""
padicmidi.io.midi_pretty — optional MIDI adapter using ``pretty_midi``.

This adapter exists for compatibility with workflows that prefer
``pretty_midi``. It reproduces the same ``(t_on, t_off, pitch, velocity)``
quadruples emitted by :mod:`padicmidi.io.midi_mido`, but with two
caveats:

* Numerical outputs may differ from the gold standard at the 4–th5th
  decimal place because ``pretty_midi`` uses a different tick-to-second
  resolution model under tempo changes.
* This adapter requires the optional dependency ``pretty_midi``;
  install with ``pip install padicmidi[pretty]``.

Use this adapter only when you cannot install ``mido`` or when you want
to cross-validate the parsing layer.
"""

from __future__ import annotations

from typing import List, Tuple

NoteEvent = Tuple[float, float, int, int]


def _require_pretty_midi():
    try:
        import pretty_midi
    except ImportError as exc:  # pragma: no cover - exercised only when pretty_midi missing
        raise ImportError(
            "padicmidi.io.midi_pretty requires the optional dependency 'pretty_midi'. "
```

```
            "Install with: pip install padicmidi[pretty]"
        ) from exc
    return pretty_midi


def parse_midi_notes_seconds(path: str) -> List[NoteEvent]:
    """Return ``(t_on, t_off, pitch, velocity)`` in seconds via ``pretty_midi``."""
    pretty_midi = _require_pretty_midi()
    pm = pretty_midi.PrettyMIDI(path)
    events: List[NoteEvent] = []
    for instrument in pm.instruments:
        for note in instrument.notes:
            events.append((float(note.start), float(note.end), int(note.pitch), int(note.velocity)))
    events.sort(key=lambda e: e[0])
    return events


def parse_midi_notes_beats(path: str) -> List[NoteEvent]:
    """Return ``(u_on, u_off, pitch, velocity)`` in beats via ``pretty_midi``.

    Beat-time uses ``pretty_midi.time_to_tick`` divided by ticks per beat,
    therefore it follows the score grid rather than wall-clock time.
    """
    pretty_midi = _require_pretty_midi()
    pm = pretty_midi.PrettyMIDI(path)
    tpb = pm.resolution
    events: List[NoteEvent] = []
    for instrument in pm.instruments:
        for note in instrument.notes:
            u_on = pm.time_to_tick(note.start) / tpb
            u_off = pm.time_to_tick(note.end) / tpb
            events.append((float(u_on), float(u_off), int(note.pitch), int(note.velocity)))
    events.sort(key=lambda e: e[0])
    return events


__all__ = ["parse_midi_notes_seconds", "parse_midi_notes_beats"]
```

A.4 Análisis y modelos nulos (`analysis/`)

A.4.1 `src/padicmidi/analysis/null_model.py`

```
#!/usr/bin/env python3
"""
Fast null model verification: config A only, parallel processing.
Verifies original values and computes pitch/time-shuffle null models.
"""
import sys, os, csv
import numpy as np

sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))

from profinite_echo_midi import (
    parse_midi_notes_seconds,
    parse_midi_notes_beats,
    chroma_series_duration,
```

```
    chroma_series_duration_beats,
    onset_density_series,
    onset_density_series_beats,
    build_phase1_patterns,
    build_phase1_patterns_beats,
    knn_graph_phase1,
)
import networkx as nx


def tower_beta0(events, H, a_raw, p, n_max, k, cap, use_beats):
    result = {}
    for n in range(1, n_max + 1):
        N = p ** n
        if use_beats:
            u0 = events[0][0] if events else 0.0
            patterns = build_phase1_patterns_beats(
                events, H, a_raw, 1.0/12.0, u0,
                N=N, step=1, d_ioi=16, Tmax_ioi_beats=2.0)
        else:
            t0 = events[0][0] if events else 0.0
            patterns = build_phase1_patterns(
                events, H, a_raw, 0.05, t0,
                N=N, step=1, d_ioi=16, Tmax_ioi=2.0)
        if len(patterns) > cap:
            idx = np.linspace(0, len(patterns)-1, cap).astype(int)
            patterns = [patterns[i] for i in idx]
        if not patterns:
            result[n] = 0
            continue
        G = knn_graph_phase1(patterns, k, 1.0, 0.0, 0.0)
        result[n] = nx.number_connected_components(G)
    return result


def load_events_and_series(midi_path, axis):
    use_beats = axis == "beats"
    if use_beats:
        events = parse_midi_notes_beats(midi_path)
        H = chroma_series_duration_beats(events, bin_size_beats=1.0/12.0)
        a = onset_density_series_beats(events, bin_size_beats=1.0/12.0)
    else:
        events = parse_midi_notes_seconds(midi_path)
        H = chroma_series_duration(events, bin_size=0.05)
        a = onset_density_series(events, bin_size=0.05)
    return events, H, a, use_beats


def compute_mad_s23(midi_path, axis, ks, cap, shuffle_type="none", seed=42):
    events, H, a, use_beats = load_events_and_series(midi_path, axis)
    if not events:
        return 0.0, 0.0, 0

    rng = np.random.RandomState(seed)
    if shuffle_type == "pitch":
        H_shuf = H.copy()
        rng.shuffle(H_shuf)
        H = H_shuf
```

```
    elif shuffle_type == "time":
        a = a.copy()
        win = max(50, len(a) // 20)
        for s in range(0, len(a), win):
            block = a[s:s+win].copy()
            rng.shuffle(block)
            a[s:s+win] = block

    deltas = []
    for k in ks:
        b2 = tower_beta0(events, H, a, 2, 6, k, cap, use_beats)
        b3 = tower_beta0(events, H, a, 3, 5, k, cap, use_beats)
        for n in range(2, 6):
            if n in b2 and n in b3:
                deltas.append(b2[n] - b3[n])

    if not deltas:
        return 0.0, 0.0, 0
    mad = sum(abs(d) for d in deltas) / len(deltas)
    s23 = sum(1 for d in deltas if d != 0) / len(deltas)
    return mad, s23, len(deltas)


def original_from_csvs(prefix, axis, ks, csv_base):
    deltas = []
    sfx = "_beats" if axis == "beats" else ""
    for k in ks:
        p2 = os.path.join(csv_base, axis,
                          f"{prefix}_cap300_A_tower_real_bach_k{k}_p2{sfx}.csv")
        p3 = os.path.join(csv_base, axis,
                          f"{prefix}_cap300_A_tower_real_bach_k{k}_p3{sfx}.csv")
        if not os.path.exists(p2) or not os.path.exists(p3):
            continue
        with open(p2) as f:
            b2 = {int(r["n"]): int(r["beta0"]) for r in csv.DictReader(f)}
        with open(p3) as f:
            b3 = {int(r["n"]): int(r["beta0"]) for r in csv.DictReader(f)}
        for n in range(2, 6):
            if n in b2 and n in b3:
                deltas.append(b2[n] - b3[n])

    if not deltas:
        return 0.0, 0.0, 0
    mad = sum(abs(d) for d in deltas) / len(deltas)
    s23 = sum(1 for d in deltas if d != 0) / len(deltas)
    return mad, s23, len(deltas)


if __name__ == "__main__":
    base = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
    ext = os.path.join(base, "data", "external_midis")
    out = os.path.join(base, "outputs")
    ks = [8, 10, 12]

    pieces = [
        ("BWV 1049 mov.1", "bwv1049_mov1",
         os.path.join(ext, "bwv1049_brand4_mov1.bachcentral.mid")),
        ("BWV 1050 mov.2", "bwv1050_mov2",
```

```
         os.path.join(ext, "bwv1050_brand5_mov2.bachcentral.mid")),
    ]

    fmt = f"{'Piece':<20s} {'Variant':<15s} {'Axis':<8s} {'MAD':>8s} {'S23':>8s} {'N':>4s}"
    print(fmt)
    print("-" * len(fmt))

    for name, pfx, midi in pieces:
        for axis in ["seconds", "beats"]:
            mad, s23, n = original_from_csvs(pfx, axis, ks, out)
            print(f"{name:<20s} {'original':<15s} {axis:<8s} "
                  f"{mad:8.3f} {s23:8.3f} {n:4d}")

        for sh in ["pitch", "time"]:
            for axis in ["seconds", "beats"]:
                print(f"  {name} {sh}-shuffle {axis}...",
                      file=sys.stderr, flush=True)
                mad, s23, n = compute_mad_s23(midi, axis, ks, 300, sh)
                print(f"{name:<20s} {sh+'-shuffle':<15s} {axis:<8s} "
                      f"{mad:8.3f} {s23:8.3f} {n:4d}")
```

A.4.2 `src/padicmidi/analysis/audit.py`

```
#!/usr/bin/env python3
"""Aggregate per-parent SC/AI audits from build_hierarchical_maps.py.

The audit has two parallel diagnostic families:

* *_pi columns recompute the actual Coh_pi numerator.
* *_trunc columns recompute the Coh_grid numerator.

This distinction matters because the current pipeline assigns level-(n+1)
residues globally to child prototypes, so pi(q_{n+1}(.)) can differ from
q_n(trunc(.)).
"""

from __future__ import annotations

import argparse
import csv
from pathlib import Path
from statistics import mean


def read_csv(path: Path) -> list[dict[str, str]]:
    with path.open(newline="") as f:
        return list(csv.DictReader(f))


def write_csv(path: Path, rows: list[dict], fields: list[str]) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=fields)
        w.writeheader()
        w.writerows(rows)

```

```python

def parse_context(path: Path) -> tuple[str, str, int]:
    # Expected: outputs/paper_profinite_hier/<piece>/<axis>/p<p>/audit_p<p>.csv
    p_dir = path.parent
    axis_dir = p_dir.parent
    piece_dir = axis_dir.parent
    p = int(p_dir.name.removeprefix("p"))
    return piece_dir.name, axis_dir.name, p


def load_reported(path: Path) -> dict[int, dict[str, float]]:
    p = int(path.parent.name.removeprefix("p"))
    coh_path = path.parent / f"coherence_hier_p{p}.csv"
    out: dict[int, dict[str, float]] = {}
    if not coh_path.exists():
        return out
    for row in read_csv(coh_path):
        n = int(row["n"])
        out[n] = {
            "coh_pi": float(row["Coh_pi"]),
            "coh_grid": float(row["Coh_grid"]),
            "valid_b": float(row["n_samples_nplus1"]),
        }
    return out


def aggregate_one(path: Path) -> list[dict]:
    piece, axis, p = parse_context(path)
    reported = load_reported(path)
    by_n: dict[int, list[dict[str, str]]] = {}
    for row in read_csv(path):
        by_n.setdefault(int(row["n"]), []).append(row)

    rows: list[dict] = []
    for n, items in sorted(by_n.items()):
        counted = [r for r in items if r["coherent_count_pi"] != ""]
        included = [r for r in counted if r["excluded_sparsity"] == "0"]
        excluded = [r for r in items if r["excluded_sparsity"] == "1"]
        denom_dense = p ** (n + 1)
        valid_b = sum(int(r["n_valid_siblings"]) for r in counted)
        sum_pi = sum(int(r["coherent_count_pi"]) for r in counted)
        sum_trunc = sum(int(r["coherent_count_trunc"]) for r in counted)
        coh_pi_recomputed = sum_pi / denom_dense if denom_dense else 0.0
        coh_grid_recomputed = sum_trunc / denom_dense if denom_dense else 0.0
        coh_pi_valid = sum_pi / valid_b if valid_b else 0.0
        coh_grid_valid = sum_trunc / valid_b if valid_b else 0.0
        rep = reported.get(n, {})
        coh_pi_reported = rep.get("coh_pi", float("nan"))
        coh_grid_reported = rep.get("coh_grid", float("nan"))
        if abs(coh_pi_recomputed - coh_pi_reported) > 1e-6:
            raise RuntimeError(
                f"Coh_pi mismatch {path} n={n}: "
                f"recomputed={coh_pi_recomputed} reported={coh_pi_reported}"
            )
        if abs(coh_grid_recomputed - coh_grid_reported) > 1e-6:
            raise RuntimeError(
                f"Coh_grid mismatch {path} n={n}: "
                f"recomputed={coh_grid_recomputed} reported={coh_grid_reported}"
```

```
            )
        n_parents = len(included)
        n_sc_pi = sum(int(r["V_SC_pi"]) == p for r in included)
        n_ai_pi = sum(int(r["AI_pi"]) == 1 for r in included)
        n_both_pi = sum(int(r["V_SC_pi"]) == p and int(r["AI_pi"]) == 1 for r in included)
        n_sc_trunc = sum(int(r["V_SC_trunc"]) == p for r in included)
        n_ai_trunc = sum(int(r["AI_trunc"]) == 1 for r in included)
        n_both_trunc = sum(int(r["V_SC_trunc"]) == p and int(r["AI_trunc"]) == 1 for r in included)
        rows.append({
            "piece": piece,
            "axis": axis,
            "p": p,
            "n": n,
            "n_parents_total": len(items),
            "n_parents_included": n_parents,
            "n_parents_excluded_sparsity": len(excluded),
            "n_SC_pi_holds": n_sc_pi,
            "n_AI_pi_holds": n_ai_pi,
            "n_both_pi_hold": n_both_pi,
            "frac_SC_pi": n_sc_pi / n_parents if n_parents else 0.0,
            "frac_AI_pi": n_ai_pi / n_parents if n_parents else 0.0,
            "frac_both_pi": n_both_pi / n_parents if n_parents else 0.0,
            "n_SC_trunc_holds": n_sc_trunc,
            "n_AI_trunc_holds": n_ai_trunc,
            "n_both_trunc_hold": n_both_trunc,
            "frac_SC_trunc": n_sc_trunc / n_parents if n_parents else 0.0,
            "frac_AI_trunc": n_ai_trunc / n_parents if n_parents else 0.0,
            "frac_both_trunc": n_both_trunc / n_parents if n_parents else 0.0,
            "coverage": valid_b / denom_dense if denom_dense else 0.0,
            "coh_pi_recomputed": coh_pi_recomputed,
            "coh_pi_reported": coh_pi_reported,
            "coh_pi_valid": coh_pi_valid,
            "coh_grid_recomputed": coh_grid_recomputed,
            "coh_grid_reported": coh_grid_reported,
            "coh_grid_valid": coh_grid_valid,
        })
    return rows


def group_for(piece: str) -> str:
    if piece.startswith(("bwv1049", "bwv1050", "bwv1079")) or piece == "goldberg_aria":
        return "poly"
    return "mono"


def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--root", default="outputs/paper_profinite_hier")
    ap.add_argument("--out", default="outputs/audit_summary")
    args = ap.parse_args()
    root = Path(args.root)
    out_dir = Path(args.out)
    audit_paths = sorted(root.glob("*/**/p*/audit_p*.csv"))
    if not audit_paths:
        raise SystemExit(f"No audit_p*.csv files found under {root}")

    table: list[dict] = []
    for path in audit_paths:
```

```
        table.extend(aggregate_one(path))

    fields = list(table[0].keys())
    write_csv(out_dir / "audit_table.csv", table, fields)

    by_key: dict[tuple[str, str, int], list[dict]] = {}
    for row in table:
        by_key.setdefault((row["piece"], row["axis"], int(row["p"])), []).append(row)

    piece_rows = []
    for (piece, axis, p), rows in sorted(by_key.items()):
        piece_rows.append({
            "piece": piece,
            "texture_group": group_for(piece),
            "axis": axis,
            "p": p,
            "n_levels": len(rows),
            "mean_frac_SC_pi": mean(float(r["frac_SC_pi"]) for r in rows),
            "mean_frac_AI_pi": mean(float(r["frac_AI_pi"]) for r in rows),
            "mean_frac_both_pi": mean(float(r["frac_both_pi"]) for r in rows),
            "mean_frac_SC_trunc": mean(float(r["frac_SC_trunc"]) for r in rows),
            "mean_frac_AI_trunc": mean(float(r["frac_AI_trunc"]) for r in rows),
            "mean_frac_both_trunc": mean(float(r["frac_both_trunc"]) for r in rows),
            "mean_coverage": mean(float(r["coverage"]) for r in rows),
            "mean_coh_pi": mean(float(r["coh_pi_reported"]) for r in rows),
            "mean_coh_pi_valid": mean(float(r["coh_pi_valid"]) for r in rows),
        })
    write_csv(out_dir / "audit_by_piece.csv", piece_rows, list(piece_rows[0].keys()))

    with (out_dir / "SUMMARY.md").open("w") as f:
        f.write("# Audit Summary\n\n")
        f.write(f"Audited files: {len(audit_paths)}\n\n")
        f.write("`coh_pi_recomputed == coh_pi_reported` and `coh_grid_recomputed == coh_grid_reported`
            for all rows.\n\n")
        for group in ("mono", "poly"):
            group_rows = [r for r in piece_rows if r["texture_group"] == group and int(r["p"]) == 2]
            if not group_rows:
                continue
            f.write(f"## {group} p=2\n\n")
            f.write(f"- mean coverage: {mean(float(r['mean_coverage']) for r in group_rows):.4f}\n")
            f.write(f"- mean frac_SC_pi: {mean(float(r['mean_frac_SC_pi']) for r in group_rows):.4f}\n")
            f.write(f"- mean frac_AI_pi: {mean(float(r['mean_frac_AI_pi']) for r in group_rows):.4f}\n")
            f.write(f"- mean Coh_pi_valid: {mean(float(r['mean_coh_pi_valid']) for r in
                group_rows):.4f}\n\n")


if __name__ == "__main__":
    main()
```

A.4.3 `src/padicmidi/analysis/aggregate.py`

```
#!/usr/bin/env python3
"""
build_control_primes_summary_nextpp.py — Ingest Phase I Next++ CSVs from
outputs/phaseI_nextpp/raw/<piece>/<condition>/ e.g. seconds_cap300, beats_bin12_cap300.
Filename: <prefix>_<A|B|C>_tower_real_bach_k<k>_p<2|3|5|7>(_beats)?.csv
```

```
Piece and condition taken from directory path; axis/bin/cap parsed from condition name.
Output: outputs/phaseI_nextpp/summary/SUMMARY_TABLE_p2357_nextpp.csv, CONTROL_PRIMES_REPORT_nextpp.txt
"""
import csv
import re
from pathlib import Path
from collections import defaultdict

ROOT = Path(__file__).resolve().parent.parent
RAW_BASE = ROOT / "outputs" / "phaseI_nextpp" / "raw"
SUMMARY_DIR = ROOT / "outputs" / "phaseI_nextpp" / "summary"
PRIMES = (2, 3, 5, 7)

RE_FNAME = re.compile(r"^(.+)_(A|B|C)_tower_real_bach_k(\d+)_p([2357])(_beats)?\.csv$")


def load_csv(path: Path) -> dict[int, dict]:
    out = {}
    with open(path, newline="") as f:
        r = csv.DictReader(f)
        for row in r:
            n = int(row["n"])
            out[n] = {
                "beta0": int(row["beta0"]),
                "V": int(row["V"]),
                "avg_degree": float(row["avg_degree"]),
                "giant_component_frac": float(row["giant_component_frac"]),
            }
    return out


def parse_condition(cond_name: str) -> tuple[str, str, str]:
    """Return (axis, bin_beats_str, cap_str)."""
    axis = "seconds" if cond_name.startswith("seconds") else "beats"
    cap_str = "500" if "cap500" in cond_name else "300"
    bin_str = "16" if "bin16" in cond_name else ("12" if "bin12" in cond_name else "")
    return axis, bin_str, cap_str


def collect_quadruplets():
    """Yield (piece, condition, axis, bin_beats, cap, config, k, paths). piece/condition from dir
        path."""
    by_key = {}
    if not RAW_BASE.exists():
        return
    for piece_dir in sorted(RAW_BASE.iterdir()):
        if not piece_dir.is_dir():
            continue
        piece = piece_dir.name
        for cond_dir in sorted(piece_dir.iterdir()):
            if not cond_dir.is_dir():
                continue
            cond_name = cond_dir.name
            axis, bin_str, cap_str = parse_condition(cond_name)
            for path in cond_dir.glob("*_tower_real_bach_k*_p*.csv"):
                name = path.name
                m = RE_FNAME.match(name)
                if not m:
```

```
                    continue
                _, config, k_str, p_str, suffix = m.groups()
                p_val = int(p_str)
                k = int(k_str)
                key = (piece, cond_name, axis, bin_str, cap_str, config, k)
                if key not in by_key:
                    by_key[key] = {q: None for q in PRIMES}
                by_key[key][p_val] = path
    for key in sorted(by_key.keys()):
        paths = by_key[key]
        if paths.get(2) and paths.get(3):
            yield (*key, paths)


def main():
    SUMMARY_DIR.mkdir(parents=True, exist_ok=True)
    rows = []
    for piece, condition, axis, bin_beats, cap, config, k, paths in collect_quadruplets():
        data = {p: load_csv(paths[p]) if paths.get(p) else {} for p in PRIMES}
        common_n = set(data[2].keys()) & set(data[3].keys())
        if paths.get(5):
            common_n &= set(data[5].keys())
        if paths.get(7):
            common_n &= set(data[7].keys())
        common_n = sorted(common_n)
        for n in common_n:
            r = {p: data[p].get(n) for p in PRIMES}
            if not r[2] or not r[3]:
                continue
            row = {
                "piece": piece,
                "condition": condition,
                "axis": axis,
                "bin_beats": bin_beats,
                "cap": cap,
                "config": config,
                "k": k,
                "n": n,
                "beta0_p2": r[2]["beta0"],
                "beta0_p3": r[3]["beta0"],
                "beta0_p5": r[5]["beta0"] if r.get(5) else "",
                "beta0_p7": r[7]["beta0"] if r.get(7) else "",
                "giant_p2": round(r[2]["giant_component_frac"], 6),
                "giant_p3": round(r[3]["giant_component_frac"], 6),
                "giant_p5": round(r[5]["giant_component_frac"], 6) if r.get(5) else "",
                "giant_p7": round(r[7]["giant_component_frac"], 6) if r.get(7) else "",
            }
            row["Delta23"] = r[2]["beta0"] - r[3]["beta0"]
            row["Delta25"] = (r[2]["beta0"] - r[5]["beta0"]) if r.get(5) else ""
            row["Delta27"] = (r[2]["beta0"] - r[7]["beta0"]) if r.get(7) else ""
            row["DeltaG23"] = round(r[2]["giant_component_frac"] - r[3]["giant_component_frac"], 6)
            row["DeltaG25"] = round(r[2]["giant_component_frac"] - r[5]["giant_component_frac"], 6) if
                r.get(5) else ""
            row["DeltaG27"] = round(r[2]["giant_component_frac"] - r[7]["giant_component_frac"], 6) if
                r.get(7) else ""
            rows.append(row)

    fieldnames = ["piece", "condition", "axis", "bin_beats", "cap", "config", "k", "n",
```

```
                  "beta0_p2", "beta0_p3", "beta0_p5", "beta0_p7",
                  "giant_p2", "giant_p3", "giant_p5", "giant_p7",
                  "Delta23", "Delta25", "Delta27", "DeltaG23", "DeltaG25", "DeltaG27"]
    out_csv = SUMMARY_DIR / "SUMMARY_TABLE_p2357_nextpp.csv"
    with open(out_csv, "w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=fieldnames, extrasaction="ignore")
        w.writeheader()
        w.writerows(rows)
    print("Wrote", out_csv, "rows:", len(rows))

    report_lines = [
        "CONTROL PRIMES SUMMARY (Phase I Next++)",
        "========================================",
        "Conditions: seconds_cap300/500, beats_bin12/16_cap300/500.",
        "",
    ]
    for (piece, condition) in sorted(set((r["piece"], r["condition"]) for r in rows)):
        sub = [r for r in rows if r["piece"] == piece and r["condition"] == condition]
        if not sub:
            continue
        report_lines.append(f"Piece: {piece}  Condition: {condition}")
        report_lines.append(f"  Rows: {len(sub)}. Delta23 nonzero: {sum(1 for r in sub if
            r.get('Delta23') != 0)}.")
        report_lines.append("")
    report_path = SUMMARY_DIR / "CONTROL_PRIMES_REPORT_nextpp.txt"
    with open(report_path, "w") as f:
        f.write("\n".join(report_lines))
    print("Wrote", report_path)


if __name__ == "__main__":
    main()
```

A.4.4 `src/padicmidi/analysis/directionality.py`

```
#!/usr/bin/env python3
"""Directionality report for hierarchical coherence.

This script reports both the raw dense-normalized Coh_pi and the coverage-corrected
conditional score Coh_pi_valid = match / valid_b from audit_table.csv. The latter is
the appropriate statistic when coverage differs by prime (e.g. p=2 with step=2).
"""

from __future__ import annotations

import argparse
import csv
import math
from pathlib import Path
from statistics import mean


def read_csv(path: Path) -> list[dict[str, str]]:
    with path.open(newline="") as f:
        return list(csv.DictReader(f))

```

```

def group_for(piece: str) -> str:
    if piece.startswith(("bwv1049", "bwv1050", "bwv1079")) or piece == "goldberg_aria":
        return "poly"
    return "mono"


def binom_two_sided(k: int, n: int, p: float = 0.5) -> float:
    if n == 0:
        return 1.0
    probs = [math.comb(n, i) * (p ** i) * ((1 - p) ** (n - i)) for i in range(n + 1)]
    pk = probs[k]
    return min(1.0, sum(x for x in probs if x <= pk + 1e-18))


def clopper_pearson_zero_success(n: int, alpha: float = 0.05) -> tuple[float, float]:
    if n == 0:
        return (0.0, 1.0)
    # For k=0, exact upper endpoint is 1 - (alpha/2)^(1/n).
    return (0.0, 1.0 - (alpha / 2.0) ** (1.0 / n))


def classify(values: list[float], floor: float, eps: float) -> tuple[int, int, int]:
    below = sum(v < floor - eps for v in values)
    above = sum(v > floor + eps for v in values)
    at = len(values) - below - above
    return below, at, above


def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--audit-table", default="outputs/audit_summary/audit_table.csv")
    ap.add_argument("--out", default="outputs/directionality/directionality_report.txt")
    ap.add_argument("--eps", type=float, default=1e-3)
    args = ap.parse_args()

    rows = [r for r in read_csv(Path(args.audit_table)) if int(r["p"]) == 2]
    out = Path(args.out)
    out.parent.mkdir(parents=True, exist_ok=True)

    by_piece: dict[str, list[dict[str, str]]] = {}
    for row in rows:
        by_piece.setdefault(row["piece"], []).append(row)

    lines: list[str] = []
    lines.append("# Directionality Report\n")
    lines.append("Important: raw Coh_pi for p=2 is capped by coverage when step=2. ")
    lines.append("The coverage-corrected score Coh_pi_valid = match/valid_b is therefore reported
         alongside raw Coh_pi.\n\n")

    for group in ("mono", "poly", "all"):
        if group == "all":
            group_rows = rows
        else:
            group_rows = [r for r in rows if group_for(r["piece"]) == group]
        raw_vals = [float(r["coh_pi_reported"]) for r in group_rows]
        valid_vals = [float(r["coh_pi_valid"]) for r in group_rows]
        raw_below, raw_at, raw_above = classify(raw_vals, 0.5, args.eps)
```

```
        valid_below, valid_at, valid_above = classify(valid_vals, 1.0, args.eps)
        n_viol = valid_below + valid_above
        one_sided = (0.5 ** n_viol) if valid_above == 0 and n_viol else 1.0
        two_sided = binom_two_sided(valid_above, n_viol) if n_viol else 1.0
        ci = clopper_pearson_zero_success(n_viol) if valid_above == 0 else (float("nan"), float("nan"))
        lines.append(f"## {group}\n\n")
        lines.append(f"- rows: {len(group_rows)}\n")
        lines.append(f"- raw Coh_pi vs 0.5: below={raw_below}, at={raw_at}, above={raw_above}\n")
        lines.append(f"- Coh_pi_valid vs 1.0: below={valid_below}, at={valid_at},
            above={valid_above}\n")
        lines.append(f"- exact binomial one-sided p-value (above fewer than half among valid-score
            violations): {one_sided:.6g}\n")
        lines.append(f"- exact binomial two-sided p-value: {two_sided:.6g}\n")
        if n_viol:
            lines.append(f"- 95% Clopper-Pearson CI for P(above | violation), if above=0: [{ci[0]:.4f},
                {ci[1]:.4f}]\n")
        lines.append("\n")

    lines.append("## By piece\n\n")
    lines.append("piece,group,N,raw_below,raw_at,raw_above,valid_below,valid_at,valid_above,mean_raw,mean_valid\n")
    for piece, prs in sorted(by_piece.items()):
        raw_vals = [float(r["coh_pi_reported"]) for r in prs]
        valid_vals = [float(r["coh_pi_valid"]) for r in prs]
        rb, ra, rup = classify(raw_vals, 0.5, args.eps)
        vb, va, vup = classify(valid_vals, 1.0, args.eps)
        lines.append(
            f"{piece},{group_for(piece)},{len(prs)},{rb},{ra},{rup},{vb},{va},{vup},"
            f"{mean(raw_vals):.6f},{mean(valid_vals):.6f}\n"
        )

    out.write_text("".join(lines))


if __name__ == "__main__":
    main()
```

A.5 Ejecutables de línea de comandos (`cli/`)

A.5.1 `src/padicmidi/cli/main.py` — ejecutable unificado

```
#!/usr/bin/env python3
"""
padicmidi.cli.main — unified ``padicmidi`` command-line entry point.

Dispatches to the specialised sub-commands (run-one, run-suite, benchmark,
job-list, mutopia). This is the canonical executable advertised by the
manuals: ``padicmidi <subcommand> [options]``.
"""

from __future__ import annotations

import sys

from padicmidi.cli import benchmark, job_list, mutopia, run_one, run_suite

SUBCOMMANDS = {
```

```
    "run-one":   run_one.main,
    "run-suite": run_suite.main,
    "benchmark": benchmark.main,
    "job-list":  job_list.main,
    "mutopia":   mutopia.main,
}

USAGE = """\
padicmidi v1.0.0 — p-adic hierarchical analysis of symbolic music
Author: J. Rogelio Pérez-Buendía (CIMAT-Mérida) · ORCID 0000-0002-7739-4779

Usage:
  padicmidi <subcommand> [arguments...]
  padicmidi --help
  padicmidi <subcommand> --help

Sub-commands:
  run-one    Analyse a single MIDI file (one piece, one prime).
  run-suite  Analyse a corpus across multiple primes and axes.
  benchmark  Run the canonical BWV 1007 benchmark.
  job-list   Generate a job list for a corpus.
  mutopia    Download CC-licensed MIDI files from Mutopia.

For details on each sub-command, run:
  padicmidi <subcommand> --help

Web demo (no installation needed):
  Open `web-demo/applet.html` (English) or
       `web-demo/applet-es.html` (Spanish) in any modern browser.
"""


def main(argv: list[str] | None = None) -> int:
    argv = sys.argv[1:] if argv is None else list(argv)
    if not argv or argv[0] in ("-h", "--help", "help"):
        print(USAGE)
        return 0
    sub = argv[0]
    if sub not in SUBCOMMANDS:
        print(f"padicmidi: unknown sub-command '{sub}'\n", file=sys.stderr)
        print(USAGE, file=sys.stderr)
        return 2
    sys.argv = [f"padicmidi {sub}"] + argv[1:]
    rc = SUBCOMMANDS[sub]()
    return int(rc) if rc is not None else 0


if __name__ == "__main__":
    raise SystemExit(main())
```

A.5.2 `src/padicmidi/cli/run_one.py`

```
#!/usr/bin/env python3
"""
padicmidi.cli.run_one — analyse a single MIDI file (entry point: ``padicmidi-run-one``).

```

```
Wrapper around :mod:`padicmidi.core.hierarchical` that mirrors the
historical ``run_one_piece.py`` script but with two improvements:

* Calls the motor in-process (no ``subprocess``).
* Exposes the random seed and all hyper-parameters via the CLI.

Examples
--------
::

    padicmidi-run-one bwv1007-1.mid bwv1007_pre beats 2 5 \\
        --out results_local/bwv1007_pre/beats/p2

    padicmidi-run-one bwv1007-1.mid bwv1007_pre beats 3 4 \\
        --K 16 --Kchild 2 --M 800 --step 2 --seed 42 \\
        --out results_local/bwv1007_pre/beats/p3
"""

from __future__ import annotations

import argparse
import csv
import json
import sys
from pathlib import Path

import numpy as np

from padicmidi import __version__
from padicmidi.core.config import (
    DEFAULT_BIN_BEATS,
    DEFAULT_BIN_SECONDS,
    DEFAULT_K,
    DEFAULT_KCHILD,
    DEFAULT_M,
    DEFAULT_SEED,
    DEFAULT_STEP,
    SUPPORTED_PRIMES,
    default_nmax,
)
from padicmidi.core.hierarchical import (
    build_X_beats,
    build_X_seconds,
    run_hierarchical,
)


def _build_argparser() -> argparse.ArgumentParser:
    ap = argparse.ArgumentParser(
        prog="padicmidi-run-one",
        description=(
            "Analyse a single MIDI file with the p-adic hierarchical motor "
            "and write coherence/audit/prototype CSVs to an output folder."
        ),
    )
    ap.add_argument("midi", help="Path to the MIDI file (absolute or relative).")
    ap.add_argument("piece", help="Short identifier for the piece (used in output folder name).")
    ap.add_argument("axis", choices=["beats", "seconds"], help="Time axis.")
```

```
    ap.add_argument("p", type=int, help=f"Prime in {SUPPORTED_PRIMES}.")
    ap.add_argument("Nmax", type=int, nargs="?", default=None, help="Maximum tower level (default depends on p).")
    ap.add_argument("--out", required=True, help="Output directory.")
    ap.add_argument("--K", type=int, default=DEFAULT_K, help=f"K for K-means (default {DEFAULT_K}).")
    ap.add_argument("--Kchild", type=int, default=DEFAULT_KCHILD, help=f"Children per parent (default {DEFAULT_KCHILD}).")
    ap.add_argument("--M", type=int, default=DEFAULT_M, help=f"Subsample size for K-means (default {DEFAULT_M}).")
    ap.add_argument("--step", type=int, default=DEFAULT_STEP, help=f"Window step (default {DEFAULT_STEP}).")
    ap.add_argument("--seed", type=int, default=DEFAULT_SEED, help=f"Random seed (default {DEFAULT_SEED}).")
    ap.add_argument("--bin-beats", type=float, default=DEFAULT_BIN_BEATS, help=f"Bin size (beats axis, default {DEFAULT_BIN_BEATS:.6f}).")
    ap.add_argument("--bin", type=float, default=DEFAULT_BIN_SECONDS, help=f"Bin size (seconds axis, default {DEFAULT_BIN_SECONDS}).")
    ap.add_argument("--version", action="version", version=f"padicmidi {__version__}")
    return ap


def main(argv: list[str] | None = None) -> int:
    args = _build_argparser().parse_args(argv)

    if args.p not in SUPPORTED_PRIMES:
        print(f"error: --p must be one of {SUPPORTED_PRIMES}; got {args.p!r}", file=sys.stderr)
        return 2
    Nmax = args.Nmax if args.Nmax is not None else default_nmax(args.p)

    midi_path = Path(args.midi).expanduser().resolve()
    if not midi_path.exists():
        print(f"error: MIDI file not found: {midi_path}", file=sys.stderr)
        return 2

    out_dir = Path(args.out).expanduser().resolve()
    out_dir.mkdir(parents=True, exist_ok=True)

    if args.axis == "seconds":
        X = build_X_seconds(str(midi_path), bin_size=args.bin)
    else:
        X = build_X_beats(str(midi_path), bin_size_beats=args.bin_beats)

    if len(X) < 2:
        print("error: series too short (less than 2 bins)", file=sys.stderr)
        return 1

    rng = np.random.default_rng(args.seed)
    prototypes_n, f_n, pi_maps, coherence_rows, audit_rows = run_hierarchical(
        X, args.p, Nmax, args.step, args.K, args.Kchild, args.M, rng
    )

    p = args.p
    params = {
        "piece": args.piece,
        "midi": str(midi_path),
        "axis": args.axis,
        "p": p,
        "Nmax": Nmax,
```

```
        "K": args.K,
        "Kchild": args.Kchild,
        "M": args.M,
        "step": args.step,
        "seed": args.seed,
        "bin_seconds": args.bin,
        "bin_beats": args.bin_beats,
        "padicmidi_version": __version__,
    }
    (out_dir / "params.json").write_text(json.dumps(params, indent=2))
    (out_dir / "params.txt").write_text(
        " ".join(f"{k}={v}" for k, v in params.items()) + "\n"
    )

    for n, prot in prototypes_n.items():
        flat = prot.reshape(prot.shape[0], -1)
        np.savetxt(out_dir / f"S_n_prototypes_p{p}_n{n}.csv", flat, delimiter=",")
    for n, pi_list in pi_maps.items():
        with open(out_dir / f"pi_p{p}_n{n+1}_to_n{n}.csv", "w", newline="") as fh:
            w = csv.writer(fh)
            w.writerow(["child_id", "parent_id"])
            for cid, pid in enumerate(pi_list):
                w.writerow([cid, pid])
    for n, fn in f_n.items():
        with open(out_dir / f"f_p{p}_n{n}.csv", "w", newline="") as fh:
            w = csv.writer(fh)
            w.writerow(["residue", "prototype_id"])
            for a in sorted(fn.keys()):
                w.writerow([a, fn[a]])

    with open(out_dir / f"coherence_hier_p{p}.csv", "w", newline="") as fh:
        w = csv.DictWriter(fh, fieldnames=["n", "Coh_pi", "Coh_grid", "n_samples_n",
            "n_samples_nplus1"])
        w.writeheader()
        w.writerows(coherence_rows)

    audit_fields = [
        "n", "parent_class", "n_valid_siblings", "excluded_sparsity",
        "V_SC_pi", "AI_pi", "coherent_count_pi",
        "V_SC_trunc", "AI_trunc", "coherent_count_trunc",
        "parent_prototype",
    ]
    with open(out_dir / f"audit_p{p}.csv", "w", newline="") as fh:
        w = csv.DictWriter(fh, fieldnames=audit_fields)
        w.writeheader()
        w.writerows(audit_rows)

    print(
        f"[padicmidi-run-one] piece={args.piece} axis={args.axis} p={p} Nmax={Nmax}: "
        f"wrote {out_dir} ({len(coherence_rows)} coherence rows, {len(audit_rows)} audit rows).",
        flush=True,
    )
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

A.5.3 `src/padicmidi/cli/run_suite.py`

```python
#!/usr/bin/env python3
"""
padicmidi.cli.run_suite — Phase I run on a suite of MIDIs (entry point: ``padicmidi-run-suite``).

Runs the Phase I (echo) motor on each MIDI in a folder and writes
results under a configurable output directory. By default analyses the
six BWV 1007 movements bundled in ``data/midi/`` of the package.

This is a thin orchestration wrapper around :mod:`padicmidi.core.echo`,
invoking it as a subprocess to keep CLI argument parsing isolated.
"""

from __future__ import annotations

import argparse
import subprocess
import sys
from pathlib import Path

from padicmidi import __version__

DEFAULT_PIECES: list[tuple[str, str]] = [
    ("1pre", "cs1-1pre.mid"),
    ("2all", "cs1-2all.mid"),
    ("3cou", "cs1-3cou.mid"),
    ("4sar", "cs1-4sar.mid"),
    ("5men", "cs1-5men.mid"),
    ("6gig", "cs1-6gig.mid"),
]


def _build_argparser() -> argparse.ArgumentParser:
    ap = argparse.ArgumentParser(
        prog="padicmidi-run-suite",
        description="Run Phase I on a suite of MIDI files (defaults to the BWV 1007 movements).",
    )
    ap.add_argument(
        "--midi-dir",
        type=Path,
        required=True,
        help="Directory containing the MIDI files referenced by --pieces (or default cs1-* set).",
    )
    ap.add_argument(
        "--out-dir",
        type=Path,
        required=True,
        help="Output directory; subfolders 'seconds/' and 'beats/' will be created.",
    )
    ap.add_argument(
        "--pieces",
        nargs="*",
        metavar="SHORTNAME=FILENAME",
        help="Override the default list (e.g. 1pre=cs1-1pre.mid 2all=cs1-2all.mid). Default uses the
            BWV 1007 set.",
    )
    ap.add_argument("--cap", type=int, default=300, help="Window cap (default 300, paper default).")
```

```
    ap.add_argument("--bin-beats", type=float, default=0.083333)
    ap.add_argument("--axes", nargs="+", choices=["seconds", "beats"], default=["seconds", "beats"])
    ap.add_argument("--version", action="version", version=f"padicmidi {__version__}")
    return ap


def _parse_pieces(spec: list[str] | None) -> list[tuple[str, str]]:
    if not spec:
        return DEFAULT_PIECES
    out: list[tuple[str, str]] = []
    for item in spec:
        if "=" not in item:
            raise SystemExit(f"--pieces entry must be SHORTNAME=FILENAME; got {item!r}")
        short, name = item.split("=", 1)
        out.append((short.strip(), name.strip()))
    return out


def main(argv: list[str] | None = None) -> int:
    args = _build_argparser().parse_args(argv)
    pieces = _parse_pieces(args.pieces)

    midi_dir = args.midi_dir.expanduser().resolve()
    if not midi_dir.is_dir():
        print(f"error: --midi-dir does not exist: {midi_dir}", file=sys.stderr)
        return 2

    args.out_dir.mkdir(parents=True, exist_ok=True)
    out_seconds = args.out_dir / "seconds"
    out_beats = args.out_dir / "beats"
    out_seconds.mkdir(exist_ok=True)
    out_beats.mkdir(exist_ok=True)

    for short, name in pieces:
        midi_path = midi_dir / name
        if not midi_path.exists():
            print(f"[run-suite] skipping missing MIDI: {midi_path}", file=sys.stderr)
            continue
        for axis in args.axes:
            out_dir = out_seconds if axis == "seconds" else out_beats
            prefix = str(out_dir) + f"/cs1_{short}_cap{args.cap}_"
            cmd = [
                sys.executable,
                "-m", "padicmidi.core.echo",
                str(midi_path),
                "--phase1-only",
                "--save", prefix,
                "--cap", str(args.cap),
            ]
            if axis == "beats":
                cmd += ["--time-axis", "beats", "--bin-beats", str(args.bin_beats)]
            print("[run-suite]", " ".join(cmd))
            ret = subprocess.run(cmd)
            if ret.returncode != 0:
                print(f"[run-suite] failed on {short}/{axis}", file=sys.stderr)
                return ret.returncode
    print("[run-suite] finished.")
    return 0
```

```

if __name__ == "__main__":
    sys.exit(main())
```

A.5.4 `src/padicmidi/cli/benchmark.py`

```
#!/usr/bin/env python3
"""
padicmidi.cli.benchmark — Phase I benchmark across the bundled benchmark set.

Entry point: ``padicmidi-benchmark``. Runs the Phase I motor for each
piece in the benchmark set (BWV 1007 movements + binary/ternary toys)
and writes prefixed outputs under the user-supplied output directory.

Defaults reproduce the gold-standard runs of the companion papers.
"""

from __future__ import annotations

import argparse
import subprocess
import sys
from pathlib import Path

from padicmidi import __version__

BENCHMARK_PIECES: list[tuple[str, str]] = [
    ("cs1_1pre", "cs1-1pre.mid"),
    ("cs1_2all", "cs1-2all.mid"),
    ("cs1_3cou", "cs1-3cou.mid"),
    ("cs1_4sar", "cs1-4sar.mid"),
    ("cs1_5men", "cs1-5men.mid"),
    ("cs1_6gig", "cs1-6gig.mid"),
    ("toy_binary", "toy_binary.mid"),
    ("toy_ternary", "toy_ternary.mid"),
]


def main(argv: list[str] | None = None) -> int:
    ap = argparse.ArgumentParser(
        prog="padicmidi-benchmark",
        description="Run the Phase I benchmark for all pieces in the BWV 1007 + toys set.",
    )
    ap.add_argument(
        "--midi-dir",
        type=Path,
        required=True,
        help="Directory containing the benchmark MIDI files.",
    )
    ap.add_argument(
        "--out-dir",
        type=Path,
        required=True,
        help="Output directory; subfolders 'seconds/' and 'beats/' will be created.",
    )
```

```
    ap.add_argument("--seconds-only", action="store_true", help="Only run seconds axis.")
    ap.add_argument("--beats-only", action="store_true", help="Only run beats axis.")
    ap.add_argument("--dry-run", action="store_true", help="Print commands only.")
    ap.add_argument("--version", action="version", version=f"padicmidi {__version__}")
    args = ap.parse_args(argv)

    midi_dir = args.midi_dir.expanduser().resolve()
    if not midi_dir.is_dir():
        print(f"error: --midi-dir does not exist: {midi_dir}", file=sys.stderr)
        return 2

    args.out_dir.mkdir(parents=True, exist_ok=True)
    out_seconds = args.out_dir / "seconds"
    out_beats = args.out_dir / "beats"

    run_seconds = not args.beats_only
    run_beats = not args.seconds_only

    for piece, midi_name in BENCHMARK_PIECES:
        midi_path = midi_dir / midi_name
        if not midi_path.exists():
            print(f"[benchmark] skip (missing): {midi_name}", file=sys.stderr)
            continue

        if run_seconds:
            out_seconds.mkdir(parents=True, exist_ok=True)
            prefix = str(out_seconds) + f"/{piece}_"
            cmd = [
                sys.executable, "-m", "padicmidi.core.echo",
                str(midi_path),
                "--phase1-only",
                "--save", prefix,
            ]
            print(" ".join(cmd))
            if not args.dry_run:
                ret = subprocess.run(cmd)
                if ret.returncode != 0:
                    print(f"[benchmark] failed on {piece}/seconds", file=sys.stderr)
                    return ret.returncode

        if run_beats:
            out_beats.mkdir(parents=True, exist_ok=True)
            prefix = str(out_beats) + f"/{piece}_"
            cmd = [
                sys.executable, "-m", "padicmidi.core.echo",
                str(midi_path),
                "--phase1-only",
                "--save", prefix,
                "--time-axis", "beats",
                "--bin-beats", "0.083333",
            ]
            print(" ".join(cmd))
            if not args.dry_run:
                ret = subprocess.run(cmd)
                if ret.returncode != 0:
                    print(f"[benchmark] failed on {piece}/beats", file=sys.stderr)
                    return ret.returncode

```

```
    print("[benchmark] finished.")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

A.5.5 `src/padicmidi/cli/job_list.py`

```
#!/usr/bin/env python3
"""
padicmidi.cli.job_list — generate a job manifest for the bundled corpus.

Entry point: ``padicmidi-job-list``. Writes a file ``job_list.txt`` in
the current working directory with one line per job in the form

    <midi> <piece_id> <axis> <p> <Nmax>

ready to be consumed by SLURM array jobs, GNU parallel, or hand
piped through ``padicmidi-run-one``.
"""

import argparse
import sys

from padicmidi import __version__

# Movimientos por suite:
#  1=Prelude, 2=Allemande, 3=Courante, 4=Sarabande, 5=Menuet/Gavotte, 6=Gigue

PIECES = [
    # ── BWV 1007 Mutopia (LilyPond 2018, Public Domain) ──────────────
    # Fuente canónica: un solo tempo por movimiento, sin artefactos
    ("bwv1007-1.mid", "bwv1007_pre"),   # Prelude   2/2  70 BPM
    ("bwv1007-2.mid", "bwv1007_all"),   # Allemande 3/4  80 BPM
    ("bwv1007-3.mid", "bwv1007_cou"),   # Courante  3/4  60 BPM
    ("bwv1007-4.mid", "bwv1007_sar"),   # Sarabande 3/4 120 BPM
    ("bwv1007-5.mid", "bwv1007_men"),   # Menuet    3/4 120 BPM
    ("bwv1007-6.mid", "bwv1007_gig"),   # Gigue     6/8 120 BPM ← experimento clave p=2 vs p=3

    # ── BWV 1007 Grossman 1997 (beats-axis only) ─────────────────────
    # Incluir para replicabilidad; NO usar eje de segundos (artefactos de tempo)
    ("cs1-1pre.mid", "cs1_pre"),
    ("cs1-2all.mid", "cs1_all"),
    ("cs1-3cou.mid", "cs1_cou"),
    ("cs1-4sar.mid", "cs1_sar"),
    ("cs1-5men.mid", "cs1_men"),
    ("cs1-6gig.mid", "cs1_gig"),

    # ── BWV 1008 Mutopia (Suite 2 Re menor, CC-BY-SA 3.0) ────────────
    ("bwv1008-1.mid", "bwv1008_pre"),   # Prelude
    ("bwv1008-2.mid", "bwv1008_all"),   # Allemande
    ("bwv1008-3.mid", "bwv1008_cou"),   # Courante
    ("bwv1008-4.mid", "bwv1008_sar"),   # Sarabande
    ("bwv1008-5.mid", "bwv1008_men"),   # Menuet
    ("bwv1008-6.mid", "bwv1008_gig"),   # Gigue
```

```python

    # ── BWV 1009 Mutopia (Suite 3 Do mayor, cellosuite3) ──────────────
    # ZIP contiene 5 movimientos (sin Gigue separado)
    ("cellosuite3-1.mid", "bwv1009_pre"),
    ("cellosuite3-2.mid", "bwv1009_all"),
    ("cellosuite3-3.mid", "bwv1009_cou"),
    ("cellosuite3-4.mid", "bwv1009_sar"),
    ("cellosuite3-5.mid", "bwv1009_men"),

    # ── Toys ──────────────────────────────────────────────────────────
    ("toy_binary.mid",  "toy_binary"),
    ("toy_ternary.mid", "toy_ternary"),
]

# Ejes y primos
AXES = ["beats", "seconds"]
PRIMES = {2: 6, 3: 5, 5: 4, 7: 3}  # prime: Nmax

# Para cs1_* restringir a beats (artefacto de tempo en seconds)
CS1_PIECES = {p for _, p in PIECES if p.startswith("cs1_")}

def main(argv: list[str] | None = None) -> int:
    ap = argparse.ArgumentParser(
        prog="padicmidi-job-list",
        description="Generate a job manifest (job_list.txt) for the bundled MIDI corpus.",
    )
    ap.add_argument("--out", default="job_list.txt", help="Output file (default job_list.txt).")
    ap.add_argument("--version", action="version", version=f"padicmidi {__version__}")
    args = ap.parse_args(argv)

    jobs = []
    for midi, piece in PIECES:
        axes = ["beats"] if piece in CS1_PIECES else AXES
        for axis in axes:
            for p, Nmax in PRIMES.items():
                jobs.append(f"{midi} {piece} {axis} {p} {Nmax}")

    with open(args.out, "w") as f:
        f.write("\n".join(jobs) + "\n")

    print(f"{len(jobs)} jobs written to {args.out}")
    print(f"  BWV1007 Mutopia (2 axes): 6 × 2 × 4 = {6*2*4} jobs")
    print(f"  BWV1007 Grossman (beats): 6 × 1 × 4 = {6*1*4} jobs")
    print(f"  BWV1008 Mutopia (2 axes): 6 × 2 × 4 = {6*2*4} jobs")
    print(f"  BWV1009 Mutopia (2 axes): 5 × 2 × 4 = {5*2*4} jobs")
    print(f"  Toys (2 axes):            2 × 2 × 4 = {2*2*4} jobs")
    print(f"  TOTAL: {len(jobs)}")
    print(f"\nFirst jobs:")
    for j in jobs[:6]:
        print(f"  {j}")
    print("  ...")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

A.5.6 `src/padicmidi/cli/mutopia.py`

```python
#!/usr/bin/env python3
"""
padicmidi.cli.mutopia — download CC-licensed MIDIs from the Mutopia project.

Entry point: ``padicmidi-mutopia``. Downloads ZIPs from
``mutopiaproject.org`` and extracts the BWV 1007/1008/1009 MIDI files
to a destination folder (defaults to the current working directory).
"""

from __future__ import annotations

import argparse
import io
import os
import sys
import urllib.request
import zipfile
from pathlib import Path

from padicmidi import __version__

ZIPS: list[tuple[str, str]] = [
    ("BWV1007", "https://www.mutopiaproject.org/ftp/BachJS/BWV1007/bwv1007/bwv1007-mids.zip"),
    ("BWV1008", "https://www.mutopiaproject.org/ftp/BachJS/BWV1008/bwv1008/bwv1008-mids.zip"),
    ("BWV1009", "https://www.mutopiaproject.org/ftp/BachJS/BWV1009/cellosuite3/cellosuite3-mids.zip"),
]

SKIP = ["viola", "bwv1007.mid", "cellosuite3.mid", "bwv1008.mid"]

USER_AGENT = "Mozilla/5.0 (academic research; padicmidi/{})".format(__version__)


def main(argv: list[str] | None = None) -> int:
    ap = argparse.ArgumentParser(
        prog="padicmidi-mutopia",
        description="Download Bach Cello Suites MIDIs from the Mutopia Project (CC-licensed).",
    )
    ap.add_argument(
        "--out",
        type=Path,
        default=Path("."),
        help="Destination directory (default: current directory).",
    )
    ap.add_argument("--timeout", type=int, default=30, help="HTTP timeout in seconds (default 30).")
    ap.add_argument("--version", action="version", version=f"padicmidi {__version__}")
    args = ap.parse_args(argv)

    dest = args.out.expanduser().resolve()
    dest.mkdir(parents=True, exist_ok=True)
    print(f"Destination: {dest}\n")

    total = 0
    headers = {"User-Agent": USER_AGENT}
    for label, url in ZIPS:
        print(f"-> {label}: {url.split('/')[-1]}", flush=True)
        try:
```

```
            req = urllib.request.Request(url, headers=headers)
            data = urllib.request.urlopen(req, timeout=args.timeout).read()
            print(f"   ZIP downloaded: {len(data) // 1024} KB")
            with zipfile.ZipFile(io.BytesIO(data)) as zf:
                for name in sorted(zf.namelist()):
                    if not name.lower().endswith(".mid"):
                        continue
                    base = os.path.basename(name)
                    if any(s in base for s in SKIP):
                        print(f"   skip: {base}")
                        continue
                    out_path = dest / base
                    with zf.open(name) as src, open(out_path, "wb") as dst:
                        dst.write(src.read())
                    print(f"   OK: {base} ({out_path.stat().st_size} bytes)")
                    total += 1
        except Exception as exc:
            print(f"   FAILED: {exc}", file=sys.stderr)

    print(f"\nNew MIDIs: {total}")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

A.6 Configuración del paquete

A.6.1 `pyproject.toml`

```
[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "padicmidi"
version = "1.0.0"
description = "A Python Toolkit for Hierarchical, Ultrametric, and p-adic Analysis of Symbolic Music Data"
readme = "README.md"
requires-python = ">=3.10"
license = { file = "LICENSE" }
authors = [
    { name = "J. Rogelio Pérez-Buendía", email = "rogelio@cimat.mx" }
]
keywords = [
    "p-adic",
    "music information retrieval",
    "symbolic music",
    "MIDI",
    "profinite",
    "ultrametric",
    "hierarchical clustering",
    "topological data analysis",
    "arithmetic data analysis"
]
classifiers = [
    "Development Status :: 5 - Production/Stable",
    "Intended Audience :: Science/Research",
    "License :: OSI Approved :: MIT License",
    "Operating System :: OS Independent",
```

```
    "Programming Language :: Python :: 3",
    "Programming Language :: Python :: 3.10",
    "Programming Language :: Python :: 3.11",
    "Programming Language :: Python :: 3.12",
    "Programming Language :: Python :: 3.13",
    "Topic :: Multimedia :: Sound/Audio :: MIDI",
    "Topic :: Scientific/Engineering :: Mathematics",
    "Topic :: Scientific/Engineering :: Information Analysis"
]
dependencies = [
    "numpy>=2.0,<3.0",
    "scipy>=1.13,<2.0",
    "networkx>=3.2,<4.0",
    "scikit-learn>=1.4,<2.0",
    "matplotlib>=3.8,<4.0",
    "mido>=1.3,<2.0",
    "joblib>=1.3,<2.0"
]

[project.optional-dependencies]
pretty = ["pretty_midi>=0.2.10"]
dev = [
    "pytest>=8.0",
    "pytest-cov",
    "ruff",
    "mypy"
]

[project.scripts]
padicmidi             = "padicmidi.cli.main:main"
padicmidi-run-one     = "padicmidi.cli.run_one:main"
padicmidi-run-suite   = "padicmidi.cli.run_suite:main"
padicmidi-benchmark   = "padicmidi.cli.benchmark:main"
padicmidi-job-list    = "padicmidi.cli.job_list:main"
padicmidi-mutopia     = "padicmidi.cli.mutopia:main"

[project.urls]
Homepage      = "https://github.com/yoyontzin/padicmidi"
Documentation = "https://github.com/yoyontzin/padicmidi#readme"
Issues        = "https://github.com/yoyontzin/padicmidi/issues"

[tool.setuptools.packages.find]
where = ["src"]

[tool.pytest.ini_options]
minversion = "8.0"
testpaths = ["tests"]
python_files = ["test_*.py"]
addopts = "-q --strict-markers"

[tool.ruff]
line-length = 110
target-version = "py310"
```

A.6.2 `requirements-pinned.txt`

```
# Pinned to exact versions used by the author for bit-equivalent reproduction.
# Tested on macOS arm64 (M-series), Python 3.13.3.
# Minor numerical drift may occur on other platforms; for distribution see pyproject.toml.

numpy==2.4.3
scipy==1.17.1
networkx==3.6.1
scikit-learn==1.8.0
matplotlib==3.10.8
```

```
mido==1.3.3
pretty_midi==0.2.11
joblib==1.5.3

# Indirect (kept for full reproducibility):
contourpy==1.3.3
cycler==0.12.1
fonttools==4.62.0
importlib_resources==6.5.2
kiwisolver==1.5.0
packaging==26.0
pillow==12.1.1
pyparsing==3.3.2
python-dateutil==2.9.0.post0
six==1.17.0
threadpoolctl==3.6.0
```

A.6.3 `LICENSE` (MIT)

```
MIT License

Copyright (c) 2026 Jesús Rogelio Pérez Buendía
(publishes academically as "J. Rogelio Pérez-Buendía",
 ORCID 0000-0002-7739-4779, https://www.cimat.mx/~rogelio.perez)

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

B Manifiesto SHA-256 del dossier

El siguiente manifiesto enumera todos los archivos del dossier INDAUTOR con su hash SHA-256 individual. Cualquier modificación posterior de cualquiera de estos archivos invalidaría el hash global del ZIP de fuentes. El manifiesto se regenera automáticamente con el script `scripts/make_indautor_dossier.sh`.

```
# Manifest of files in the INDAUTOR dossier — 2026-04-30T06:53:49Z

```
3834c4fdffea1bc845031729c4a123f05538f1a0f7394c5079d3b0f66a066d89 indautor/LEEME-PRIMERO-USB.txt
1d342f04563d0a28a2a051a5eb07b79355184425fed0bc8e9c72e48858ef370a indautor/REPORTE-TECNICO.pdf
d19651adc3e0de90a2f6028313d7be4c4625834ff78b38b51483ed3696ae7e91 indautor/RPDA-03-prefilled.md
f7e0abc0400a6cfe4041a857fd7eea181d95f90c5fb0d5f7579032375e6423ae indautor/VERSION-REGISTRADA.md
698e8303e0f28352441e455d55b7ac1f1ce1f99953b31b8de232350a69a0f253 indautor/carta-poder.md
e3d1b8b32ee6dbcccbd60381bc9151ffa60287bf5f6bf18141a27b92877acf66 indautor/carta-titularidad.md
d4e65cacd50b06f380b83e3be741d5e330009e7c455b499f07adcbe800075161 indautor/codigo-fuente/__init__.py
```
```

```
9e4d37338d0e7db7799be78816027aa11038c86e3dae2fe36de02664717969c5  indautor/codigo-fuente/analysis/__init__.py
a9e6a12be276ea420e71160ab41fc734d6eab52c1c9fe87a9cdf3a2568b8693e  indautor/codigo-fuente/analysis/aggregate.py
7b9dc76d1ecf3b828c0c37c841e905b5c729c073f0b3cf59a05361738d1b66be  indautor/codigo-fuente/analysis/audit.py
790a2e6469b2567f10734fecf99840590865c04277692e28b8d162398f46c348  indautor/codigo-fuente/analysis/directionality.py
1443155897346cb8ab63b08db2dcc12dbb163e7022969f50db6a5e630e33db15  indautor/codigo-fuente/analysis/null_model.py
86958c2563dba963707ab75078f8d2e3739a10caf2fe63590e66e49664cc6c5c  indautor/codigo-fuente/cli/__init__.py
d0779ff228489dc670ace6148dafc7cd32df1f39257f8a8554c56d0734b82054  indautor/codigo-fuente/cli/benchmark.py
d8d532a01fec089f22143f18ad8649103d64f245c661d96ca737b78a576debcc  indautor/codigo-fuente/cli/job_list.py
b90c2e0512a7570cf19e5b78262647b1c7cdb08902dbacc01322b72542fb541a  indautor/codigo-fuente/cli/main.py
3507f4f5851d97a71589cfa84532566a86df269f378e7447f7cef80b74abd607  indautor/codigo-fuente/cli/mutopia.py
3928b8957403ec0041530d9a7aaf5c2bae277705e01300982c91db91ef2d4492  indautor/codigo-fuente/cli/run_one.py
d5ea3e566a383e9d3024dbe5b021a9eabc6c63fceb8783058d9ecd219aabd29e  indautor/codigo-fuente/cli/run_suite.py
d1e2af7c64d1e82bdf6fc062742e5bd7f242896e40b46c66ced636142e4c9e83  indautor/codigo-fuente/core/__init__.py
fbeb4264ecc79cad312880b30d75c2de801c562158061dbd33174f6cf666d1d2  indautor/codigo-fuente/core/config.py
8aa8aee8d258c84e5f2911a99657f35d0fb118c7e0bb830d21e6ad293f2db6c3  indautor/codigo-fuente/core/echo.py
2aa1cba59d2b91626b985c7daf435b00388303de99f793eab7da99457080bd52  indautor/codigo-fuente/core/hierarchical.py
c657cbef8b171dc8dcfc4ea67d48ce8a7ba25b96c98f7666cd5dc738a668c22c  indautor/codigo-fuente/figs/__init__.py
fe17e1c8e76d53801a736301f4c1dd68aba2b7a273c170f44a06e02f60139cbb  indautor/codigo-fuente/figs/hierarchical_figs.py
b70060f3548c73950f175e74292f730c90b2d618801e0d6cbe6dd0d4bbb785cb  indautor/codigo-fuente/figs/ladder_balls.py
2585d8f8110f104e3f41440e672c2158377a8ca25e1aa3a52cf0871daf7295e4  indautor/codigo-fuente/figs/padic_balls.py
34f4a8168cadfce8d0c8649cdd689b23fa6392e16e8705111111815c076b47819  indautor/codigo-fuente/figs/padic_tree.py
d81b4ad04827526e62ec0a0662dcdd3e3572dfb7c41f32955c260772e4cbeeac  indautor/codigo-fuente/figs/spectral_precision.py
16f68e6c801e003fa3256442ca35aae7659dea110613ce1695b5086e1f010536  indautor/codigo-fuente/io/__init__.py
41c37e17d68ed7aa6f007af1ab97e59484be94dbb23834a1081b8a464cfd91f4  indautor/codigo-fuente/io/midi_mido.py
6d92120f2e1180630b7a07c419cd987808f2d7ee8670b5f7860f858b02095eea  indautor/codigo-fuente/io/midi_pretty.py
85e2b0e80eaf548f140d96e97d9ddcdc2ff1402089e8e81e43cf57226a398639  indautor/codigo-fuente/legacy/__init__.py
6fdccbbe9534ce0e4785afe919a93c67dfe674e939452d6d5ce5f85b87a89d56  indautor/codigo-fuente/legacy/
     control_primes_summary_legacy.py
7bae9c823a2e643ccea06b8e2a47872e5d061028cdb3e7ddf7054957c4c7032b  indautor/descripcion-funcional.md
ba144977777ec4a5d1be0310bc4273d0930f8f7ea92710cd90d229a0565cf639  indautor/evidencia-corridas/01_quickstart_run.log
930ab96ec89bb9a83c0ca0abb5ba0d58d41d504883195e93140afa4e6fc8ad2e  indautor/evidencia-corridas/04_test_suite_output.txt
b0474320c7de1cd15b619aaca6bb764e7218156b84b93b549fa1235ba465f54b  indautor/evidencia-corridas/quickstart_p2/
     S_n_prototypes_p2_n1.csv
ddebaa9edd2a5b0ba0046cf4d717fda3079d7dabd0a9f6c652140364934de5c5  indautor/evidencia-corridas/quickstart_p2/
     S_n_prototypes_p2_n2.csv
d6c27da8f84665c8e2ca88f22804e298d53792394f9907bba12bf4c8cb40d7fe  indautor/evidencia-corridas/quickstart_p2/
     S_n_prototypes_p2_n3.csv
9226ac308e7dc4aaff35ebb4a44ff35a7f30764dcda3e29287c1953dee34f63c  indautor/evidencia-corridas/quickstart_p2/
     S_n_prototypes_p2_n4.csv
6504319260d94a1f9fd1cd9412ddd3c63f0cdfcdf4a3de52e94f4649cf4e6f52  indautor/evidencia-corridas/quickstart_p2/
     S_n_prototypes_p2_n5.csv
c6aad984061bfd84f073bfb3c16c57c55a0cc68d761f7e6a19cab92713883a3b  indautor/evidencia-corridas/quickstart_p2/
     S_n_prototypes_p2_n6.csv
4244112a70f1bdafafbdfaedd8b32914fd2046b6243c60a37139abdf2e607da8  indautor/evidencia-corridas/quickstart_p2/audit_p2.csv
c73f944bea8a36dbee16cf8940ac6bdd6c84991695482656723da4b0e14695ba  indautor/evidencia-corridas/quickstart_p2/
     coherence_hier_p2.csv
a438816f46a0ab9a666c3270cbfe802965afa7d0c5acdc6091c17c5036355dd9  indautor/evidencia-corridas/quickstart_p2/f_p2_n1.csv
4e282a2da1bdbab38b1238de5c795a59bfead307dcee3d8c20d7c3ac717bf713  indautor/evidencia-corridas/quickstart_p2/f_p2_n2.csv
0df5c6b5cc5b9b0790d11ce58c38c3e71789da693707c19296dd710a66de2de2  indautor/evidencia-corridas/quickstart_p2/f_p2_n3.csv
c2d8544473132d3aa7698bfee8263b7d699d87ce55d107832f41d61ae31c5ef0  indautor/evidencia-corridas/quickstart_p2/f_p2_n4.csv
fe4b00bad65abea775c9899a6007d15f402b9937c19108de182aca808cd9b6cf  indautor/evidencia-corridas/quickstart_p2/f_p2_n5.csv
106b97fc79fa92f099917d499929acb988ef56bb12838427cc7107543b839e3b  indautor/evidencia-corridas/quickstart_p2/f_p2_n6.csv
a00e4295725b47e1114e12fb9fa204b89dddf9d6f7a69fb3081d361f7dafc83d  indautor/evidencia-corridas/quickstart_p2/params.json
a12d2f48a46ae615a9c267f8588223c7c6a6169708c0a8d3003fa73877d39b65  indautor/evidencia-corridas/quickstart_p2/params.txt
1424d793fd39e7f8ebd7f11e5ebcea92aca61c9e3fe4534b844359906bb752dc  indautor/evidencia-corridas/quickstart_p2/
     pi_p2_n2_to_n1.csv
35ee53a947b55e7ce5d9c7e1b8b5cbee046e5b947b6e300c9c82ad38a13a5612  indautor/evidencia-corridas/quickstart_p2/
     pi_p2_n3_to_n2.csv
d2503331b21c60dcfded94b088654368f1d9cf6f235154bae600638c1b72e27f  indautor/evidencia-corridas/quickstart_p2/
     pi_p2_n4_to_n3.csv
0b1a288f5766269a3ab3a3302bf6a396d937df0aca7c29a5b801df884eb9e0d9  indautor/evidencia-corridas/quickstart_p2/
     pi_p2_n5_to_n4.csv
57a08b02e2fc9fe667e748826f538f87334a7f7b91bbe16afb771d3c849cc502  indautor/evidencia-corridas/quickstart_p2/
     pi_p2_n6_to_n5.csv
8a3948a7587740b354bed527f35109fe2e1e98fd7b65ddbf9c8efe938a371a7f  indautor/manual-tecnico.md
fb6cf0d531203c6e951320313838c95001ff1fe8a94b66f881f41cbe40dca68c  indautor/manual-usuario.md
5ef5cef140f2d6e8fab95e48c58a5c24118a306d54cad14c78b1b04e8eccb552  indautor/padicmidi-v1.0.0-source.zip
```

```
10a2359cc9bedab4c143fe303b9393765d2b052f8211f52c580433d51e086ae1  indautor/pdf-imprimibles/RPDA-03-prefilled.pdf
15653e5a8a089cd664ee14e191b0a9b763d039015201ef64e2b4f4d91de22e32  indautor/pdf-imprimibles/VERSION-REGISTRADA.pdf
48700448d816175a4865914f6042653a7baaaeeda393709cf7487d1d47e23565  indautor/pdf-imprimibles/carta-poder.pdf
a40c222c9eaa1ae086ec636f7c73c93ec5aa0249fcaca6c6693bf47b68b37678  indautor/pdf-imprimibles/carta-titularidad.pdf
d8efc0af3ecb8ffd7e8d49b6c764ecdf24c9aba3ddbbd048f37e3dd91ee2e02e  indautor/pdf-imprimibles/descripcion-funcional.pdf
964c56809d2b51520de7a5fb2e561d1b798684d71a08f2a00f86fffd75826a06  indautor/pdf-imprimibles/manifiesto-archivos.pdf
215abee212f01b8bc7ec6e8232e78b991f1434c70291bd36d9a904211a6d7c82  indautor/pdf-imprimibles/manual-tecnico.pdf
9c0ece5672fd0489ed7a14162ee1b3103849fba27ab867a6c91cf1968786d542  indautor/pdf-imprimibles/manual-usuario.pdf
```

C Cómo subir el software a la página personal

El paquete está preparado para ser publicado en `http://personal.cimat.mx:8181/~rogelio.perez/desarrollo-tecnologico/` (URL canónica futura: `https://www.cimat.mx/~rogelio.perez/desarrollo-tecnologico/`). La carpeta `web-site/` contiene exactamente cuatro archivos HTML autocontenidos listos para subir tal cual:

```
web-site/├──
 index.html       Landing en español (página principal)├──
 index.en.html    Landing en inglés├──
 applet.html      Applet de análisis (inglés)└──
 applet-es.html   Applet de análisis (español)
```

Vía SSH / SCP.

```
# Subir los cuatro archivos HTML al servidor:
scp web-site/*.html rogelio@personal.cimat.mx:~/public_html/desarrollo-tecnologico/padicmidi/

# O bien por rsync:
rsync -av web-site/ rogelio@personal.cimat.mx:~/public_html/desarrollo-tecnologico/padicmidi/
```

Vía sitio Hugo (Hugo Blox). Si la página personal del autor está construida con Hugo, copiar los applets a `static/`:

```
cp web-demo/applet.html web-demo/applet-es.html \
   ~/sitio-hugo/static/desarrollo-tecnologico/padicmidi/
```

y enlazar desde la página de *Desarrollo de software* con un `<iframe>` o un botón *"Abrir la demo web"*, en línea con el patrón ya empleado para *p-adic GRN Suite*.

Aviso legal sugerido para la página de descarga.

> ***Derechos de autor (obra software).*** *© 2026 Jesús Rogelio Pérez Buendía. Derechos de autor en trámite ante el INDAUTOR (formato RPDA-03), con base en los artículos 13 fracción XI y 24 a 26 de la Ley Federal del Derecho de Autor. El código fuente se distribuye bajo* licencia MIT*; la documentación y los resultados verificados, bajo* Creative Commons Attribution 4.0 Internacional *(CC-BY 4.0). Se reservan los derechos morales reconocidos por los artículos 18 a 23 de la LFDA.*

D Resumen de archivos del dossier INDAUTOR

Archivo	Contenido
`indautor/RPDA-03-prefilled.md`	Pre-llenado del formato oficial
`indautor/carta-titularidad.md`	Carta de titularidad firmada
`indautor/descripcion-funcional.md`	Descripción funcional ampliada
`indautor/VERSION-REGISTRADA.md`	Identificación de la versión y hash global
`indautor/manifiesto-archivos.md`	Hashes SHA-256 individuales
`indautor/manual-usuario.md`	Manual de usuario (copia)
`indautor/manual-tecnico.md`	Manual técnico (copia)
`indautor/codigo-fuente/`	Copia del paquete `src/padicmidi/`
`indautor/evidencia-corridas/`	Logs de tests y quickstart con CSV resultantes
`indautor/padicmidi-v1.0.0-source.zip`	Paquete fuente firmado por hash
`indautor/REPORTE-TECNICO.pdf`	Este documento

Referencias

[1] Pérez-Buendía, J.~R., *Prime-power indexed multiscale graph diagnostics for symbolic temporal data: methodological exploration and delimitation via BWV~1007*, sometido a *Journal of Mathematics and Music* (Taylor & Francis), 2026.

[2] Pérez-Buendía, J.~R., *Profinite hierarchical patterns and prime-indexed multiscale invariants in symbolic music*, sometido, 2026.